

MEDIATEK

MT8668 DSI Bring Up SOP

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	柴莹	正式版

目录

版本记录.....2

目录.....3

表格目录.....6

1 概述.....7

1.1 目的..... 7

1.2 DSI HW 架构 7

1.3 DSI Interface 能力 7

1.4 DSI 屏幕 Timing 要求..... 7

2 缩略词.....8

2.1 缩略词..... 8

3 DSI Android Porting 指南.....9

3.1 Android SW 架构..... 9

3.2 Android 通用文件路径..... 9

3.3 LK2 Driver Porting 10

3.3.1 LK2 文件架构..... 10

3.3.2 MIPI Panel Driver -jd9365da_wxga_dsi_vdo.c..... 10

3.3.3 SerDes Panel Driver - serdes_dsi_vdo.c..... 10

3.3.4 DTS 文件介绍 11

3.3.5 DTS 参数介绍 11

3.3.6 如何新增一个 LK 的 Driver 17

3.3.7 LK Driver Function 介绍..... 18

3.3.8 LK 参数 get_params 介绍..... 19

3.3.9 DSI0 BootLogo 设定..... 23

3.3.10 DWS 设定..... 23

3.4 Kernel Driver Porting..... 25

3.4.1 Kernel Driver 文件路径 25

3.4.2 Kernel Driver 文件参考 25

3.4.3 DTS 文件介绍 26

3.4.4 DTS 参数介绍 26

3.4.5 如何添加一个新的 Kernel Driver..... 26

3.4.6 Kernel Driver Function 介绍 28

3.4.7	Kernel 参数介绍	29
3.5	disp_pwm 方式背光控制	34
3.5.1	disp_pwm dws 设定	34
3.5.2	leds dts 设定	35
3.5.3	disp_pwm LK 流程	37
3.5.4	disp_pwm Kernel 流程	38
4	DSI Yocto Porting 指南	39
4.1	Yocto 通用文件路径	39
4.2	LK & Kernel Driver Porting	39
4.2.1	DSI0 BootLogo 设定	39
5	DSI Hypervisor Porting 指南	41
5.1	Hypervisor SW 架构	41
6	DSI Superframe Porting 指南	42
6.1	Superframe 概念	42
6.2	Superframe 架构	42
6.3	Superframe DTS 设定	42
6.3.1	Physical DSI 节点	43
6.3.2	Virtual DSI 节点	43
6.3.3	加串器节点	44
6.3.4	Superframe Setting 节点	45
6.4	如何关闭 Superframe	48
6.5	DSI Superframe Timing	49
6.5.1	Panel to DSI Timing Function	49
6.5.2	Panel to Display Function	50
6.5.3	Log 确认 Timing 信息	50
6.6	Superframe SerDes 控制	50
7	DSI Dual-link Porting 指南	52
7.1	Driver 中要如何配置	52
7.1.1	Physical DSI0 节点	52
7.1.2	Physical DSI1 节点	53
7.1.1	加串器节点	53
7.1.2	Dual-link Setting 节点	54
8	SerDes 调试指南	59
8.1	Ser-max96789 Debug Register	59

8.2	Des-max96752 Debug Register	62
8.3	Debug 流程	62
9	DSI 输出时序控制	64
9.1	LK Boot 流程	64
9.2	Kernel Boot or Resume 流程	65
10	DSI DSC Porting 指南	67
10.1	DSC 介绍	67
10.2	DSC 参数	67
10.3	DSC SW 介绍	69
11	LCM 问题调试	70
11.1	DSI Pattern	70
11.2	DSI Register 介绍	71
11.3	DSI Register Dump 介绍	73
11.4	DSI 调试流程	73
11.5	帧率异常	74
11.6	I2C 通信异常	74
12	DSI DPHY CTS 测试问题	76
12.1	测试仪器	76
12.2	测试平台准备	76
12.3	常见 Failed 项目分析	77
12.3.1	1.3.4/1.4.4 VOD0/VOD1 Pulse & 1.3.7 VCMTX	77
12.3.2	1.3.8 Voltage Mismatch & 1.4.8 VCMTX Mismatch	79
12.3.3	1.3.1~3 & 1.3.13~16 MIPI Timing	79
12.3.4	1.3.11~12 Tr/Tf & 1.5.4 Data to Clock Skew	82
12.3.5	什么情况开 DEM	84
12.3.6	1.5.5 & 1.5.6 HS Skew Calibration Burst	84
13	LCM Timing 介绍	86
13.1	Power On/Off Sequence	86
13.2	Interface Timing	86
13.3	Display Timing	86
14	DSI Interface 介绍	88
14.1	HS	88
14.2	LP	88
14.3	HS & LP 状态切换图	88
14.4	DSI MIPI D-PHY Timing Spec	89
14.4.1	Data Lane Spec	89
14.4.2	Clock Lane Spec	90

14.5 Video Mode 介绍..... 91

 14.5.1 HFP Keep HS..... 91

 14.5.2 DSI_Frame_Rate_Cal_for_Customer 介绍..... 92

15 DSI 波形量测.....94

 15.1 如何测量帧率..... 94

 15.2 如何测量信号..... 95

16 DSI 屏幕评估.....97

附件一 附加条款.....98

表格目录

表 2-1. 缩略词8

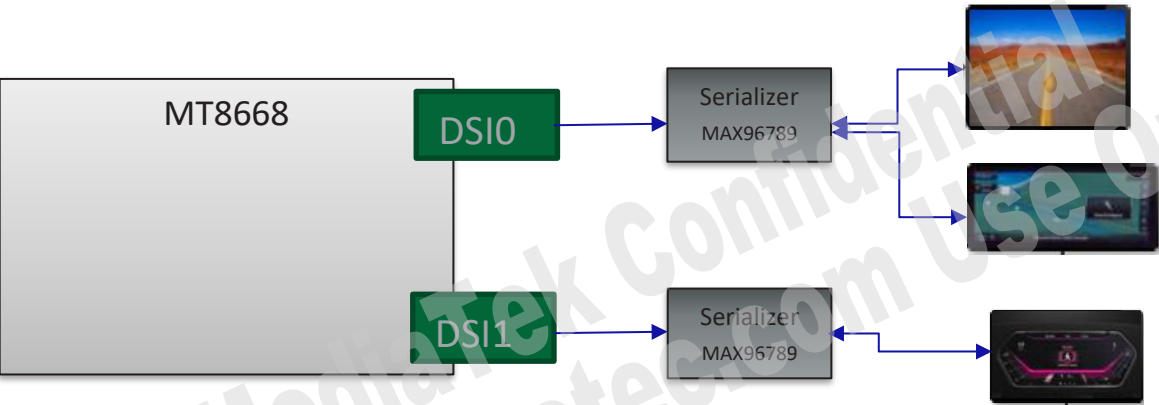
1 概述

1.1 目的

本文档主要介绍 MT8668 DSI Interface 屏幕显示的 Driver Porting 和 Issue Debug 的方法。

1.2 DSI HW 架构

MT8668 总共有两路 DSI Port，公版搭配加串器 max96789 实现单屏或者多屏显示。可以实现不等高非对称的超级帧显示。



1.3 DSI Interface 能力

Port	DSI0	DSI1
MT8668	4 lanes/3 trios DPHY: 3.3Gbps/lane CPHY: 2.88Gbps/trio	4 lanes/3 trios DPHY: 3.3Gbps/lane CPHY: 2.88Gbps/trio

1.4 DSI 屏幕 Timing 要求

具体请参考章节 14.5.2。Display 对屏幕的 timing 有要求，请提供屏幕的详细 Timing 给 MTK 评估。如果已经做过评估，请忽略。

2 缩略词

2.1 缩略词

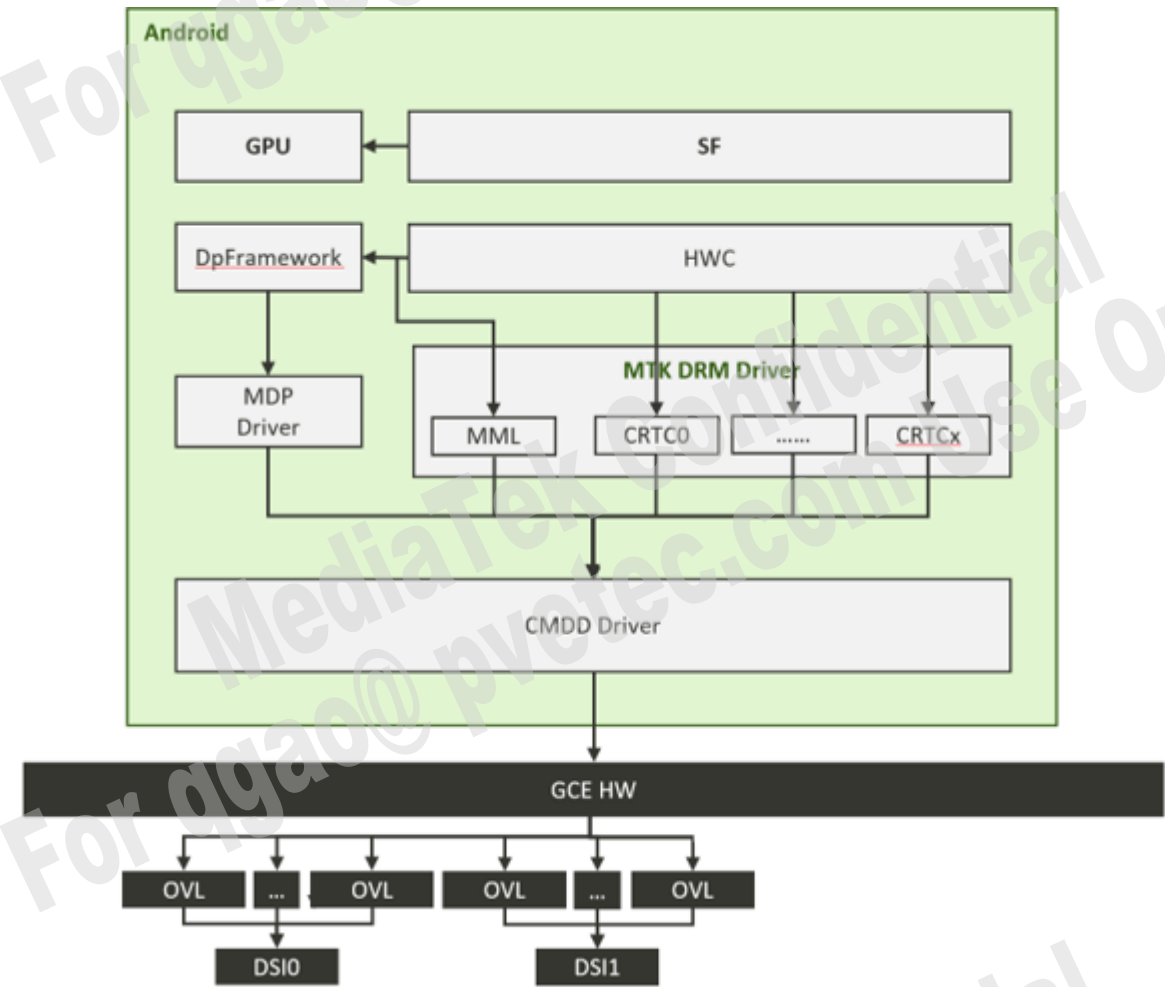
表 2-1. 缩略词

缩略词	解释
DPI	Display Pixel Interface
DSI	Display Serial Interface
HBP	Horizontal Back Porch
HFP	Horizontal Front Porch
HS	High speed
Htotal	Horizontal total pixels
JEIDA	Japan Electronic Industry Development Association
LP	Low Power
LVDS	Low-Voltage Differential Signaling
MIPI	Mobile Industry Processor Interface
RX	Receive
TX	Transmit
UI	Unit Interval
VBP	Vertical Back Porch
VESA	Video Electronics Standards Association
VFP	Vertical Front Porch
Vtotal	Vertical Total Lines
Serdes	Serializer+Deserializer
Ser	Serializer
Des	Deserializer

3 DSI Android Porting 指南

System	Branch	Kernel Version
Android B	alps-mp-b0.mp1	Kernel-6.12

3.1 Android SW 架构



3.2 Android 通用文件路径

文件	路径
LK2	vendor\mediatek\proprietary\bootable\bootloader\lk2
Android Kernel	Kernel\kernel_device_modules-6.12
Project	Device\mediateksample\auto8668p1_64

文件	路径
dws	Vendor\mediatek\proprietary\tools\dct\dws\mt6881\ auto8668p1_64.dws

3.3 LK2 Driver Porting

LK 部分的 driver 主要用于 boot logo 的显示，DSI0 可以在 LK 显示 boot logo，DSI1 不支持 LK 显示，默认从 Kernel 启动。

3.3.1 LK2 文件架构

橙色区域文件是 Porting Driver 需要修改或者参考的文件。

LCM Driver	dev\lcm\serdes_dsi_vdo\serdes_dsi_vdo.c dev\lcm\jd9365da_wxga_dsi_vdo\jd9365da_wxga_dsi_vdo.c dev\lcm\mt65xx_lcm_list.c dev\lcm\include\lcm_drv.h
LK Project.mk	Project\auto8668p1_64.mk
Kernel DTS	arch\arm64\boot\dts\mediatek\cust_mt8668_display_interface.dtsi arch\arm64\boot\dts\mediatek\cust_mt8668_display_config_main.dtsi arch\arm64\boot\dts\mediatek\cust_mt8668_display_serdes_config.dtsi
Display Driver	Platform\mediatek\mt6881\disp\ddp_dsi.c, disp_lcm.c, primary_display.c, mt_disp_drv.c, ddp_manager.c, ddp_dsc.c, ddp_dither.c...

3.3.2 MIPI Panel Driver -jd9365da_wxga_dsi_vdo.c

如果 Porting MIPI Panel Driver，可以参考这份文件，此 Driver 在公版有点亮 MIPI 屏幕。

3.3.3 SerDes Panel Driver - serdes_dsi_vdo.c

如果 Porting SerDes Panel Driver，可以参考这份文件，Driver 的写法是从 dts Panel 的节点里面 Parser 出对应的 Panel Timing 和 SerDes 的设置。如果沿用这个 Driver，可以把 dts 中的参数更改为要 Bringup 的 SerDes 参数和屏幕 Timing 即可。

3.3.4 DTS 文件介绍

Mt6881.dts	Chip 的 dts 相关设定，DSI 等 Display 根节点会放在这份文件里面，节点默认关闭
Auto8668p1_64.dts	Project dts，可以拉到文件最后，查看使用的 dtsi 文件
cust_mt8668_display_interface.dtsi	Interface (DSI), Panel, SerDes bridge, GPIO, leds 等相关 Display 设定，默认节点关闭。
cust_mt8668_display_config_main.dtsi	覆盖 cust_mt8668_display_interface.dtsi，开启需要打开的节点
cust_mt8668_display_serdes_config.dtsi	SerDes panel setting

注：一般情况修改橙色部分的文件即可。

3.3.5 DTS 参数介绍

单屏的 DTS Setting 包含三部分：DSI 节点，加串器的节点与 panel SerDes setting 节点。其中 DSI 与加串器设定都在 cust_mt8668_display_interface.dtsi 文件里，SerDes panel setting 的设定在 cust_mt8668_display_serdes_config.dtsi 文件中。可以根据 porting 的屏幕参数和 HW 情况做客制化。

3.3.5.1 DSI 节点

MIPI Panel 以子节点方式挂在 DSI 节点下，Panel Driver 会注册为 MIPI Device Driver。加串器（96789）作为 bridge 通过 remote-endpoint 挂在 dsi 的 port 上。如果平台没有 mipi panel driver，可以把 panel1 的节点去掉，只保留加串器部分的即可。

```
&dsi0 {①
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
    panel1 {②
        compatible = "boe,jd9365da";
        reg = <0>;
        power-gpios = <&pio 195 0>;
        reset-gpios = <&pio 60 0>;
        pinctrl-names = "default";
        port {
            panel1_in: endpoint {
                remote-endpoint = <&dsi_out>;
            };
        };
    };
};
```

```
};

ports {
    port {
        reg = <0>;
        dsi_out: endpoint {
            remote-endpoint = <&max96789_in>;③
        };
    };
};

&mipi_tx_config0 {④
    status = "okay";
};
```

序号	释义
①	DSI0 的节点
②	MIPI Panel 的节点，没有可以删除
③	加串器的 port 节点，必填。DSI Port 通过 Remote Endpoint 相连
④	MIPI PHY 节点，每个 DSI 都对应一个 PHY 的节点，DSI 节点打开时，PHY 的节点也要开启。

3.3.5.2 加串器节点

Ser max96789 是 I2C Device，会挂在对应的 I2C 的节点下。需要根据 HW 原理图配置 I2C 和 GPIO。

```
&i2c10 {①
    status = "okay";

    max96789: max96789@40 {②
        compatible = "maxim,max96789,dsi0";
        status = "disabled";
        reg = <0x40>;③
        reset-gpios = <&pio 98 0>;④
        // only for test
        //interrupt-parent = <&pio>;⑤
        //interrupts = <187 IRQ_TYPE_EDGE_RISING>;
        pinctrl-names = "default";
        setting = <&config_compatible>;⑥
        port {
            max96789_in: endpoint {
                remote-endpoint = <&dsi_out>;⑦
            };
        };
    };
};
```

序号	释义
①	max96789 在哪个 I2C 下，必填
②	Ser max96789 的节点
③	Max96789 I2C 地址，必填
④	Max96789 Power Down Pin，不同 GPIO 需要客制化
⑤	使用中断方式的 hotplug 使用，默认用轮询
⑥	屏幕兼容的 Setting，如果没有做兼容设定，可以直接引用屏幕的节点，例如 Setting = <&huayang_1080>;
⑦	remote-endpoint 指向 DSI，与 DSI port 相连

3.3.5.3 I2C 命令格式说明

格式	释义
写命令格式 1	<i2c addr> <reg width> <reg addr> <data> <delay ms>
写命令格式 2	<i2c addr> <reg width> <cmd len> <data>
读命令格式	<i2c addr> <reg width> <reg addr> <mask> <except data>
i2c addr	要操作设备的 i2c client 地址
reg width	i2c 设备的 reg 地址长度，只支持 16/8/0 三种 16 表示 reg 地址是 2 个 byte 8 表示 reg 地址是 1 个 byte 0 表示不指定 reg 地址，后面全是数据
reg addr	要读写的 reg 的地址
data	要写到 reg 的数据
delay ms	发送完该命令后 delay 多少毫秒
cmd len	reg 地址长度为 0 时，后面有多少 byte 的数据
mask	读出的数据跟这个数据做与操作
except data	读出的数据与 mask 做完与操作后预期的数据

3.3.5.4 屏幕兼容设定

Max96789 的节点中有包含兼容的屏幕设定，方法是给屏端地址为 0x1a 的 MCU 发一串指令，通过读出来值，看是哪个屏幕。例如读出来值&0xff (mask) 后，如果是 0xff，那么就可以拿到&huayang_1080 的节点。如何来做屏幕兼容，需要根据屏幕的 HW 情况来判断，这部分需要根据不同的情况做客制化。

```
config_compatible: compatible-node {①
    comp-cmd = <②
        0x1a 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0xd4
    >;③
    comp-type =④
        <0x1a 0x08 0x08 0xf0 0x10 &config_bt>,⑤
        <0x1a 0x08 0x08 0xff 0xff &huayang_1080>;
};
```

序号	释义
①	多个屏幕兼容节点，没有可以不添加
②	多屏兼容的 cmd，使用多屏兼容时必须填写
③	多屏兼容时写的命令，参考写命令格式 1
④	预期的回读对比数据，使用多屏兼容时必须填写
⑤	预期的多屏兼容设置，参考读命令格式 使用多屏兼容时必须填写，读出来的值跟 mask 做与操作，再跟 except data 做对比，如果一致，则使用该行 Panel Setting。

3.3.5.5 Panel Setting 节点

屏幕 setting 的节点包含加串器，解串器，背光以及 Panel 的 Timing 设定。这部分需要根据屏幕的设定来更改。设定放在 cust_mt8668_display_serdes_config.dtsi 文件中，并且挂在加串器 max96789 的节点下。

```
&max96789 {
    huayang_1080: config1 {①
        pre-init-cmd = <②
            0x40 0x10 0x0001 0x08 0x00③
            0x40 0x10 0x0203 0x00 0x00
            0x40 0x10 0x1404 0x29 0x00
            0x40 0x10 0x1504 0x29 0x00
            0x40 0x10 0x14a4 0xc8 0x00
            0x40 0x10 0x15a4 0xc8 0x00
            0x40 0x10 0x140a 0x00 0x00
        >;
```

```

0x40 0x10 0x150a 0x00 0x00
0x40 0x10 0x140b 0x00 0x00
0x40 0x10 0x150b 0x00 0x00
0x40 0x10 0x0330 0x04 0x00
0x40 0x10 0x0331 0x33 0x00
0x40 0x10 0x0333 0xe4 0x00
0x40 0x10 0x0308 0x7c 0x00
0x40 0x10 0x0053 0x10 0x00
0x40 0x10 0x005b 0x12 0x14
0x40 0x10 0x0140 0x20 0x00
0x40 0x10 0x0002 0x53 0x05
0x40 0x10 0x0010 0x31 0xc8
0x40 0x10 0x02dc 0x04 0x00
0x40 0x10 0x02dd 0xaa 0x00
0x40 0x10 0x02de 0x6a 0x00
>;
post-init-cmd = <>; ④
deinit-cmd = <⑤
0x40 0x10 0x0010 0x80 0x20
>;
linka-init-cmd = <⑥
0x4c 0x10 0x06ff 0x11 0x0
0x4c 0x10 0x01ce 0x4e 0x0
0x4c 0x10 0x020c 0x03 0x0
0x4c 0x10 0x020d 0xaa 0x0
0x4c 0x10 0x020e 0x4a 0x0
0x1a 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00
0xd⑦
>;
ser-status = <⑧
0x40 0x10 0x00d 0x80 0x80
0x40 0x10 0x102 0x80 0x80
0x40 0x10 0x55d 0x70 0x70
>;
linka-status = <⑨
0x40 0x10 0x01f 0x08 0x08
0x48 0x10 0x6ff 0x11 0x11
>;
panel-timing-a {⑩
width = <1920>;
height = <1080>;
hfp = <40>;
hsa = <42>;
hbp = <78>;
vfp = <24>;
vsa = <3>;
vbp = <9>;
fps = <60>;

```

```

width-mm = <129>; ⑪
height-mm = <64>;

};

};

```

序号	解释
①	屏幕 setting 节点
②	Pre Ser 初始化命令，在 bridge pre_enable 中执行。
③	初始化序列，参考写命令格式 1
④	post ser 初始化命令，在 bridge pre_enable 中执行，不需要可以不用添加
⑤	ser disabled 命令，在 bridge disabled 中执行
⑥	des linkA 初始化命令，在 bridge enable 中执行
⑦	背光序列
⑧	检测 ser 状态，读 i2c 命令，注意只支持如下 3 条命令并且要按顺序填写 0x40 0x10 0x00d 0x80 0x80 # 串行器是否能 detect 到 0x40 0x10 0x102 0x80 0x80 # 96789 检查 PCLK 是否 detect 到 0x40 0x10 0x55d 0x70 0x70 # 96789 检查 HS/VIS/DE 是否 detect 到 如果其他的 ser 需要客制化这部分寄存器
⑨	检测 linka 状态，注意只支持两条命令且要保持顺序，第一条检测是否连接，第二条检测是否被初始化 0x40 0x10 0x01f 0x08 0x08 # max96789 检查 linka 是否有接入 0x48 0x10 0x6ff 0x11 0x11 # max96789 检查 linka 是否有被初始化，通过往一个用不到且能读写的 reg 里写任意值再读出来判断它是否被初始化，可以根据情况客制化
⑩	屏幕 Timing 设定，必须填写
⑪	屏的物理尺寸，单位 mm，用来计算 PPI。可以从屏幕的 Spec 中拿到。

3.3.6 如何新增一个 LK 的 Driver

以 SerDes Driver serdes_dsi_vdo 为例：

1. 新增 panel 文件：vendor\mediatek\proprietary\bootable\bootloader\lk2\dev\lcm，如下图

A dev/lcm/serdes_dsi_vdo/rules.mk

A dev/lcm/serdes_dsi_vdo/serdes_dsi_vdo.c

2. 修改 vendor\mediatek\proprietary\bootable\bootloader\lk2\project\\$(project).mk 文件，将 Panel Driver 设置为新增文件，注意命名规则

```
22
23 MODULE_DEPS += app/$(BOOT_APP)
24 MTK_LCM_LIST_SUPPORT="serdes_dsi_vdo"
25 MTK_LCM_COMMON_DRIVER_SUPPORT := no
```

3. 修改 vendor\mediatek\proprietary\bootable\bootloader\lk2\dev\lcm\mt65xx_lcm_list.c 文件，将 Panel Driver 加入 list

```
34
35 LCM_DRIVER *lcm_driver_list[] = {
36
37 #if defined(VTDR6126A_VDO)
38     &vtdr6126a_vdo_lcm_drv,
39 #endif
40
41 #if defined(SERDES_DSI_VDO)
42     &serdes_dsi_vdo_lcm_drv,
43 #endif
44
```

4. 修改文件：vendor\mediatek\proprietary\bootable\bootloader\lk2\dev\lcm\include\lcm_drv.h，Extern 新增 Panel

```
1138 extern LCM_DRIVER vtdr6126a_vdo_lcm_drv;
1139 extern LCM_DRIVER serdes_dsi_vdo_lcm_drv;
1140
```

5. 新增 Driver 中实现相关的参数和 Function 设定。

3.3.7 LK Driver Function 介绍

LK LCM Driver 主要为实现 LCM_DRIVER 结构体的成员，方便后续 Display 获取相关参数和控制初始化时序。如果需要增加定制化的 Function，可以在 LCM_DRIVER 里面添加实现。

```
LCM_DRIVER serdes_dsi_vdo_lcm_drv = {
    .name = "serdes_dsi_panel_drv",
    .set_util_funcs = lcm_set_util_funcs,
    .get_params = lcm_get_params,
    .init = lcm_init,
    .suspend = lcm_suspend,
    .resume = lcm_resume,
    .compare_id = lcm_compare_id,
    .init_power = lcm_init_power,
    .resume_power = lcm_resume_power,
    .suspend_power = lcm_suspend_power,
    .set_fdt = lcm_get_fdt_info,
};
```

Function 名称	解释	是否需要修改 (Yes/No/Optional)
name	定义 Driver 名字	Y
set_util_funcs	lcm_util 结构体初始化	N
get_params	获取 Panel 基本参数，详见下一章节	Y
init_power	控制上电时序	Y
init	Panel 或者 bridge 的初始化 注：代码默认按照 lcm_init_power -> lcm_init 的顺序跑。此时 DSI 还没有送信号出来，对时序有要求需要注意。	Y
suspend suspend_power	休眠吃 Kernel lcm driver 里的配置，可不配置	N
resume resume_power	唤醒吃 Kernel lcm driver 里的配置，可不配置	N
compare_id	项目兼容多款屏时使用：若 MTK_LCM_LIST_SUPPORT 中定义了多个屏，开机时会轮流跑每个屏驱的初始化和	O

Function 名称	解释	是否需要修改 (Yes/No/Optional)
	该函数读取寄存器，若读取到的值符合预期，则说明该驱动适配当前屏。	
Set_fdt	从 fdt 中 parser dts 中的加串器以及屏幕的参数。公版已经做好，如果有其它定制化的部分需要自己添加。	O
set_backlight	如果背光需要在出视频流之后打开，可以在这个函数里面实现。	O

3.3.8 LK 参数 get_params 介绍

LCM Driver 的 lcm_get_params 函数里会将当前 Panel 的参数传给 DSI 和 Display Driver，从而让 MIPI TX 端输出符合 Panel 要求的 MIPI 波形，客户可以参考表格并根据场景 Panel Spec 修改 lcm_get_params 里的参数。SerDes Driver 中需要配置的 default 参数如下图所示，以供参考。SerDes Driver 中如果 dts 中有设定，Driver 会从 dts 中 Parse 出对应的 Panel 设定，覆盖 default 设定。

```
static void lcm_get_params(LCM_PARAMS *params)
{
    memset(params, 0, sizeof(LCM_PARAMS));

    memcpy(params, &g_serdes.lcm_ext_params, sizeof(LCM_PARAMS));
    if (params->crop_width[0] && params->crop_width[1]) {
        dprintf(INFO, "super frame! [%d*%d]+[%d*%d]=[%d*%d]\n", params->crop_width[0],
```

```
static void lcm_set_default_para(struct serdes *ser_des)
{
    ser_des->reset_gpio = 37;

    ser_des->serdes_iic_id = 1;
    ser_des->ser_iic_addr = 0x40;
    ser_des->desdef->des_iic_addr = 0x4a;
    ser_des->super_frame = false;

    ser_des->lcm_ext_params.type = LCM_TYPE_DSI;
    ser_des->lcm_ext_params.lcm_if = LCM_INTERFACE_DSI0;
    ser_des->lcm_ext_params.width = 1920;
    ser_des->lcm_ext_params.height = 1080;
    ser_des->lcm_ext_params.dsi.mode = SYNC_PULSE_VDO_MODE;
    ser_des->lcm_ext_params.dsi.LANE_NUM = LCM_FOUR_LANE;
    ser_des->lcm_ext_params.dsi.data_format.color_order = LCM_COLOR_ORDER_RGB;
    ser_des->lcm_ext_params.dsi.data_format.trans_seq = LCM_DSI_TRANS_SEQ_MSB_FIRST;
    ser_des->lcm_ext_params.dsi.data_format.padding = LCM_DSI_PADDING_ON_LSB;
    ser_des->lcm_ext_params.dsi.data_format.format = LCM_DSI_FORMAT_RGB888;

    ser_des->lcm_ext_params.dsi.packet_size = 256;
    ser_des->lcm_ext_params.dsi.PS = LCM_PACKED_PS_24BIT_RGB888;

    ser_des->lcm_ext_params.dsi.vertical_sync_active = 2;
    ser_des->lcm_ext_params.dsi.vertical_backporch = 28;
    ser_des->lcm_ext_params.dsi.vertical_frontporch = 230;
    ser_des->lcm_ext_params.dsi.vertical_active_line = 1920;

    ser_des->lcm_ext_params.dsi.horizontal_sync_active = 20;
    ser_des->lcm_ext_params.dsi.horizontal_backporch = 20;
    ser_des->lcm_ext_params.dsi.horizontal_frontporch = 40;
    ser_des->lcm_ext_params.dsi.horizontal_active_pixel = 1080;
    ser_des->lcm_ext_params.dsi.ssc_disable = 1;
    ser_des->lcm_ext_params.dsi.cont_clock = 1;
    ser_des->lcm_ext_params.dsi.PLL_CLOCK = 478;
    ser_des->lcm_ext_params.dsi.vdo_per_frame_lp_enable = 1;
}
```

一般情况下上图的 SerDes 设定可以覆盖 SerDes 的使用场景，其他参数可以不设定。具体的参数介绍，可以参考如下说明。vendor\mediatek\proprietary\bootable\bootloader\lk2\dev\lcm\include\lcm_drv.h, LCM_PARAMS 结构体中部分参数未使用到，参考下表修改 Yes/Optional 的参数即可。

参数	解释	是否需要修改 (Yes/No/Optional)
type	Panel Interface 类型，可选 DSI/DPI/DBI	Y
width/height	每帧的图像数据的像素宽高	Y
mode	屏正常送图时的传输模式，分为以下四种 (详见 <i>mipi_DSI_specification</i>)，根据 LCM Spec 配置： CMD_MODE SYNC_PULSE_VDO_MODE	Y

参数	解释	是否需要修改 (Yes/No/Optional)
	SYNC_EVENT_VDO_MODE BURST_VDO_MODE	
switch_mode_enable	mode 切换	O
switch_mode	切换的 mode	O
Packet_size	未使用的参数, 无需修改	N
PS	选择 pixel stream type, 常见有以下四种: LOOSELY-RGB666 PACKED-RGB666 PACKED-RGB565 PACKED-RGB888	Y
LANE_NUM	MIPI Lane 数	Y
physical_width	Panel 物理宽高, 单位 mm	O
physical_height		O
data_format	LCD 数据格式, 包含以下参数: color_order: RGB SWAP, 支持 RGB/BGR 顺序传输 trans_seq: Data bit 的传输顺序, 即最高位(MSB)/最低位(LSB)先传输 Padding: 未使用参数, 无需修改 Format: 数据格式, 例如 RGB888/666/565...	Y
vertical_sync_active	Display timing vertical params. VSA	Y
vertical_backporch	Display timing vertical params. VBP	Y
vertical_frontporch	Display timing vertical params. VFP	Y
vertical_active_line	Display timing vertical params. VACT	Y
vertical_frontporch_for_low_power	Low Power Feature, 表示当系统无需画面刷新时, 调整 VFP 为该值来降帧, 以达到降低功耗目的 (前提要求屏端支持动态切换该帧率)。通过 $FPS/fps_new = vtotal_new/VTOTAL$ 计算得到 vfp_low_power。	O
horizontal_sync_active	Display timing vertical params. HSA	Y
horizontal_backporch	Display timing vertical params. HBP	Y
horizontal_frontporch	Display timing vertical params. HFP	Y
horizontal_active_pixel	Display timing vertical params. HACT	Y

参数	解释	是否需要修改 (Yes/No/Optional)
ssc_disable	展频开关，展频可减少对其它信号的电磁干扰，但要求 Panel 支持展频。 1 表示关闭展频，0 表示开启展频。 SerDes 一般情况建议关闭展频	O
ssc_range	展频范围，可设定 0~5，单位为‰ 例如默认 ssc_range = 5，即向下展频宽度为 5‰。	O
PLL_CLOCK	MIPI RX 采样的 Clock	Y
data_rate	对于 DSI 的 data rate = 2 * PLL_CLOCK	Y
data_rate_khz	MIPI 采样率，equal data_rate * 1000，有 FPS 精度要求时需配置	O
cont_clock	开启连续时钟。 1: Enable 连续时钟，clock lane 不进入 LP。 0: Disable 连续时钟，clock lane 每帧进入一次 LP (每行间保持 hs) SerDes 一般情况建议设为连续时钟。	O
clk_lp_per_line_enable	1: Lock lane 每行进入一次 LP (仅在 cont_clock = 0 时才生效)。	O
esd_check_enable	ESD check 功能配置： esd_check_enable = 1; // 开启 ESD Check customization_esd_check_enable = 1; // read LCM register customization_esd_check_enable = 0 // DSI_TE EINT	O
customization_esd_check_enable		O
lcm_esd_check_table[]		O
IsCphy	使能 CPHY 接口时配置。cphy 设置为 1，dphy 设置为 0。	O
vdo_per_frame_lp_enable	使能 data 每帧回一次 LP (但要求 clk_lp_per_line_enable ≠ 1，即 clock 不能每行回 LP)。细节介绍请参考 章节 14.5.1 。	O
lane_swap_en	Lane 硬件连接错误时使能	O
lane_swap	Lane0/1/2/3 序的 SWAP	O

参数	解释	是否需要修改 (Yes/No/Optional)
pn_swap	P/N SWAP (仅支持在同一个 lane 内交换)	O
HS_TRAIL	PHY Timing 配置, MIPI Test Timing Fail 时配置	O
HS_ZERO		O
HS_PRPE		O
LPX		O

3.3.9 DSI0 BootLogo 设定

首先到如下路径找到当前屏幕尺寸所对应的 Logo 资源文件夹, 如果没有所需大小的资源文件夹, 也可以修改裁剪 LK Logo 对应当前使用屏幕的分辨率, 然后到如下路径中新增一个图片资源, 添加方法如下:

1. 在下面目录下新增或查找图片资源文件 vendor/mediatek/proprietary/external/BootLogo/logo/
注: 新增的图片资源以目录名为前缀, 可参考其他目录。
2. 将下面的.mk 文件中的 BOOT_LOGO config 修改为上方找到的或新增的图片目录。
device/mediatekprojects/{project_name}/ProjectConfig.mk
3. 由于当前的纯 Android 公版会启用多分辨率 Logo 功能, 因此 BOOT_LOGO 配置为 uhd。对应的 uhd 文件夹中包含多个不同分辨率的 uboot Logo 资源, 用于在显示 Logo 时根据所连接屏幕的显示宽度和高度适配显示不同尺寸的 Logo。

在添加不同尺寸的 uboot Logo 图片后, 需要在 BootLogo/logo/rules.mk 文件的 LOGO_RESOURCE_OBJ_LIST1 中相应地添加图片记录, 以便将其打包进 logo.img 中。每个新增的不同分辨率 Logo 的索引值 (index) 对应其在 LOGO_RESOURCE_OBJ_LIST1 中被添加的顺序位置 (索引从 0 开始)。

对应的 Logo 显示相关的源代码路径如下:

```
vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/common/logo
vendor/mediatek/proprietary/bootable/bootloader/lk2/lib/libshowlog
```

3.3.10 DWS 设定

dws 路径: \vendor\mediatek\proprietary\tools\dct\dws\mt6881\<Project>.dws

可以通过 DCT 修改或直接修改 dws 文件, 需要根据屏使用的 GPIO 和 I2C 配置, 以下为配置后的 dws 中 GPIO 和 I2C 的模板:

GPIO

```

<gpio174>
    <eint_mode>>false</eint_mode>
    <def_mode>0</def_mode>
    <inpull_en>true</inpull_en>
    <inpull_selhigh>>false</inpull_selhigh>
    <def_dir>OUT</def_dir>
    <out_high>>false</out_high>
    <varName0>GPIO_LCM_LED_EN</varName0>
    <smt>true</smt>
    <ies>true</ies>
</gpio174>

```

I2C

```

<gpio125>
    <eint_mode>>false</eint_mode>
    <def_mode>1</def_mode>
    <inpull_en>true</inpull_en>
    <inpull_selhigh>true</inpull_selhigh>
    <def_dir>IN</def_dir>
    <out_high>>false</out_high>
    <varName0>GPIO_I2C1_SCA_PIN</varName0>
    <smt>true</smt>
    <ies>true</ies>
</gpio125>
<gpio126>
    <eint_mode>>false</eint_mode>
    <def_mode>1</def_mode>
    <inpull_en>true</inpull_en>
    <inpull_selhigh>true</inpull_selhigh>
    <def_dir>IN</def_dir>
    <out_high>>false</out_high>
    <varName0>GPIO_I2C1_SDA_PIN</varName0>
    <smt>true</smt>
    <ies>true</ies>
</gpio126>

```

进入系统后，可以通过以下指令确定 GPIO/I2C 是否配置成功：

```
#cat /proc/mtk_gpio/soc.pinctrl
```

```

./ # cat /proc/mtk_gpio/soc.pinctrl
pins base: 299
PIN: (MODE) (DIR) (DOUT) (DIN) (DRIVE) (SMT) (IES) (PULL_EN) (PULL_SEL) (R1 R0)
000: 0001001111
001: 0000001110
002: 0000001110

```

注：

- MODE: Aux.Function selection, range: 0~7
- DIR: 0 for input mode; 1 for output mode (This is register value of MTK's DIR bit)
- DOUT/DIN: 0 for low; 1 for high
- DRIVE: Driving current selection, range: 0/1/2/3/4/5/6/7, 定义参考平台对应 GPIO table

- IES/SMT: 0 for disable; 1 for enable
- PULLEN/R1/R0: 0 for disable; 1/2/3 for enable
- For pin with 2 pull resistors, R1 and R0 are shown
- For pin with 1 pull resistor, R1 and R0 are not shown
- PULLSEL: 0 for selecting pull-down resistor; 1 for pull-up resistor

3.4 Kernel Driver Porting

Kernel Driver 用于 Kernel 启动的 DSI1 对应的 LCM（DSI0 默认 LK 启动），并且控制所有的 DSI Port 的 LCM Suspend 和 Resume。

3.4.1 Kernel Driver 文件路径

名称	路径
LCM Driver	kernel/kernel_device_modules_6.12/drivers/gpu/drm/panel/bridge-serdes.c
	kernel/kernel_device_modules_6.12/drivers/gpu/drm/panel/bridge-serdes.h
	kernel/kernel_device_modules_6.12/drivers/gpu/drm/panel/panel-boe-jd9365da-vod.c
	kernel/kernel_device_modules_6.12/drivers/gpu/drm/panel/Kconfig
	kernel/kernel_device_modules_6.12/drivers/gpu/drm/panel/BUILD.bazel
Project.mk	Device/mediateksample/{Project}/ProjectConfig.mk
Kernel config	kernel/kernel_device_modules_6.12/arch/arm64/configs/mgk_64_k612_defconfig
Kernel kleaf	Kernel/kernel_device_modules_6.12/kernel/kleaf/mgk_64.bzl
KO Order Table	Device/mediateksample/{Project}/ko_order_table.csv
Kernel DTS	arch/arm64/boot/dts/mediatek/cust_mt8668_display_interface.dtsi
	arch/arm64/boot/dts/mediatek/cust_mt8668_display_config_main.dtsi
	arch/arm64/boot/dts/mediatek/cust_mt8668_display_serdes_config.dtsi
Display Driver	kernel/kernel_device_modules-6.12/drivers/gpu/drm/mediatek/Mediatek_v2/mtk_dsi.c, mtk_mipi_tx.c, mtk_disp_dsc.c, mtk_drm_crtc.c, mtk_panel_ext.c,...

3.4.2 Kernel Driver 文件参考

SerDes 的 Panel Driver，请参照 Panel 目录下的 bridge-serdes.c，Driver 的写法是从 dts Panel 的节点里面 Parse 出对应的 Panel Timing 和 SerDes 的设置。如果沿用这个 Driver，可以把 dts 中的参数更改为要 Bringup 的 SerDes 参数和屏幕 Timing。如果需要 Porting MIPI 的 Panel Driver，请参照 panel 目录下的 panel-boe-jd8365da-vdo.c

3.4.3 DTS 文件介绍

请参考章节 3.3.4。

3.4.4 DTS 参数介绍

请参考章节 3.3.5。

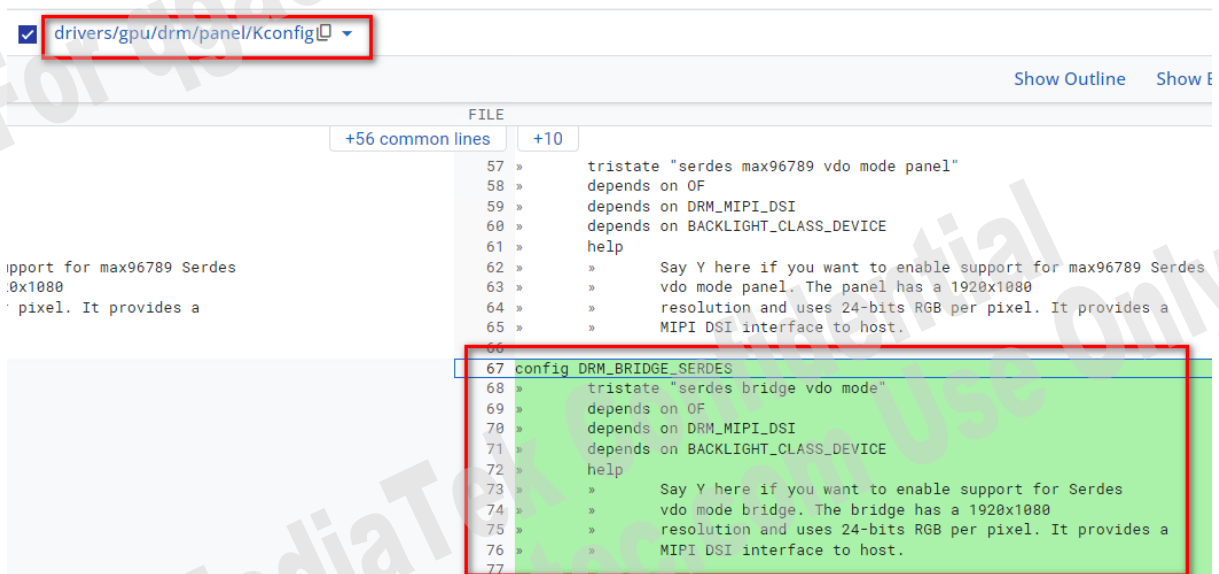
3.4.5 如何添加一个新的 Kernel Driver

Kernel 的 Driver 都是以 KO 的方式加载的，如果需要添加新的 Driver，需要注意在 Kleaf 和 KO table 中添加。Panel 的 KO 是 ramdisk KO，需要注意添加的位置会影响初始化的顺序，建议参考公版添加位置。

1. 添加新的 Driver。

A	drivers/gpu/drm/panel/bridge-serdes.h
A	drivers/gpu/drm/panel/bridge-serdes.c
M	drivers/gpu/drm/panel/BUILD.bazel
M	drivers/gpu/drm/panel/Kconfig

2. 在 kconfig 加入新的 Driver。



3. 在 panel BUILD.bazel 中加入新的 Driver。

drivers/gpu/drm/panel/BUILD.bazel

FILE	
+98 common lines	+10
99	"panel-nt37801-cmd-fhd-10bit",
100	"panel-nt37801-cmd-ltpo",
101	"panel-nt37801-cmd-spr",
102	"panel-nt37701a-cmd",
103	"panel-nt37701a-cmd-mipichg",
104	"panel-truly-td4330-cmd",
105	"panel-samsung-s68fc01-vdo-aod",
106	"panel-hx-nt37701-dphy-cmd",
107	"panel-hx-nt37701-dphy-cmd-120hz",
108	"panel-vtdr6126a-vdo",
109	"bridge-serdes",
110	]
111	

4. 修改 defconfig 文件。

arch/arm64/configs/mgk_64_k612_defconfig

FILE	
+251 common lines	+10
252	CONFIG_REGULATOR_RT5133=m
253	CONFIG_DEVICE_MODULES_REGULATOR_RT6160=m
254	CONFIG_REGULATOR_MT6983_ALPHA_SUPPORT=m
255	CONFIG_MTK_VMM_SPM=m
256	CONFIG_MTK_VMM_SPM_MT6989=m
257	CONFIG_MTK_VMM_NOTIFIER=m
258	CONFIG_MTK_VMM_NOTIFIER_MT6991=m
259	CONFIG_MTK_VMM_NOTIFIER_MT6993=m
260	CONFIG_MTK_PMIC_PROTECT=m
261	CONFIG_DRM_PANEL_SERDES=m
262	CONFIG_DRM_BRIDGE_SERDES=m
263	CONFIG_DRM_PANEL_BOE_JD0365DA_VDO=m

5. 在 kleaf 中添加 driver。

kernel/leaf/mgk_64.bzl

FILE	
+786 common lines	+10
at(kernel_version),	787
_slc".format(kernel_version),	788
arm_smmu_v3".format(kernel_version),	789
mtk_smmuv3-mpam-mon".format(kernel_ver	790
sion),	
mtk_smmuv3-pmu".format(kernel_version)	791
n".format(kernel_version),	792
le-unused".format(kernel_version),	793
rmat(kernel_version),	794
mp:mrdump".format(kernel_version),	795
rt4831a_drv".format(kernel_version),	796
.format(kernel_version),	797
798	798
799	799

6. 在 KO order table 里面添加 KO, table 的 module 会被顺序加载, 请将新增的 Panel Module 添加到合适的位置, 避免太早或太晚 init. 请参照公版添加 Panel Module 的位置。

auto8668p1_64/ko_order_table.csv

Show Outline Show Entire File Show I

```

r.ko, ramdisk, Y, Y, user/userdebug/eng
        , Y, user/userdebug/eng
65 drm_dma_helper.ko, /drivers/gpu/drm/drm_dma_helper.ko, ramdisk, Y, Y, user/userdebug/eng
66 i2c-mt65xx.ko, /.../kernel_device_modules-6.12/drivers/i2c/busses/i2c-mt65xx.ko, ramdisk, Y, Y, user
    ebug/eng
drivers/gpu/drm/mediatek/mediatek_v2/mtk_panel_ext.k 67 mtk_panel_ext.ko, /.../kernel_device_modules-6.12/drivers/gpu/drm/mediatek/mediatek_v2/mtk_panel
    o, ramdisk, Y, Y, user/userdebug/eng
device_modules-6.12/drivers/gpu/drm/panel/panel-alph 68 panel-boe-jd9365da-vdg.ko, /.../kernel_device_modules-6.12/drivers/gpu/drm/panel/panel-boe-jd936
    ebug/eng
device_modules-6.12/drivers/gpu/drm/panel/panel-alp 69 bridge-serdes.ko, /.../kernel_device_modules-6.12/drivers/gpu/drm/panel/bridge-serdes.ko, ramdis
    er/userdebug/eng
ardebug/eng

```

7. 在 dts 中加入 Driver。

```

133 &i2c10 {
134     status = "okay";
135
136     max96789: max96789@40 {
137         compatible = "maxim,max96789,dsi0";
138         status = "disabled";
139         reg = <0x40>;
140         reset-gpios = <&pio 98 0>;
141         // only for test
142         //interrupt-parent = <&pio>;
143         //interrupts = <187 IRQ_TYPE_EDGE_RISING>;
144         pinctrl-names = "default";
145         setting = <&config_compatible>;
146         port {
147             max96789_in: endpoint {
148                 remote-endpoint = <&dsi_out>;
149             };
150         };
151     };
152 }

```

8. 实现 Driver 中的 Function 和参数设定。

3.4.6 Kernel Driver Function 介绍

Kernel SerDes driver DSI 是通过 DRM panel 的 function 来实现 SerDes 的开关操作以及 display mode 的参数传递。

```

static const struct drm_panel_funcs lcm_drm_funcs = {
    .disable = serdes_panel_disable,
    .unprepare = serdes_panel_unprepare,
    .prepare = serdes_panel_prepare,
    .enable = serdes_panel_enable,
    .get_modes = serdes_panel_get_modes,
};

```

MTK DSI Panel 扩展客制化函数

```

static struct mtk_panel_funcs ext_funcs = {
    .get_real_vdo_timing = serdes_panel_get_real_vdo_timing,
    .get_link_status = serdes_panel_get_link_status,
};

```

Function 名称	解释	是否需要修改 (Yes/No/Optional)
probe	Panel 注册函数，LCM 的初始化和探测	Y
remove	用于移除并释放 LCM 相关的资源	Y
prepare	用于准备 LCM Power On 和初始化，此时 DSI 还没有开始送信号出来。	Y
enable	用于启动 DRM Panel，也可以控制背光。此时 DSI 已经 Start，已经有 MIPI 信号输出。对时序有要求的话，需要注意。	Y
disable	用于启动 DRM Panel，点亮背光	Y
unprepare	用于取消 DRM Panel 的初始化	Y
get_modes	用于获取并设置 Panel 的显示模式	Y
reset	用于控制屏幕的 Reset	O
ata_check	SerDes 屏幕不需要设定	N
get_real_vdo_timing	SerDes 当 superframe 时，给 DSI 送实际输出的 timing，superframe 必须实现	O
get_link_status	SerDes 屏幕是否有接上，开机的时候会 check 状态，并且会上报 connector 状态	O

3.4.7 Kernel 参数介绍

1. default_mode 结构体是 DRM 会通过 get_modes 函数获取 Panel 的 Timing 信息，这部分参数必须要配置。

```
static const struct drm_display_mode default_mode = {
    .clock = PCLK,
    .hdisplay = FRAME_WIDTH,
    .hsync_start = FRAME_WIDTH + HFP,
    .hsync_end = FRAME_WIDTH + HFP + HSA,
    .htotal = FRAME_WIDTH + HFP + HSA + HBP,
    .vdisplay = FRAME_HEIGHT,
    .vsync_start = FRAME_HEIGHT + VFP,
    .vsync_end = FRAME_HEIGHT + VFP + VSA,
    .vtotal = FRAME_HEIGHT+ VFP + VSA + VBP,
};
```

SerDes Driver 中通过 serdes_bridge_get_modes 函数实现，可以通过 log 确认 timing 参数设定是否正常。

```
int serdes_bridge_get_modes(struct drm_bridge *bridge,
pr_info("%s[i2c%d] port[%d]+\n", __func__, ser_des-
if (connector) {

    mode.hdisplay = ser_des->panel_timing[port].width;
    mode.hsync_start = mode.hdisplay + ser_des->panel_timing[port].hfp;
    mode.hsync_end = mode.hsync_start + ser_des->panel_timing[port].hsa;
    mode.htotal = mode.hsync_end + ser_des->panel_timing[port].hbp;

    mode.vdisplay = ser_des->panel_timing[port].height;
    mode.vsync_start = mode.vdisplay + ser_des->panel_timing[port].vfp;
    mode.vsync_end = mode.vsync_start + ser_des->panel_timing[port].vsa;
    mode.vtotal = mode.vsync_end + ser_des->panel_timing[port].vbp;
    mode.clock = mode.htotal * mode.vtotal * ser_des->panel_timing[port].fps / 1000;

    pr_info("%s[i2c%d]: Display timing: %dx%d@%d [%d,%d,%d %d,%d,%d]\n", __func__,
        ser_des->client->adapter->nr,
        mode.hdisplay, mode.vdisplay, ser_des->panel_timing[port].fps,
        mode.hsync_start, mode.hsync_end, mode.htotal,
        mode.vsync_start, mode.vsync_end, mode.vtotal);
}
```

参数	解释	是否需要修改 (Yes/No/Optional)
clock	以 kHz 为单位的像素时钟频率，total 的频率，包含 blank 区间。这个 clock 为 pixel clock，计算公式为： htotal x vtotal x fps	Y
hdisplay	水平方向的显示分辨率宽度	Y
hsync_start	水平同步信号开始的位置	Y
hsync_end	水平同步信号结束的位置	Y
htotal	水平方向的总像素数	Y
vdisplay	垂直方向的显示分辨率高度	Y
vsync_start	垂直同步信号开始的位置	Y
vsync_end	垂直同步信号结束的位置	Y
vtotal	垂直方向的总像素数	Y

2. MTK 扩展 Panel 参数结构体 ext_params 必须实现。

```
static struct mtk_panel_params ext_params = {
    .pll_clk = 446,
    .data_rate = DATARATE,
    //.vfp_low_power = 112,
    .cust_esd_check = 0,
    .esd_check_enable = 0,
    .lcm_esd_check_table[0] = {
        .cmd = 0x0A, .count = 1, .para_list[0] = 0x9C,
    }
};
```

```
},
.ssc_enable = 0,
.physical_width = FRAME_WIDTH,
.physical_height = FRAME_HEIGHT,
};
```

SerDes Driver 中，是在 driver probe 时会赋值。

```
static int serdes_iic_driver_probe(struct i2c_client *c
serdes_get_panel_timing(ser_des, &mode);
if (ser_des->link_type != LINK_TYPE_SUPERFRAME) {
    ser_des->panel_data.ext_params.crop_width[0] = ser_des->panel_timing[0].width;
    ser_des->panel_data.ext_params.crop_width[1] = ser_des->panel_timing[1].width;
    ser_des->panel_data.ext_params.crop_height[0] = ser_des->panel_timing[0].height;
    ser_des->panel_data.ext_params.crop_height[1] = ser_des->panel_timing[1].height;
} else {
    ser_des->panel_data.ext_params.crop_width[0] = 0;
    ser_des->panel_data.ext_params.crop_width[1] = 0;
    ser_des->panel_data.ext_params.crop_height[0] = 0;
    ser_des->panel_data.ext_params.crop_height[1] = 0;
}

ser_des->panel_data.ext_params.pll_clk =
    ser_des->panel_timing[ser_des->driver_data->port].pll ?
    ser_des->panel_timing[ser_des->driver_data->port].pll :
    DIV_ROUND_UP_ULL(mode.clock * 3, 1000);
pr_info("%s: pll_clk = %d\n", __func__, ser_des->panel_data.ext_params.pll_clk);
ser_des->panel_data.ext_params.prefetch_time =
    ser_des->panel_timing[ser_des->driver_data->port].prefetch;

ser_des->panel_data.ext_params.vdo_keep_hs_perline = 1;
```

参数	解释	是否需要修改 (Yes/No/Optional)
pll_clk	MIPI PLL 时钟频率，此处配置会覆盖使用 PCLK 计算出的 MIPI Clock，在超级帧模式下，必须设置 pll_clk	Y
data_rate	数据传输速率，等于 pll_clk*2，此处配置会覆盖 pll_clk 配置的 MIPI Clock，如果需要精确的 datarate，可以配置次参数	Y
phy_timcon	DSI 物理层时间配置，此处配置会覆盖计算得出的 MIPI 时序	O
crop_width	superframe 单个屏幕的裁剪宽度，在超级帧模式下，应设置为与面板之一的宽度/高度相对应	O
crop_height	superframe 单个屏幕的裁剪高度，在超级帧模式下，应设置为与面板之一的宽度/高度相对应	O

参数	解释	是否需要修改 (Yes/No/Optional)
vfp_low_power	低功耗模式下的垂直前沿间隔，进入 idle mode 时改变 VFP，从而降低刷新率。 会造成帧率频繁抖动，SerDes/Bridge 屏建议关闭	N
esd_check_enable	ESD check 开关	N
cust_esd_check	自定义 ESD check 方式，SerDes/Bridge 屏由于屏幕在远端，此功能无法使用	N
lcm_esd_check_table	LCM ESD DDIC table，ESD READ DDIC 需要读取的寄存器地址和数量	N
ssc_enable	SSC (展频时钟) 使能，减少 EMI，会造成帧率和时钟轻微抖动。 SerDes/Bridge 不一定支持，如需要请确认后配置	O
ssc_range	SSC 范围，1%~5%，展频范围由 SPEC 规范	O
physical_width	物理宽度 (mm)，上层会使用此参数调整 DPI，在超级帧模式下，physical_width 应设置为两个面板宽度之和。	Y
physical_height	物理高度 (mm)，上层会使用此参数调整 DPI。在超级帧模式下，physical_height 应设置为两个面板中较大的一个的高度。	Y
dsc_params	用于配置 DSC 详细参数，如需要使用请咨询	O
prefetch_time	调整 VSYNC 时间点，调整时序以满足特定的时序要求	O
output_mode	输出模式， Single Port Mode: 常规模式 Dual Port Mode: 使用两路 DSI 来输出一幅画面 dsc single mode: 开启 DSC，dsc dual port setting 在 dsc_params 中	O
is_cphy	C-PHY 配置，使能 C-PHY	O
lane_swap_en	MIPI Lane 交换，可选值：MIPITX_PHY_LANE_0~3，CK、RX	O
lane_pn_swap	交换 Lane PN，只能交换同一组 Lane 的 PN，不能交换不同组的	O
Vdo_keep_hs_perline	使能 data 每帧回一次 LP，建议开启。细节介绍请参考 章节 14.5.1	Y

3. 添加 DSI 基本格式属性，必须配置。

```
desc = of_device_get_match_data(&dsi->dev);
dsi->lanes = desc->lanes;
dsi->format = desc->format;
dsi->mode_flags = desc->mode_flags;
ctx->desc = desc;
ctx->dsi = dsi;
```

DSI->mode_flags 参考如下设定:

```
static const struct panel_desc serdes_panel_desc = {
    .bpc = 8,
    .size = {
        .width_mm = 141,
        .height_mm = 226,
    },
    .lanes = 4,
    .format = MIPI_DSI_FMT_RGB888,
    .mode_flags = MIPI_DSI_MODE_VIDEO | MIPI_DSI_MODE_VIDEO_SYNC_PULSE |
        MIPI_DSI_MODE_LPM,
};
```

SerDes driver 中 serdes_bridge_attach 中赋值, 这部分一定要添加, 否则默认会是 CMD mode。

```
static int serdes_bridge_attach(struct drm_bridge *brid
{
    pr_info("%s: register dsi device fail: %ld!\n",
    return -1;
}
dsi->lanes = 4;
dsi->format = MIPI_DSI_FMT_RGB888;
dsi->mode_flags = MIPI_DSI_MODE_VIDEO;
ret = devm_mipi_dsi_attach(ser_des->dev, dsi);
if (ret < 0) {
    pr_info("%s: attach dsi fail!\n", __func__);
    return -1;
}
```

参数	解释	是否需要修改 (Yes/No/Optional)
lanes	使用的 DSI 数据通道数, DSI 支持 1~4 条 lane, lane 数越多, 总带宽越大, 目前使用的大多数 panel 都为 4 条 lane	Y
format	DSI 传输的像素格式, 默认为 MIPI_DSI_FMT_RGB888, 如需输出其他 format, 需另行咨询评估	Y
mode_flags	DSI 模式, 定义传输模式和特性。 主要配置 DSI 使用的 4 种 mode MIPI_DSI_MODE_VIDEO: 启用 VDO mode, 未配置则为 CMD mode。 MIPI_DSI_MODE_VIDEO_BURST: 使能 BURST_VDO_MODE MIPI_DSI_MODE_VIDEO_SYNC_PULSE: 使能 SYNC_PULSE_VDO_MODE	Y

参数	解释	是否需要修改 (Yes/No/Optional)
	MIPI_DSI_MODE_NO_EOT_PACKET: HS 禁用 EoT packet MIPI_DSI_CLOCK_NON_CONTINUOUS: 使能非连续时钟, 非连续时钟会在 blanking 区间进入 LP mode, 会有额外的 data phy cycle。 MIPI_DSI_MODE_LPM: 使能 LP send cmd, 拿掉后会在 HS send cmd。目前默认 LP send cmd, 此 config 未使用	

3.5 disp_pwm 方式背光控制

背光的亮度控制方式有多种, 有普通 PWM 控制, I2C 通信的 Driver 产生 PWM 控制, 屏幕端 MCU 产生 PWM 控制, disp_pwm 控制等方式。使用那种方式是由屏幕的 HW 来决定的。在调背光的 Feature 时需要明确屏幕用的是那种, 这个章节只说明背光由 disp_pwm 控制的情况要如何配置, MT8668 可以支持两路 disp_pwm。

3.5.1 disp_pwm dws 设定

HW 原理图中确认使用的 disp_pwm 哪个 GPIO, 下图是 co_device_module\include\dt-bindings\pinctrl\mt6881-pinfunc.h 里面查到 disp_pwm Pin。具体是从哪个 pin 输出, 需要原理图确认, 把 DWS 中的 GPIO Pin Function 设定为 disp_pwm。以 PINMUX_GPIO6__FUNC_DISP_PWM 为例, 需要将 GPIO6 def_mode 设定为 1。

```
#define PINMUX_GPIO6__FUNC_DISP_PWM (MTK_PIN_NO(6) | 1)
#define PINMUX_GPIO7__FUNC_DISP_PWM1 (MTK_PIN_NO(7) | 1)
#define PINMUX_GPIO52__FUNC_DISP_PWM1 (MTK_PIN_NO(52) | 3)
#define PINMUX_GPIO61__FUNC_DISP_PWM (MTK_PIN_NO(61) | 1)
```

下图中 DWS 的设定供参考。由于 GPIO 可能会被其他模块使用, 需要在 dts 中确认是否有其他模块使用这根 GPIO。如果有被使用, 则需要删除, 不然 GPIO 的 Mode 可能会被更改, 导致没有波形输出。

```
<gpio6>
    <eint_mode>false</eint_mode>
    <def_mode>1</def_mode>
    <inpull_en>true</inpull_en>
    <inpull_selhigh>false</inpull_selhigh>
    <def_dir>IN</def_dir>
    <out_high>false</out_high>
```

```

    <varName0>GPIO_PWM_BL_PIN</varName0>
    <smt>true</smt>
    <ies>true</ies>
</gpio6>

```

3.5.2 leds dts 设定

背光的亮度值会通过 leds 节点传下来一个 Brightness Level 值，然后这个值会转换为 Duty 和 Period 的值给到 disp_pwm Driver 去设定占空比。leds 的节点中会引用 disp_pwm 节点，来实现背光的控制。leds 的根节点在 mt6897.dtsi 中，如果需要多屏的 leds 控制，需要建立多个 leds 节点。下图中是两个 leds 节点的参考。两路 disp_pwm 的节点在 mt6897.dtsi 中，如果需要开启，需要先把对应节点中 status 由 “disabled” 改为 “okay”。

```

mtk_leds: mtk_leds {
    backlight { ①
        label = "lcd-backlight";
        max-brightness = <2047>;②
        max-hw-brightness = <255>;③
    };
};

mtk_leds1: mtk_leds1 {
    backlight {
        label = "lcd-backlight1";
        max-brightness = <2047>;
        max-hw-brightness = <255>;
    };
};

&mtk_leds {
    compatible = "mediatek,pwm-leds";
    backlight {
        led_mode = <5>;④
        pwm_config = <0 1 0 0 0>;⑥
        pwms = <&disp_pwm 0 39385>;⑤
        pwm-names = "lcd-backlight";
    };
};

&mtk_leds1 {⑦
    compatible = "mediatek,pwm-leds";
    backlight {
        led_mode = <5>;
        pwm_config = <0 1 0 0 0>;
        pwms = <&disp_pwm1 0 39385>;
        pwm-names = "lcd-backlight1";
    };
};

disp_pwm: disp-pwm0@1100e000 {
    compatible = "mediatek,disp_pwm0",
        "mediatek,mt6881-disp-pwm0";

```

```

reg = <0 0x1100e000 0 0x1000>;
interrupts = <GIC_SPI 254 IRQ_TYPE_LEVEL_HIGH 0>;
#pwm-cells = <2>;
clocks = <&cksys_clk CLK CK_DISP_PWM_SEL>;
clock-names = "main";
status = "okay";
};

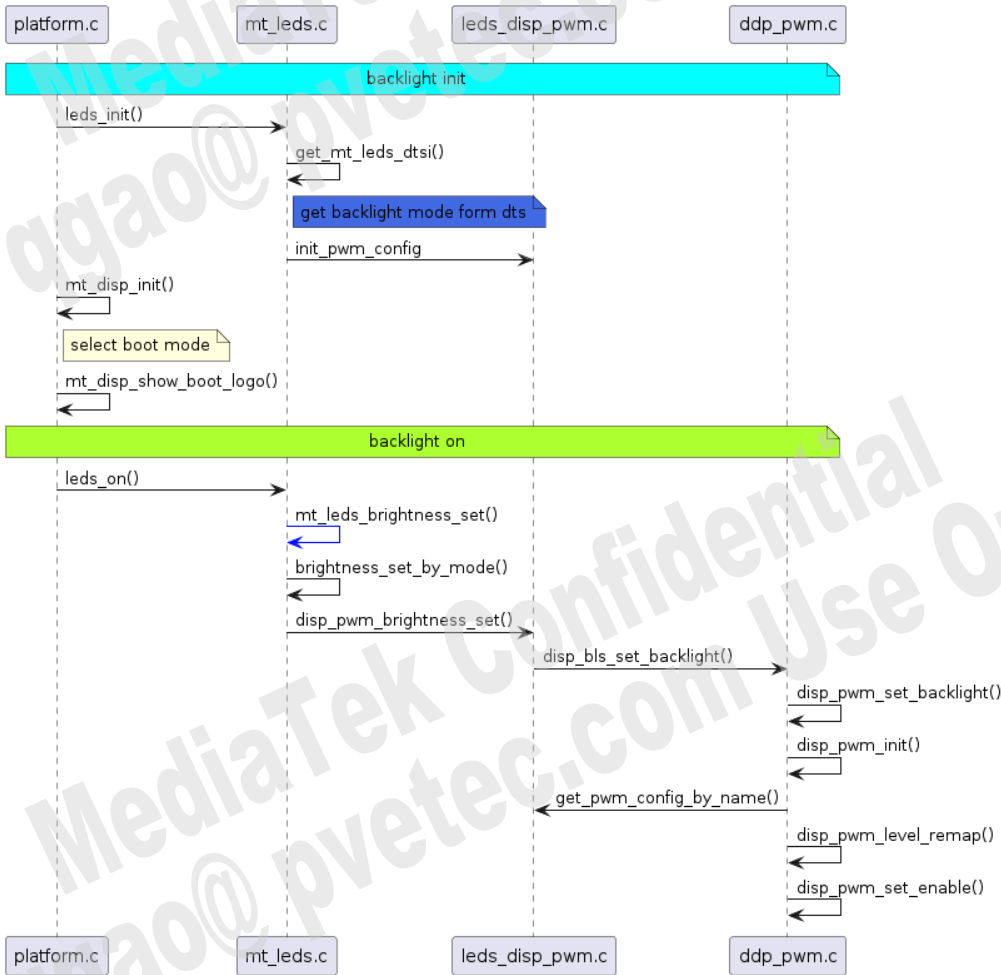
disp_pwm1: disp_pwm1@1100f000 {
    compatible = "mediatek,disp_pwm0",
                "mediatek,mt6881-disp_pwm0";
    reg = <0 0x1100f000 0 0x1000>;
    interrupts = <GIC_SPI 254 IRQ_TYPE_LEVEL_HIGH 0>;
    #pwm-cells = <2>;
    clocks = <&cksys_clk CLK CK_DISP_PWM_SEL>;
    clock-names = "main";
    status = "okay";
};

```

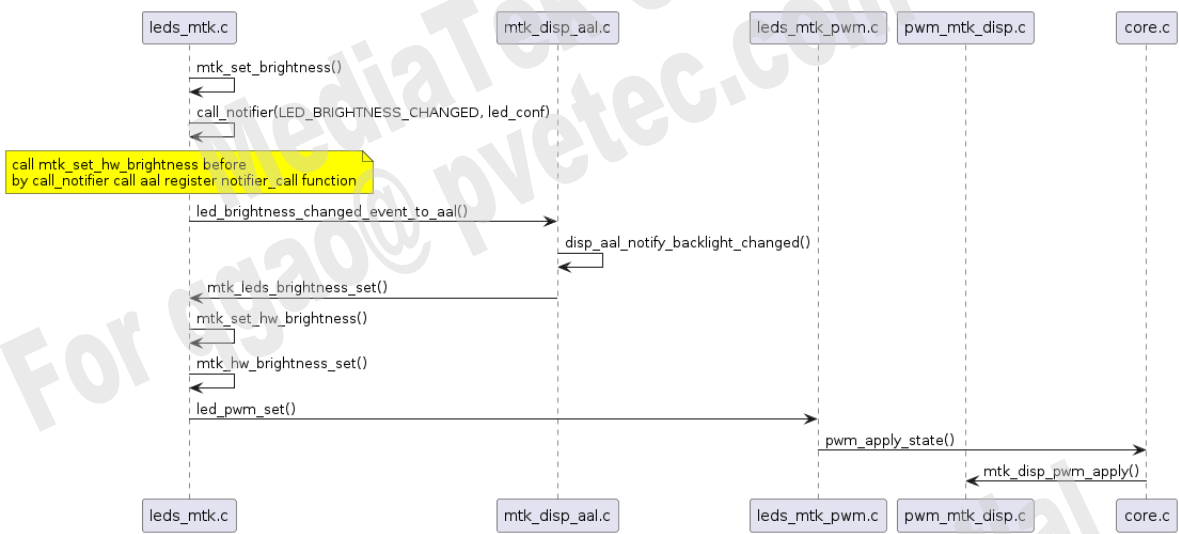
序号	解释
①	第 1 路背光的节点
②	允许用户设置的最大背光值
③	最大硬件背光值，背光 IC 支持到 255 级则为 255 (若对亮度级没有特别要求则保持默认值)
④	PWM 的控制方式： 4: LCM PWM 5: DISP_PWM 6: I2C PWM
⑤	格式为: <&Source Channel Period_ns> Source: disp_pwm 来源，如 disp_pwm，disp_pwm1 需要注意确认 dts 中 source 的节点是否有打开 Channel: disp_pwm 中未使用，无需配置 peroid_ns: 即输出 PWM 波形的周期(单位：ns)，即 PWM 频率的倒数
⑥	仅 LK 使用 ，格式为<clock_source div>，后 3 个参数 disp_pwm 没有使用。 0=26M, div=1，默认输出 12.695kHz。div 范围 0~255，固定亮度值输出。 $\text{pwm 输出频率} = \frac{26 * 10^6}{1024 * (div + 1)}$ 当 Kernel 起来之后，会按照⑤中设定的 PWM 频率输出。
⑦	第 2 路 leds 节点，用的 disp_pwm1 控制背光

3.5.3 disp_pwm LK 流程

LK 的背光亮度和 PWM 频率是一个固定的值。其中 PWM 频率默认为 12.965kHz，调整范围有限。



3.5.4 disp_pwm Kernel 流程



4 DSI Yocto Porting 指南

System	Branch	Kernel Version
Yocto	master-spm	Kernel-6.12

4.1 Yocto 通用文件路径

文件	路径
LK2	src\bsp\lk2\
Kernel	src\Kernel\linux\v6.12_mt8668\co_device_modules\
meta	Meta\meta-mediatek-mt8668\
Kernel DTS	Src/kernel/linux/v6.12_mt8668/co_device_modules/arch/arm64/boot/dts/mediatek/
Kernel config	Src/kernel/linux/v6.12_mt8668/co_device_modules/arch/arm64/configs/mgk_64_k612_defconfig
Kernel kleaf	Src/kernel/linux/v6.12_mt8668/co_device_modules/kernel/kleaf/mgk_64.bz1
KO Order Table	meta/meta-mediatek-mt8668/recipes-kernel/linux/ko_order_table/auto8668p1_64/ko_order_table.csv
dws	src\devtools\dot\dws\mt6881\auto8668p1_64.dws

注：对于 DSI Panel Driver，Yocto 与 Android 的差异主要是文件路径和 project 的差异，Driver 部分是一样的。

4.2 LK & Kernel Driver Porting

LK 部分的 Driver 主要用于 BootLogo 的显示，DSI0 可以在 LK 显示 BootLogo，DSI1 没有支持 LK 显示，默认从 Kernel 启动。Yocto 部分 LK & kernel driver 与 android 部分一致，只有文件路径或者 project 的差异。其它部分请参考 Android 的部分。

4.2.1 DSI0 BootLogo 设定

首先到如下路径找到当前屏幕大小所对应的 Logo 资源文件夹，如果没有所需大小的资源文件夹，也可以修改裁剪 LK Logo 对应当前使用屏幕的分辨率，然后到如下路径中新增一个图片资源，添加方法如下：

注：Hyper 双系统只会在 Yocto 端有 Logo，Android 端是不会有 LK Logo 的！

1. 在下面目录下新增或查找图片资源文件

`src/apps/atom-base/progs/makelogo`

注：新增的图片资源以目录名为前缀，可参考其他目录。

2. 将对应当前使用的 `conf` 文件中的 `BOOT_LOGO config` 修改配置为上方找到的或新增的图片目录名。

`meta/meta-mediatek-mt8668/conf/machine/{project}.conf`

公版默认都是配置在 `meta/meta-mediatek-mt8668/conf/machine/auto8668p1_64.conf` 中，因为其他 `project` 都会继承这个 `project`。

3. 因当前公版支持多分辨率 Logo 功能，所以对应的 `BOOT_LOGO` 配置的是 `uhd`，该配置下，将会 build in 多个不同分辨率的 `uboot Logo` 资源，以供显示时根据当前 `Display` 对应的屏幕宽、高来选择不同分辨率 Logo 的索引值 (`index`) 进行 Show。

上述的 Logo Index，是需要在 `makelogo` 目录下的 `rules.mk` 中对应分辨率图片资源添加到 `RESOURCE_OBJ_LIST` 中的顺序而定，同时也需要将 `add` 的新图片对应添加到 `bmp_to_raw` 操作中。

对应的 Logo 显示相关源代码路径：

`src/bsp/lk2/platform/mediatek/common/logo`

`src/bsp/lk2/lib/libshowlogo`

5 DSI Hypervisor Porting 指南

System	Branch	Kernel Version
Android B	alps-mp-b0.mp1	Kernel-6.12
Yocto	master-spm	Kernel-6.12

5.1 Hypervisor SW 架构

多系统时，Yocto 为 host 端，Android 为 Server 端。DSI 的 Driver 以及 SerDes 的 Driver 都会放在 Yocto 端，Android 端会通过虚拟化实现，并没有实体的 DSI 和 SerDes Driver。所以在 Yocto 端多系统与单 Yocto 的差异为不同的 Project。当 Porting Hypervisor Project 时候，需要注意 Project 部分的差异，其他部分 Driver Porting 请参考第 4 章。

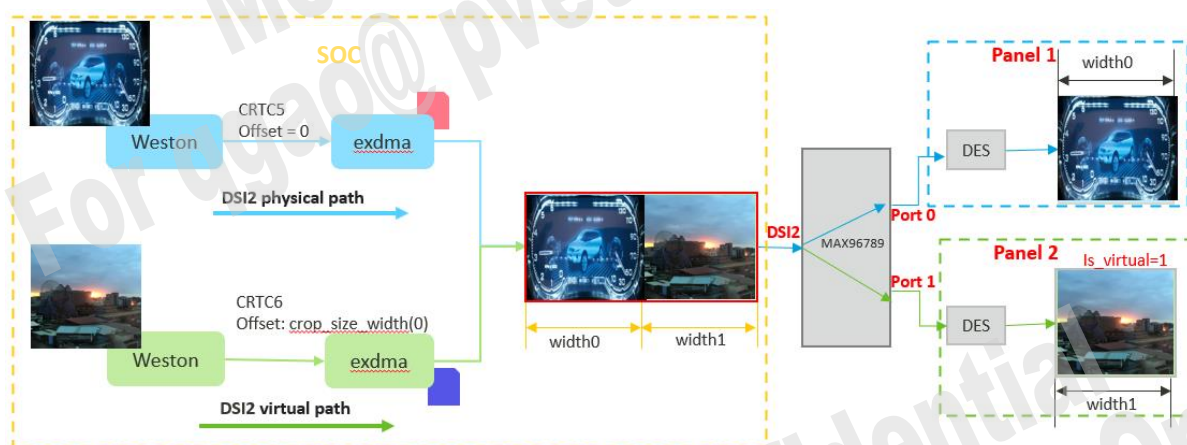
6 DSI Superframe Porting 指南

DSI 支持超级帧，可以实现一个 DSI Port 输出两个屏幕的画面，从而支持更多的屏幕。超级帧包含等高和不等高两种情况。

6.1 Superframe 概念

SoC 将两个屏幕的画面 Side by Side 拼成一张大帧的画面送给 DSI Port，DSI Port 再输出给加串器，加串器把收到的数据分割成两个屏幕的画面，分别送给不同的解串器显示在不同的屏幕上。**对于不等高的 superframe 的支持需要提前做 SoC, SerDes, 屏厂的 3 方评估。**不等高 superframe 需要 SoC 按照 SerDes 可以支持的比例做小图（高度小的）的插值，插值后会是一张与大图等高的图，再与大图做左右拼接后通过 DSI 输出。加串器收到拼接后的图除了分割，还需要将小图中的数据做过滤，丢掉插入的 dummy 数据。再分别送给解串器显示。所以需要确认大屏和小屏的高度比例，SerDes 是否可以拆分的同时 SoC 是否可以插值。另外由于 DSI 输出的 V blanking timing 只有一种，需要确认屏幕 Blanking Timing 调整后是否超出了规格。

6.2 Superframe 架构



6.3 Superframe DTS 设定

Superframe DTS 设定包含 4 部分：Physical 的 DSI 节点，Virtual 的 DSI 节点，加串器的节点还有 Superframe 的设定的节点。

6.3.1 Physical DSI 节点

与单屏部分设定一致，可以参考章节 3.3.5.1。

```
&dsi0 {
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
    panel1 {
        compatible = "boe,jd9365da";
        reg = <0>;
        power-gpios = <&pio 195 0>;
        reset-gpios = <&pio 60 0>;
        pinctrl-names = "default";
        port {
            panel1_in: endpoint {
                remote-endpoint = <&dsi_out>;
            };
        };
    };

    ports {
        port {
            reg = <0>;
            dsi_out: endpoint {
                remote-endpoint = <&max96789_in>;
            };
        };
    };
};

&mipi_tx_config0 {
    status = "okay";
};
```

6.3.2 Virtual DSI 节点

virt_dsi0 是虚拟出一路 DSI，DSI 的控制还是在 dsi0 上。另外 port 指向 max96789_v0。

```
&virt_dsi0 {①
    reg = <0>;
    ports {
        port {
            dsiv0_out: endpoint {
                remote-endpoint = <&max96789_v0_in>;②
            };
        };
    };
};
```

序号	释义
①	Virtual path virt_dsi0 的节点
②	指向虚拟的 max96789 节点

6.3.3 加串器节点

加串器也会虚拟出一个 virtual 的节点，同时 master 配置为实体的 max96789 节点。

```
&i2c10 {
    status = "okay";

    max96789: max96789@40 {
        compatible = "maxim,max96789,dsi0";
        status = "disabled";
        reg = <0x40>;
        reset-gpios = <&pio 98 0>;
        // only for test
        //interrupt-parent = <&pio>;
        //interrupts = <187 IRQ_TYPE_EDGE_RISING>;
        pinctrl-names = "default";
        setting = <&huayang_dsi_superframe>; ①
        port {
            max96789_in: endpoint {
                remote-endpoint = <&dsi_out>;
            };
        };
    };

    max96789_v0: max96789-v0@4a { ②
        compatible = "maxim,max96789,dsi,virtual";
        status = "disabled";
        reg = <0x4a>;
        master = <&max96789>; ③
        port {
            max96789_v0_in: endpoint {
                remote-endpoint = <&dsiv0_out>;
            };
        };
    };
};
```

序号	释义
①	配置为超级帧的屏幕参数
②	虚拟的 max96789 节点

序号	释义
①	配置为超级帧的屏幕参数
③	保存实体的 max96789 节点

6.3.4 Superframe Setting 节点

Superframe Setting 与其他 Panel 的 Setting 一样，都挂在 max96789 的节点下，需要将 superframe 的 flag 设为 1 给 driver 使用。

```
&max96789 {
    huayang_dsi_superframe: config2 { ①
        superframe = <1>; ②

        pre-init-cmd = <③
            0x40 0x10 0x0002 0x73 0x00
            0x40 0x10 0x0053 0x10 0x00
            0x40 0x10 0x0057 0x21 0x00
            0x40 0x10 0x0332 0x4e 0x00
            0x40 0x10 0x0333 0xe4 0x00
            0x40 0x10 0x0004 0xf2 0x00
            0x40 0x10 0x0308 0x5c 0x00
            0x40 0x10 0x0311 0x03 0x00
            0x40 0x10 0x0331 0x03 0x00
            0x40 0x10 0x0330 0x06 0x00
            0x40 0x10 0x031c 0x98 0x00
            0x40 0x10 0x0321 0x24 0x00
            0x40 0x10 0x031d 0x98 0x00
            0x40 0x10 0x0322 0x24 0x00
            0x40 0x10 0x0326 0xe4 0x00
            0x40 0x10 0x03a4 0xc1 0x00
            0x40 0x10 0x032a 0x07 0x00
            0x40 0x10 0x0002 0x73 0x05
            0x40 0x10 0x02dc 0x04 0x00
            0x40 0x10 0x02dd 0xaa 0x00
            0x40 0x10 0x02de 0x6a 0x00
            0x40 0x10 0x02d9 0x04 0x00
            0x40 0x10 0x02da 0xa9 0x00
            0x40 0x10 0x02db 0x69 0x00
            0x40 0x10 0x0385 0x50 0x00
            0x40 0x10 0x0386 0x02 0x00
            0x40 0x10 0x0387 0x00 0x00
            0x40 0x10 0x03a5 0x18 0x00
            0x40 0x10 0x03a7 0x00 0x00
            0x40 0x10 0x03a6 0xa0 0x00
            0x40 0x10 0x03a8 0x38 0x00
            0x40 0x10 0x03a9 0x04 0x00
        >
```

```

0x40 0x10 0x03aa 0x50 0x00
0x40 0x10 0x03ac 0x0a 0x00
0x40 0x10 0x03ab 0x00 0x00
0x40 0x10 0x03ad 0x00 0x00
0x40 0x10 0x03ae 0x0f 0x00
>;
i2c-remap-cmd = <④
0x40 0x10 0x0010 0x21 0x64
0x4c 0x10 0x0000 0x90 0x00
0x48 0x10 0x0073 0x31 0x00
0x48 0x10 0x0042 0x36 0x00
0x48 0x10 0x0043 0x34 0x00
0x48 0x10 0x0044 0x62 0x00
0x48 0x10 0x0045 0x28 0x00
0x40 0x10 0x0010 0x22 0x64
0x4c 0x10 0x0000 0x94 0x00
0x4a 0x10 0x0073 0x32 0x00
0x4a 0x10 0x0042 0x38 0x00
0x4a 0x10 0x0043 0x34 0x00
0x4a 0x10 0x0044 0x64 0x00
0x4a 0x10 0x0045 0x28 0x00
0x40 0x10 0x0010 0x23 0x64
0x4a 0x10 0x0050 0x01 0x00
>;
post-init-cmd = <>;⑤
linka-init-cmd = <⑥
0x48 0x10 0x01ce 0x4c 0x00
0x48 0x10 0x06ff 0x11 0x00
0x48 0x10 0x020c 0x03 0x00
0x48 0x10 0x020d 0xaa 0x00
0x48 0x10 0x020e 0x4a 0x00
0x1b 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00
0xd4
>;
linkb-init-cmd = <⑦
0x4a 0x10 0x01ce 0x4c 0x00
0x4a 0x10 0x06ff 0x11 0x00
0x4a 0x10 0x020c 0x03 0x00
0x4a 0x10 0x020d 0xaa 0x00
0x4a 0x10 0x020e 0x4a 0x00
0x1c 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00
0xd4
>;
linka-deinit-cmd = <⑧
0x1b 0x00 0x0b 0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00
0x85
>;

```

```

linkb-deinit-cmd = <9>
                0x1c 0x00 0x0b 0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00
0x85
>;

ser-status = <10>
            0x40 0x10 0x0d 0x80 0x80
            0x40 0x10 0x102 0x80 0x80
            0x40 0x10 0x55d 0x70 0x70
>;

linka-status = <11>
            0x40 0x10 0x1f 0x08 0x08
            0x48 0x10 0x6ff 0x11 0x11
>;

linkb-status = <12>
            0x40 0x10 0x1f 0x10 0x10
            0x4a 0x10 0x6ff 0x11 0x11
>;

panel-timing-a { 13
    width = <1920>;
    height = <1080>;
    hfp = <40>;
    hsa = <41>;
    hbp = <79>;
    vfp = <24>;
    vsa = <3>;
    vbp = <9>;
    fps = <60>;
    width-mm = <129>;
    height-mm = <64>;
};

panel-timing-b { 14
    width = <1920>;
    height = <1080>;
    hfp = <40>;
    hsa = <41>;
    hbp = <79>;
    vfp = <24>;
    vsa = <3>;
    vbp = <9>;
    fps = <60>;
    width-mm = <129>;
    height-mm = <64>;
};
};

```

序号	释义
①	superframe Setting 节点
②	是否是 superframe, superframe 设为 1
③	加串器 max96789 超级帧的 pre 初始化设定。
④	当同款屏幕 des I2C 地址或者 touch 地址冲突的时候, 需要将 I2C 地址做 remap
⑤	Post 初始化序列, 如果有需要就填入, 没有可以删除
⑥	Linka 的屏幕 (DSI0 对应的屏幕) des 和背光等初始化序列
⑦	Linkb 的屏幕 (virt_DSI0 对应的屏幕) des 和背光等初始化序列
⑧	Linka deinit 序列
⑨	Linkb deinit 序列
⑩	检测 ser 状态, 是读 i2c 命令, 注意只支持如下 3 条命令并且要按顺序填写 0x40 0x10 0x00d 0x80 0x80 # 串行器是否能 detect 到 0x40 0x10 0x102 0x80 0x80 # 96789 检查 PCLK 是否 detect 到 0x40 0x10 0x55d 0x70 0x70 # 96789 检查 HS/VIS/DE 是否 detect 到 如果其他的 ser 需要客制化这部分寄存器
⑪	检测 linka 状态, 注意只支持两条命令且要保持顺序, 第一条检测是否连接, 第二条检测是否被初始化 0x40 0x10 0x01f 0x08 0x08 # max96789 检查 linka 是否有接入 0x48 0x10 0x6ff 0x11 0x11 # max96789 检查 linka 是否有被初始化, 通过往一个用不到且能读写的 reg 里写任意值再读出来判断它是否被初始化, 可以根据情况客制化
⑫	检测 linkb 状态, 同 linka
⑬	Linka 接的屏幕的 timing
⑭	Linkb 接的屏幕的 timing

6.4 如何关闭 Superframe

如果 DSI0 不需要输出超级帧, 只需要输出单屏。可以将 dts 中 virt_dsi0 的节点关掉, 同时将加串器的节点中 Setting 改为单屏的设定, 或者兼容屏幕的设定。

```
&max96789_1 {
    status = "okay";
    setting = <&huayang_1080p>;
};
```

```
&virt_dsi0 {
    status = "disabled";
};
&max96789_v0 {
    status = "disabled";
};
```

6.5 DSI Superframe Timing

DSI 在超级帧的时候输出的是两个屏幕的 Size，DSI Timing 需要按照两个屏幕的设定，目前的 Code Flow 中已经在超级帧的时候把 desa 和 desb 的 Panel Timing 横向做了相加。送给 Display 的 Timing 需要的是单个屏幕的 timing。

6.5.1 Panel to DSI Timing Function

```
static struct mtk_panel_funcs ext_funcs = {
    .get_real_vdo_timing = serdes_panel_get_real_vdo_timing,
    .get_link_status = serdes_panel_get_link_status,
};
```

```
static int serdes_get_real_timing(struct serdes_bridge *serdes,
    struct drm_display_mode *mode)
{
    struct serdes_bridge *ser_des = serdes->driver_data->port ? serdes->master_serdes : serdes;
    int port = serdes->driver_data->port;

    if (serdes->link_type == LINK_TYPE_SUPERFRAME) {
        mode->hdisplay = ser_des->panel_timing[0].width
            + ser_des->panel_timing[1].width;
        mode->hsync_start = mode->hdisplay +
            ser_des->panel_timing[0].hfp + ser_des->panel_timing[1].hfp;
        mode->hsync_end = mode->hsync_start
            + ser_des->panel_timing[0].hsa + ser_des->panel_timing[1].hsa;
        mode->htotal = mode->hsync_end
            + ser_des->panel_timing[0].hbp + ser_des->panel_timing[1].hbp;

        mode->vdisplay = ser_des->panel_timing[0].height;
        mode->vsync_start = mode->vdisplay + ser_des->panel_timing[0].vfp;
        mode->vsync_end = mode->vsync_start + ser_des->panel_timing[0].vsa;
        mode->vtotal = mode->vsync_end + ser_des->panel_timing[0].vbp;

        mode->clock = mode->htotal * mode->vtotal * ser_des->panel_timing[0].fps / 1000;
    } else {
```

6.5.2 Panel to Display Function

```
static const struct drm_panel_funcs lcm_drm_funcs = {
    .disable = serdes_panel_disable,
    .unprepare = serdes_panel_unprepare,
    .prepare = serdes_panel_prepare,
    .enable = serdes_panel_enable,
    .get_modes = serdes_panel_get_modes,
};

int serdes_bridge_get_modes(struct drm_bridge *bridge,
                           struct drm_display_mode *mode)
{
    mode->hdisplay = ser_des->panel_timing[port].width;
    mode->hsync_start = mode->hdisplay + ser_des->panel_timing[port].hfp;
    mode->hsync_end = mode->hsync_start + ser_des->panel_timing[port].hsa;
    mode->htotal = mode->hsync_end + ser_des->panel_timing[port].hbp;

    mode->vdisplay = ser_des->panel_timing[port].height;
    mode->vsync_start = mode->vdisplay + ser_des->panel_timing[port].vfp;
    mode->vsync_end = mode->vsync_start + ser_des->panel_timing[port].vsa;
    mode->vtotal = mode->vsync_end + ser_des->panel_timing[port].vbp;
    mode->clock = mode->htotal * mode->vtotal * ser_des->panel_timing[port].fps / 1000;

    pr_info("%s[i2c%d]: Display timing: %dx%d@%d [%d,%d,%d %d,%d,%d]\n", __func__,
            ser_des->client->adapter->nr,
            mode->hdisplay, mode->vdisplay, ser_des->panel_timing[port].fps,
            mode->hsync_start, mode->hsync_end, mode->htotal,
            mode->vsync_start, mode->vsync_end, mode->vtotal);
    model = drm_mode_duplicate(connector->dev, &mode);
}
```

6.5.3 Log 确认 Timing 信息

```
5.000501][T600829] modprobe: get_real_timing[panell-2]: panel0 timing[1920x1080@60]
5.001534][T600829] modprobe: get_real_timing[panell-2]: panel0 timing[112 24 32][16 3 24]
5.002636][T600829] modprobe: get_real_timing[panell-2]: panel0 timing[112 24 32][16 3 24]
5.003669][T600829] modprobe: get_real_timing[panell-2]: panel0 timing[112 24 32][16 3 24]
5.004775][T600829] modprobe: get_real_timing: to display timing: 1920x1080@60Hz, clock=140690
5.005923][T600829] modprobe: get_real_timing: to display blank: HFP/HSA/HBP=2032/2056/2088, VFP/VSA/VBP=1096/1099/1123
5.007356][T600829] modprobe: get_real_timing: to dsi blank: 3840x1080@60Hz, PLL=845
5.008403][T600829] modprobe: get_real_timing: to dsi blank: HFP/HSA/HBP=224/48/64, VFP/VSA/VBP=16/3/24
```

需要确认 Log 中 Display Timing 为单屏的 Timing，DSI Timing 为两个屏幕的 Timing，设定才是正确的。如果不是这样，请检查设定。

6.6 Superframe SerDes 控制

超级帧的时候 SerDes driver 中会将指向 dsi0 节点的 driver 设定为 port = 0，virtual 的节点设定为 port = 1，从而实现两个 des 和屏幕的控制。如果增加其它 SerDes 的 compatible，注意 driver data 部分也要设对，不要随意更改。

```

static const struct serdes_driver_data serdes_dsi0_data = {
    .port = 0,
    .type = SERDES_FOR_DSI,
    .init_flag_mask = (1 << 0),
};
static const struct serdes_driver_data serdes_dsi1_data = {
    .port = 0,
    .type = SERDES_FOR_DSI,
    .init_flag_mask = (1 << 4),
};
static const struct serdes_driver_data serdes_dsi2_data = {
    .port = 0,
    .type = SERDES_FOR_DSI,
    .init_flag_mask = (1 << 5),
};
static const struct serdes_driver_data serdes_dsi_data_virtual = {
    .port = 1,
    .type = SERDES_FOR_DSI,
};

```

```

static const struct of_device_id serdes_iic_match[] = {
    {.compatible = "maxiam,max96789,dsi0", .data = &serdes_dsi0_data},
    {.compatible = "maxiam,max96789,dsi1", .data = &serdes_dsi1_data},
    {.compatible = "maxiam,max96789,dsi2", .data = &serdes_dsi2_data},
    {.compatible = "maxiam,max96789,dsi,virtual", .data = &serdes_dsi_data_virtual},
    {.compatible = "maxiam,max96851,edp", .data = &serdes_edp_data},
    {.compatible = "maxiam,max96851,edp,virtual", .data = &serdes_edp_data_virtual},
    {.compatible = "maxiam,max96851,dp", .data = &serdes_dp_data},
    {},
};

```

另外 virtual 的节点中要设定 master 的节点，virtual 部分的 driver 的控制都会指向 master 节点。

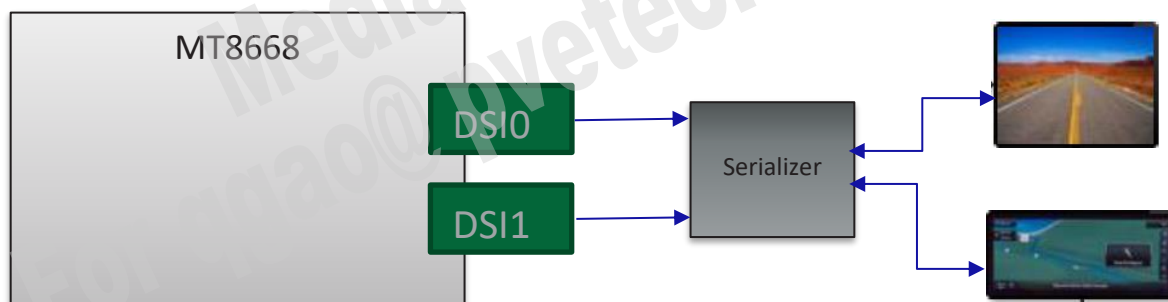
```

master_node = of_parse_phandle(dev->of_node, MASTER_NODE_NAME, 0);
if (!master_node) {
    pr_info("%s[%d]: ERROR: master node is empty!\n", __func__, client->adapter->nr);
    return -1;
}
bridge = of_drm_find_bridge(master_node);
if (!bridge) {
    pr_info("%s[%d]: waiting for PHY bridge!\n", __func__, client->adapter->nr);
    return -EPROBE_DEFER;
}
ser_des->master_serdes = bridge_to_serdes(bridge);
ser_des->link_type = ser_des->master_serdes->link_type;

```

7 DSI Dual-link Porting 指南

什么是 dual-link 场景？



如上图所示，当 2 个 DSI port 的数据都送给同一个加串器，然后加串器将数据送给不同的屏幕显示。也就是说加串器需要支持 2 个 DSI 的输入（两进两出）。这个场景适用于屏幕分辨率不高，单 link 可以支持，需要节省成本且屏幕数量少的场景。

7.1 Driver 中要如何配置

以下以 DSI0 & DSI1 使用同一个 max96789 为例。

7.1.1 Physical DSI0 节点

与单屏部分设定一致，可以参考章节 3.3.5.1。

```
&dsi0 {
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
    panel1 {
        compatible = "boe,jd9365da";
        reg = <0>;
        power-gpios = <&pio 195 0>;
        reset-gpios = <&pio 60 0>;
        pinctrl-names = "default";
        port {
            panel1_in: endpoint {
                remote-endpoint = <&dsi_out>;
            };
        };
    };
};
```

```
ports {
    port {
        reg = <0>;
        dsi_out: endpoint {
            remote-endpoint = <&max96789_in>;
        };
    };
};

&mipi_tx_config0 {
    status = "okay";
};
```

7.1.2 Physical DSI1 节点

dsi1 的节点，port 指向 max96789_v0。

```
&dsi1 { ①
    reg = <0>;
    ports {
        port {
            dsi1_out: endpoint {
                remote-endpoint = <&max96789_v0_in>; ②
            };
        };
    };
};

&mipi_tx_config1 {
    status = "okay";
};
```

序号	释义
①	Dsi1 的节点
②	指向虚拟的 max96789 节点

7.1.1 加串器节点

加串器会虚拟出一个 virtual 的节点，同时 master 配置为实体的 max96789 节点。并且 port 指向 dsi1 节点。

```
&i2c10 {
    status = "okay";

    max96789: max96789@40 {
        compatible = "maxim,max96789,dsi0";
        status = "disabled";
    };
};
```

```
reg = <0x40>;
reset-gpios = <&pio 98 0>;
// only for test
//interrupt-parent = <&pio>;
//interrupts = <187 IRQ_TYPE_EDGE_RISING>;
pinctrl-names = "default";
setting = <&dsi_dualink>; ①
port {
    max96789_in: endpoint {
        remote-endpoint = <&dsi_out>;
    };
};

max96789_v0: max96789-v0@4a { ②
    compatible = "maxim,max96789,dsi,virtual";
    status = "disabled";
    reg = <0x4a>;
    master = <&max96789>; ③
    port {
        max96789_v0_in: endpoint {
            remote-endpoint = <&dsi1_out>; ④
        };
    };
};
```

序号	释义
①	配置为 dual-link 的屏幕参数
②	虚拟的 max96789 节点
③	保存实体的 max96789 节点
④	虚拟的加串器节点指向 dsi1 的节点

7.1.2 Dual-link Setting 节点

Dual-link Setting 与其他 Panel 的 Setting 一样，都挂在 max96789 的节点下，需要将 dual-link 的 flag 设为 1 给 driver 使用。

```
&max96789 {
    dsi_dualink: config3 { ①
        dual-link = <1>; ②
    };
};
```

```
pre-init-cmd = <③
```

```

0x40 0x10 0x0002 0x73 0x00
0x40 0x10 0x0053 0x10 0x00
0x40 0x10 0x0057 0x21 0x00
0x40 0x10 0x0332 0x4e 0x00
0x40 0x10 0x0333 0xe4 0x00
0x40 0x10 0x0004 0xf2 0x00
0x40 0x10 0x0308 0x5c 0x00
0x40 0x10 0x0311 0x03 0x00
0x40 0x10 0x0331 0x03 0x00
0x40 0x10 0x0330 0x06 0x00
0x40 0x10 0x031c 0x98 0x00
0x40 0x10 0x0321 0x24 0x00
0x40 0x10 0x031d 0x98 0x00
0x40 0x10 0x0322 0x24 0x00
0x40 0x10 0x0326 0xe4 0x00
0x40 0x10 0x03a4 0xc1 0x00
0x40 0x10 0x032a 0x07 0x00
0x40 0x10 0x0002 0x73 0x05
0x40 0x10 0x02dc 0x04 0x00
0x40 0x10 0x02dd 0xaa 0x00
0x40 0x10 0x02de 0x6a 0x00
0x40 0x10 0x02d9 0x04 0x00
0x40 0x10 0x02da 0xa9 0x00
0x40 0x10 0x02db 0x69 0x00
0x40 0x10 0x0385 0x50 0x00
0x40 0x10 0x0386 0x02 0x00
0x40 0x10 0x0387 0x00 0x00
0x40 0x10 0x03a5 0x18 0x00
0x40 0x10 0x03a7 0x00 0x00
0x40 0x10 0x03a6 0xa0 0x00
0x40 0x10 0x03a8 0x38 0x00
0x40 0x10 0x03a9 0x04 0x00
0x40 0x10 0x03aa 0x50 0x00
0x40 0x10 0x03ac 0x0a 0x00
0x40 0x10 0x03ab 0x00 0x00
0x40 0x10 0x03ad 0x00 0x00
0x40 0x10 0x03ae 0x0f 0x00

```

```
>;
```

```
i2c-remap-cmd = <④
```

```

0x40 0x10 0x0010 0x21 0x64
0x4c 0x10 0x0000 0x90 0x00
0x48 0x10 0x0073 0x31 0x00
0x48 0x10 0x0042 0x36 0x00
0x48 0x10 0x0043 0x34 0x00
0x48 0x10 0x0044 0x62 0x00
0x48 0x10 0x0045 0x28 0x00
0x40 0x10 0x0010 0x22 0x64
0x4c 0x10 0x0000 0x94 0x00
0x4a 0x10 0x0073 0x32 0x00
0x4a 0x10 0x0042 0x38 0x00
0x4a 0x10 0x0043 0x34 0x00

```

```

0x4a 0x10 0x0044 0x64 0x00
0x4a 0x10 0x0045 0x28 0x00
0x40 0x10 0x0010 0x23 0x64
0x4a 0x10 0x0050 0x01 0x00
>;
post-init-cmd = <>; ⑤
linka-init-cmd = <⑥
0x48 0x10 0x01ce 0x4c 0x00
0x48 0x10 0x06ff 0x11 0x00
0x48 0x10 0x020c 0x03 0x00
0x48 0x10 0x020d 0xaa 0x00
0x48 0x10 0x020e 0x4a 0x00
0x1b 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00
0x00 0x00 0xd4
>;
linkb-init-cmd = <⑦
0x4a 0x10 0x01ce 0x4c 0x00
0x4a 0x10 0x06ff 0x11 0x00
0x4a 0x10 0x020c 0x03 0x00
0x4a 0x10 0x020d 0xaa 0x00
0x4a 0x10 0x020e 0x4a 0x00
0x1c 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00
0x00 0x00 0xd4
>;
linka-deinit-cmd = <⑧
0x1b 0x00 0x0b 0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00
0x00 0x00 0x85
>;
linkb-deinit-cmd = <⑨
0x1c 0x00 0x0b 0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00
0x00 0x00 0x85
>;
ser-status = <⑩
0x40 0x10 0x0d 0x80 0x80
0x40 0x10 0x102 0x80 0x80
0x40 0x10 0x55d 0x70 0x70
>;
linka-status = <⑪
0x40 0x10 0x1f 0x08 0x08
0x48 0x10 0x6ff 0x11 0x11
>;
linkb-status = <⑫
0x40 0x10 0x1f 0x10 0x10
0x4a 0x10 0x6ff 0x11 0x11

```

```
>;

panel-timing-a { ⑬
    width = <1920>;
    height = <1080>;
    hfp = <40>;
    hsa = <41>;
    hbp = <79>;
    vfp = <24>;
    vsa = <3>;
    vbp = <9>;
    fps = <60>;
    width-mm = <129>;
    height-mm = <64>;
};

panel-timing-b { ⑭
    width = <800>;
    height = <480>;
    hfp = <40>;
    hsa = <41>;
    hbp = <79>;
    vfp = <24>;
    vsa = <3>;
    vbp = <9>;
    fps = <60>;
    width-mm = <129>;
    height-mm = <64>;
};

};
```

序号	释义
①	Dual-link Setting 节点
②	是否是 dual-link， 设为 1
③	加串器 max96789 dual-link 的 pre 初始化设定
④	当屏幕 des I2C 地址 或者 touch 地址冲突的时候， 需要将 I2C 地址做 remap
⑤	Post 初始化序列， 如果有需要就填入， 没有可以删除
⑥	Linka 的屏幕（DSI0 对应的屏幕）des 和背光等初始化序列
⑦	Linkb 的屏幕（DSI1 对应的屏幕）des 和背光等初始化序列
⑧	Linka deinit 序列
⑨	Linkb deinit 序列
⑩	检测 ser 状态， 是读 i2c 命令， 注意只支持如下 3 条命令并且要按顺序填写 0x40 0x10 0x00d 0x80 0x80 # 串行器是否能 detect 到

序号	释义
	0x40 0x10 0x102 0x80 0x80 # 96789 检查 PCLK 是否 detect 到 0x40 0x10 0x55d 0x70 0x70 # 96789 检查 HS/VIS/DE 是否 detect 到 如果其他的 ser 需要客制化这部分寄存器
①.1	检测 linka 状态，注意只支持两条命令且要保持顺序，第一条检测是否连接，第二条检测是否被初始化 0x40 0x10 0x01f 0x08 0x08 # max96789 检查 linka 是否有接入 0x48 0x10 0x6ff 0x11 0x11 # max96789 检查 linka 是否有被初始化，通过往一个用不到且能读写的 reg 里写任意值再读出来判断它是否被初始化，可以根据情况客制化
①.2	检测 Linkb 状态，同 linka
①.3	Linka 接的屏幕的 timing
①.4	Linkb 接的屏幕的 timing

8 SerDes 调试指南

车机的平台中 DSI 输出大部分都是通过 SerDes 接屏幕显示，DSI 的屏幕显示异常时，需要与 SerDes 一起 Debug。

可以通过 SerDes Pattern 和寄存器等来辅助分析问题。SerDes I2C Register 的读写，可以通过 I2C-Tools 实现。这里总结的是我们公版在 max96789 和 max96752 的 Debug 上经验积累，以供参考。这部分是跟厂商 Debug 的记录，最终的解释请与厂商确认。

8.1 Ser-max96789 Debug Register

加串器 max96789 是 DSI RX，可以通过 RX 端收数据的情况来查看。

0x102 bit7 确认是否有收到 MIPI Clock，并且 Detect 到 Pixel Clock。如果 bit7 为 0，在点板阶段请先确认 HW 上 MIPI 信号接线是否正常，是否有做 Swap，pn 信号是否画反。

VIDEO_TX2 (0x102, 0x10A, 0x112, 0x11A)

BIT	7	6	5	4	3	2	1	0
Field	PCLKDET	DRIFT_ERR	OVERFLOW	FIFO_WARN	RSVD	LIM_HEART	RSVD	RSVD
Reset	0b0	0b0	0b0	0b0	0b1	0b0	0b1	0b0
Access Type	Read Only	Read Clears All	Read Clears All	Read Clears All		Write, Read		

BITFIELD	BITS	DESCRIPTION	DECODE
PCLKDET	7	PCLK Detection	0b0: Video transmit PCLK not detected 0b1: Video transmit PCLK detected
DRIFT_ERR	6	VID_TX PCLK Drift-Error Detection	0b0: Video transmit PCLK drift error not detected 0b1: Video transmit PCLK drift error detected
OVERFLOW	5	Overflow occurs when video input throughput is too high, or bandwidth-allocation on GMSL2 link for video is not sufficient.	0b0: Video transmit FIFO has not overflowed 0b1: Video transmit FIFO has overflowed
FIFO_WARN	4	VID_TX FIFO is more than half full	0b0: video transmit FIFO is less than or equal to half full 0b1: Video transmit FIFO is more than half full

0x55D 确认是否有收到 video 信号，并且 Detect 到 DE, H/V sync。

HS_VS_X (0x55D)

BIT	7	6	5	4	3	2	1	0
Field	-	DE_DET_X	VS_DET_X	HS_DET_X	-	-	VS_POL_X	HS_POL_X
Reset	-	0b0	0b0	0b0	-	-	0b0	0b0
Access Type	-	Read Only	Read Only	Read Only	-	-	Read Only	Read Only

BITFIELD	BITS	DESCRIPTION	DECODE
DE_DET_X	6	DE Activity is Detected on Channel X	0b0: DE is not detected 0b1: DE is detected
VS_DET_X	5	VS Activity is Detected on Channel X	0b0: VS is not detected 0b1: VS is detected
HS_DET_X	4	HS Activity is Detected on Channel X	0b0: HS is not detected 0b1: HS is detected
VS_POL_X	1	Detected VS Polarity on Channel X	0b0: Active low 0b1: Active high
HS_POL_X	0	Detected HS Polarity on Channel X	0b0: Active low 0b1: Active high

0x339, 0x33b, 0x33a, 0x33c 确认 PHY0/1 收到 LP 和 HS 的数据是否有 Error，寄存器读一次会清 0，如果有 Error 可以多读几次。

MIPI_RX9 (0x339)

BIT	7	6	5	4	3	2	1	0
Field	–	–	–	phy0_lp_err[4:0]				
Reset	–	–	–	0b00000				
Access Type	–	–	–	Read Clears All				

BITFIELD	BITS	DESCRIPTION	DECODE
phy0_lp_err	4:0	PHY0 LP Status	0bXXXX1: Unrecognized escape command received from data lane D0 0bXXXX1X: Unrecognized escape command received from CLK lane 0bXX1XX: Invalid line sequence detected from data lane D0 0bX1XXX: Invalid line sequence detected from data lane D1 0b1XXXX: Invalid line sequence detected from CLK lane

MIPI_RX10 (0x33A)

BIT	7	6	5	4	3	2	1	0
Field	phy0_hs_err[7:0]							
Reset	0x00							
Access Type	Read Clears All							

BITFIELD	BITS	DESCRIPTION	DECODE
phy0_hs_err	7:0	PHY0 High-Speed Status	0bXXXXXXXX1: HS sync pattern with one bit error detected on data lane D0 0bXXXXXXXX1X: HS sync pattern with one bit error detected on data lane D1 0bXXXXXXXX1XX: HS sync pattern with two or more bit errors detected on data lane D0 0bXXXXXXXX1XXX: HS sync pattern with two or more bit errors detected on data lane D1 0bXXX1XXXX: High speed receiver skew calibration failed on data lane D1 0bXX1XXXXX: High speed receiver skew calibration failed on data lane D0 0bX1XXXXXX: High speed receiver skew calibration run on data lane D1 0b1XXXXXXX: High speed receiver skew calibration run on data lane D0

0x3a0, 0x3a2 确认收到的 MIPI 输出是否有 ECC 和 CRC Error。

DSI32 (0x3A0)

BIT	7	6	5	4	3	2	1	0
Field	ctrl0_dsi_err_[7:0]							
Reset	0x00							
Access Type	Read Clears All							

BITFIELD	BITS	DESCRIPTION	DECODE
ctrl0_dsi_err_	7:0	DSI Controller 0 Status	0bXXXXXXXX1: DPI output dropped 0bXXXXXXXX1X: HS receiver timeout 0bXXXXXXXX1XX: Reserved 0bXXXXXXXX1XXX: ECC 1-bit error 0bXXX1XXXX: ECC error 0bXX1XXXXX: ECC two or more bits error 0bX1XXXXXX: CRC error detected

DSI34 (0x3A2)

BIT	7	6	5	4	3	2	1	0
Field	ctrl1_dsi_err_[7:0]							
Reset	0x00							
Access Type	Read Clears All							

BITFIELD	BITS	DESCRIPTION	DECODE
ctrl1_dsi_err_	7:0	DSI Controller 1 Status	0bXXXXXXXX1: DPI output dropped 0bXXXXXXXX1X: HS receiver timeout 0bXXXXXXXX1XX: Reserved 0bXXXXXXXX1XXX: ECC 1-bit error 0bXXX1XXXX: ECC error 0bXX1XXXXX: ECC two or more bits error 0bX1XXXXXX: CRC error detected

若以上寄存器如果都正常，说明 max96789 是有正确的收到 DSI 的数据，并且没有 Error。如果屏幕还是不亮需要确认屏幕的 Timing 设定是否正常，可以同时打一下 max96789 的 Pattern 给屏幕，来确认一下是否可以亮，如果 789 Pattern 也不亮，建议联系 maxin 和屏厂解决。

0x332, 0x333, 0x334, 0x335 确认 Lane Swap 和 p/n Swap 设定是否正常。

0x332	0xEE	MIPI_RX2[7:0]	phy1_lane_map[3:0]	phy0_lane_map[3:0]
0x333	0xE4	MIPI_RX3[7:0]	phy3_lane_map[3:0]	phy2_lane_map[3:0]
0x334	0x00	MIPI_RX4[7:0]	–	phy0_pol_map[2:0]
0x335	0x00	MIPI_RX5[7:0]	–	phy2_pol_map[2:0]

MIPI_RX3 (0x333)

BIT	7	6	5	4	3	2	1	0
Field	phy3_lane_map[3:0]				phy2_lane_map[3:0]			
Reset	0xE				0x4			
Access Type	Write, Read				Write, Read			
BITLEN	7:4	3:0						
DESCRIPTION	PHY3 to Port B Lane Mapping in 2x4 configuration. Works together with phy2_lane_map bitfield. PHY2 and PHY3 lane maps should be exclusive. Bits [5:4] map PHY3 data lane 0. Bits [7:6] map PHY3 data lane 1.	PHY2 to Port B Lane Mapping For 2x4 configuration, bits [1:0] map PHY2 data lane 0 to Port B. Bits [3:2] map PHY2 data lane 1 to Port B. These bits work together with phy3_lane_map bitfield. PHY2 and PHY3 lane maps should be exclusive.						
DECODE	0bXX00: PHY3 D0 is lane 0 0bXX01: PHY3 D0 is lane 1 0bXX10: PHY3 D0 is lane 2 0bXX11: PHY3 D0 is lane 3 0b00XX: PHY3 D1 is lane 0 0b01XX: PHY3 D1 is lane 1 0b10XX: PHY3 D1 is lane 2 0b11XX: PHY3 D1 is lane 3	0bXX00: PHY2 D0 is lane 0 0bXX01: PHY2 D0 is lane 1 0bXX10: PHY2 D0 is lane 2 0bXX11: PHY2 D0 is lane 3 0b00XX: PHY2 D1 is lane 0 0b01XX: PHY2 D1 is lane 1 0b10XX: PHY2 D1 is lane 2 0b11XX: PHY2 D1 is lane 3						

0x13, 0x1F 确认 GMSL Link Lock 状态。

CTRL3 (0x13)

BIT	7	6	5	4	3	2	1	0
Field	RSVD	RSVD	–	–	LOCKED	ERROR	CMU_LOCKED	–
Reset	0b0	0b0	–	–	0b0	0b0	0b0	–
Access Type			–	–	Read Only	Read Only	Read Only	–

BITFIELD	BITS	DESCRIPTION	DECODE
LOCKED	3	GMSL2 link lock status (bidirectional)	0b0: GMSL2 link not locked 0b1: GMSL2 link locked
ERROR	2	Reflects error status (inverse of ERRB pin value)	0b0: ERRB not asserted (ERRB pin = 1) 0b1: ERRB asserted (ERRB pin = 0)
CMU_LOCKED	1	Clock multiplier unit (CMU) lock status	0b0: CMU not locked 0b1: CMU locked

INTR7 (0x1F)

BIT	7	6	5	4	3	2	1	0
Field	VDDCMP_INT_FLAG	RSVD	VDDBAD_INT_FLAG	LOCK_B	LOCK_A	APRBS_B_ERR_FLAG	APRBS_A_ERR_FLAG	MIPI_ERR_FLAG
Reset	0b0	0b0	0b0	0b0	0b0	0b0	0b0	0b0
Access Type	Read Clears All		Read Clears All	Read Only	Read Only	Read Only	Read Only	Read Only

BITFIELD	BITS	DESCRIPTION	DECODE
VDDCMP_INT_FLAG	7	Combined V _{DD} comparator output Read this bitfield to clear.	0b0: Reporting disabled 0b1: Reporting enabled
VDDBAD_INT_FLAG	5	Combined V _{DD} BAD indicator Read this bitfield to clear.	0b0: Reporting disabled 0b1: Reporting enabled
LOCK_B	4	Link B Lock Status It is not possible to clear this interrupt. It keeps ERRB asserted when there is no link lock.	0b0: Flag not asserted 0b1: Flag asserted
LOCK_A	3	Link A Lock Status It is not possible to clear this interrupt. It keeps ERRB asserted when there is no link lock.	0b0: Flag not asserted 0b1: Flag asserted
		Audio B PRBS Error Flag	

0x3A4 可以关掉 max96789 的 DPI Deskew Bit0。

DSI36 (0x3A4)

BIT	7	6	5	4	3	2	1	0
Field	ctrl0_dpi_skew_fifo_size[1:0]		–	–	–	–	ctrl0_dpi_de_skew_sel	ctrl0_dpi_de_skew_en
Reset	0b00		–	–	–	–	0b0	0b0
Access Type	Write, Read		–	–	–	–	Write, Read	Write, Read
BITFIELD	BITS		DESCRIPTION		DECODE			
ctrl0_dpi_skew_fifo_size	7:6		DPI deskew FIFO size for DSI Controller 0. Choose larger FIFO size if the input has bigger skew.		0b00: Skew = ±1 PCLK cycle 0b01: Skew = ±3 PCLK cycle 0b10: Skew = ±7 PCLK cycle 0b11: Skew = ±15 PCLK cycle			
ctrl0_dpi_de_skew_sel	1		Selects skewed DE or non-skewed DE for DSI Controller 0.		0b0: Select the non-skewed DE and DOUT generated by the DPI deskew block 0b1: Select the skewed DE and DOUT output from the DSI controller			
ctrl0_dpi_deskew_en	0		Enables DPI deskew for DSI Controller 0.		0b0: Disable DPI deskew 0b1: Enable DPI deskew			

当画面出现水平方向偏移的时候，可以尝试将 deskew 开起来，并且同时需要配置 panel 的 timing 给 96789，这样 deskew 工作才正常。具体参数可以请美信来设定。

8.2 Des-max96752 Debug Register

0x108 确认是否有 Video Lock 和 Video PKT Detect

VIDEO_RX8 (0x108)

BIT	7	6	5	4	3	2	1	0
Field	VID_BLK_LEN_ERR	VID_LOCK	VID_PKT_DET	VID_SEQ_ERR	RSVD[3:0]			
Reset	0b0	0b0	0b0	0b0	0x2			
Access Type	Read Clears All	Read Only	Read Only	Read Clears All				

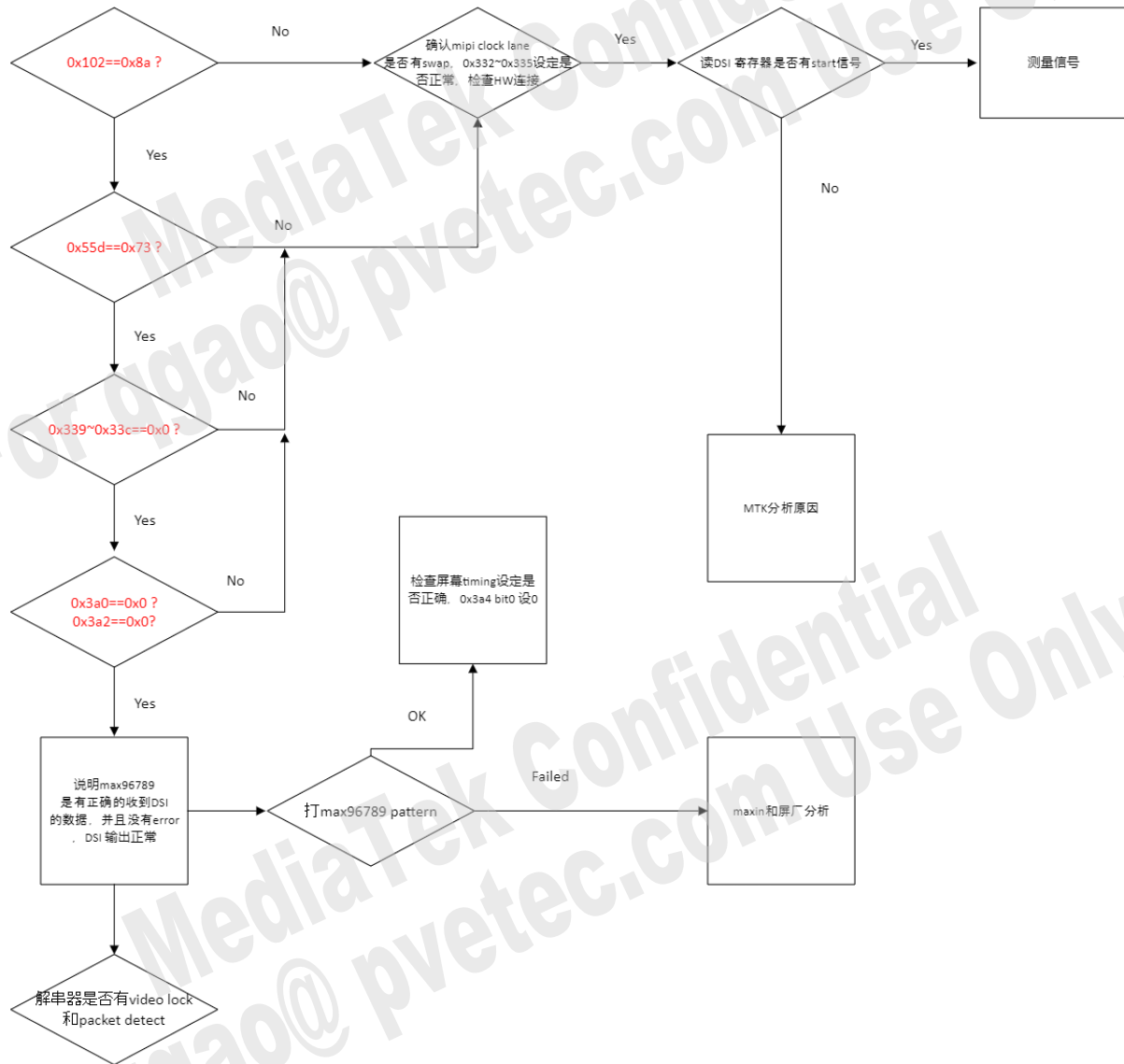
0x1ce 画面出现锯齿的时候可以尝试 Swap 一下。

OLDI1 (0x1CE)

BIT	7	6	5	4	3	2	1	0
Field	OLDI_OUT_SEL	OLDI_FOR_MAT	OLDI_4TH_LANE	OLDI_SWAP_AB	OLDI_SPL_EN	OLDI_SPL_MODE[1:0]		OLDI_SPL_POL
Reset	0x0	0x0	0x0	0x0	0x0	0x3		0x0
Access Type	Write, Read	Write, Read	Write, Read	Write, Read	Write, Read	Write, Read		Write, Read
BITFIELD	BITS		DESCRIPTION		DECODE			
OLDI_OUTS_EL	7		Selects which OLDI port to use for data output. Set this register when VID_LOCK = 0.		0b0: Port A 0b1: Port B			
OLDI_FORMAT	6		Output format of LVDS port. Set this register when VID_LOCK = 0.		0b0: OLDI 0b1: VESA			
OLDI_4TH_LANE	5		OLDI lane count. Set this register when VID_LOCK = 0.		0b0: Four lanes 0b1: Three lanes			
OLDI_SWAP_AB	4		Swaps OLDI ports A and B. Set this register when VID_LOCK = 0.		0b0: Ports A and B not swapped 0b1: Ports A and B swapped			
OLDI_SPL_EN	3		OLDI Splitter Enable. Set this register when VID_LOCK = 0.		0b0: Splitter disabled 0b1: Splitter enabled			
OLDI_SPL_MODE	2:1		OLDI Splitter Mode. Set this register when VID_LOCK = 0.		0b00: Random 0b01: Split with HS 0b10: Split with VS 0b11: Split with DE			
OLDI_SPL_POL	0		Set to 0 for dual OLDI applications. Set this register when VID_LOCK = 0.		0b0: Falling Edge 0b1: Rising Edge			

8.3 Debug 流程

下图是对以上寄存器说明和 Debug 方法的流程总结，可以按照这个流程来定位问题。



9 DSI 输出时序控制

对于 Bridge 或者是 SerDes, 会对 DSI 输出信号的时间点有要求, 例如需要在先做初始化, 再送 DSI 信号, 或者需要先送 DSI 信号, 再做初始化。本章节主要说明如何调整 LCM Driver 中初始化的时间点。

9.1 LK Boot 流程

默认的 Code Flow, `disp_lcm_init()` 会去调用 LCM Driver 中的 `init_power` 和 `init` 函数, 此时 DSI 没送信号。

```

primary_display.c > primary_display_init
int primary_display_init(void *fdt)
{
    data_config.lcm_bpp = 18;
}
data_config.dst_dirty = 1;
// (1) DSI config, DSI 还没出信号
ret = dpmgr_path_config(dp_handle: pgc->dpmgr_handle, config: &data_config, encmdq: CMDQ_DISABLE);
ret = disp_lcm_init(plcm: pgc->plcm, fdt); // (2) LCM driver->init_power & init

if (primary_display_is_video_mode()) {
    if (lcm_param->dsi_lfr_enable == 1)
        dpmgr_map_event_to_irq(dp_handle: pgc->dpmgr_handle, event: DISP_PATH_EVENT_IF_VSYNC,
                                irq_bit: DDP_IRQ_DSI0_FRAME_DONE);
    else
        dpmgr_map_event_to_irq(dp_handle: pgc->dpmgr_handle, event: DISP_PATH_EVENT_IF_VSYNC,
                                irq_bit: DDP_IRQ_RDMA0_DONE);
}

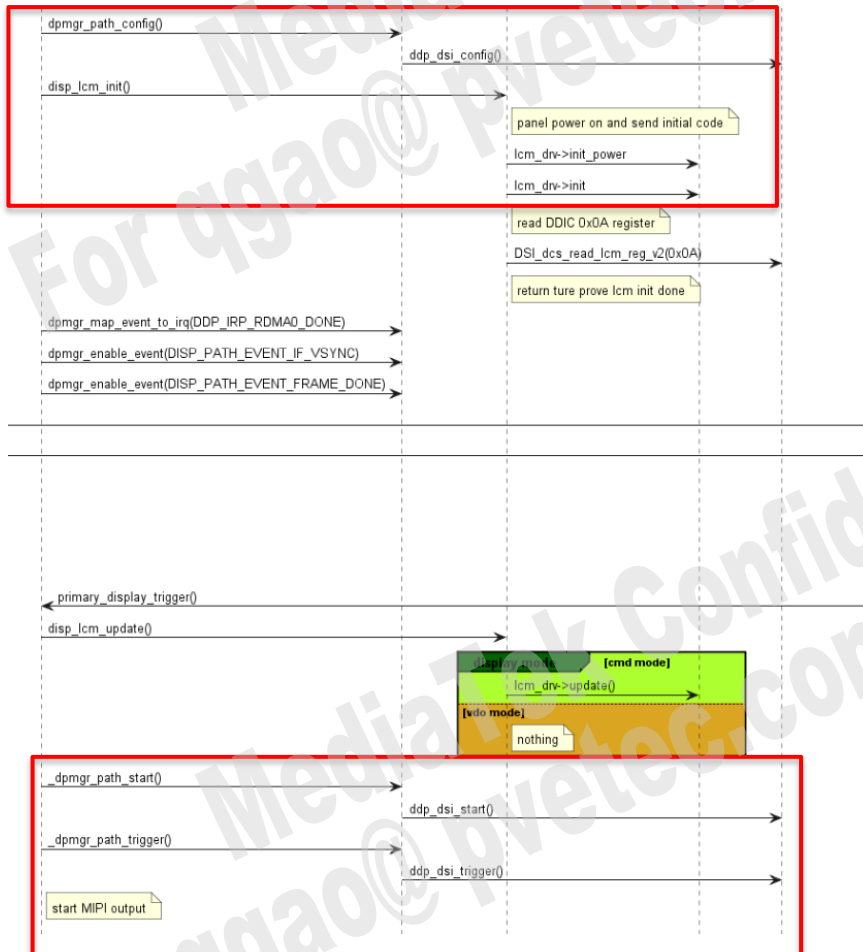
primary_display.c > primary_display_trigger
int primary_display_trigger(int blocking)
{
    x = disp_lcm_x(plcm: pgc->plcm);
    y = disp_lcm_y(plcm: pgc->plcm);
    width = disp_lcm_width(plcm: pgc->plcm);
    height = disp_lcm_height(plcm: pgc->plcm);

    if (pgc->mode == DIRECT_LINK_MODE) {
        if (dpmgr_path_is_busy(dp_handle: pgc->dpmgr_handle)) {
            if (primary_display_is_video_mode())
                return 0;
            DSI0_WaitNotBusyAfterTrigger(module: DISP_MODULE_DSI0);
        }
        DISPCHECK("trigger mode: DIRECT_LINK\n");
        disp_lcm_update(plcm: pgc->plcm, x, y, w: width, h: height, force: 0);
        // (3) DSI HS start, 只出 HS clock 信号
        dpmgr_path_start(dp_handle: pgc->dpmgr_handle, encmdq: primary_display_use_cmdq);
        if (primary_display_use_cmdq == CMDQ_DISABLE)
            // (4) DSI start, HS data 和 clock 都开始输出
            dpmgr_path_trigger(dp_handle: pgc->dpmgr_handle, trigger_loop_handle: NULL, encmdq: primary_display_use_cmdq);
    } else {
        DISPCHECK("primary display mode is WRONG(%d)\n", (unsigned int)pgc->mode);
    }
}

```

如果需要在 DSI 出 Data 和 Clock 后，跑 LCM 的初始化，需要把 `disp_lcm_init()` 放到 `dpmgr_path_trigger` 之后。另外可以把 `disp_lcm_init()` 拆成 `disp_lcm_init_power()` 和 `disp_lcm_init()` 做更细致的客制化。

在上图的 LK 开机 Flow 中，DSI & LC 的部分在下图红框中位置供参考。



9.2 Kernel Boot or Resume 流程

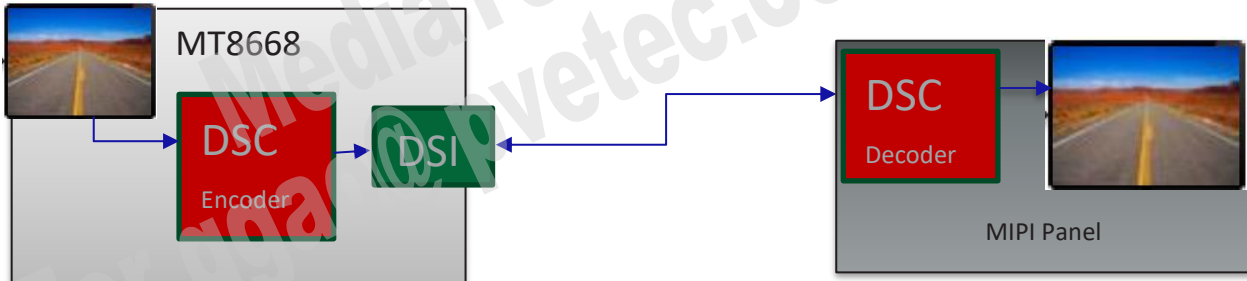
Kernel 中默认的 Flow 是 `drm_panel_prepare` 时 DSI 没有信号输出，`drm_panel_enable` 时 DSI 已经开始送信号了。可以根据需要调整 `drm_panel_prepare` 和 `drm_panel_enable` 的位置来满足时序要求。例如需要 DSI 送 clock 信号，才开始初始化，就可以把 `drm_panel_prepare` 放到 `mtk_dsi_clk_hs_mode` 之后，来保证时序，具体情况根据需求客制化。

```
co_device_module > drivers > gpu > drm > mediatek > mediatek_v2 > mtk_dsi.c > mtk_output_dsi_enable
5221 static void mtk_output_dsi_enable(struct mtk_dsi *dsi,
5346
5347     mtk_dsi_set_mode(dsi);
5348     mtk_dsi_clk_hs_mode(dsi, enter: 1); // DSI 送 HS clock 信号
5349     if (dsi->slave_dsi) {
5350         if (!mtk_dsi_is_slave(dsi) && !mtk_dsi_is_slave(dsi->slave_dsi)) {
```

```
5380  
5381     if (!mtk_dsi_is_cmd_mode(comp: &dsi->ddp_comp))  
5382         mtk_dsi_start(dsi); //DSI 送 data 和 clock 信号  
5383
```

10 DSI DSC Porting 指南

目前没有支持 VESA DSC DSI Serializer，可以支持 DSC 的 MIPI 屏幕。



10.1 DSC 介绍

DSC 代表 VESA 显示流压缩 (DSC) 标准，这是由 VESA 定义的用于压缩和解压缩图像显示流的标准。该标准规定了压缩和解压缩视频比特流的算法，包括压缩视频比特流的语法和语义。它具有实时压缩、传输、解压缩和显示的能力。使用 DSC 的好处是可以节省带宽，从而支持更高的分辨率或更高的帧率。DSC 支持 RGB 8 位的 1/2 和 1/3 压缩比，RGB 10 位支持 1/3.75 的压缩比。

10.2 DSC 参数

DSC 参数的填充主要关注 LK LCM_DSC_CONFIG_PARAMS 结构体和 Kernel mtk_panel_dsc_params 结构体中的成员，需要把 Panel Vendor 提供的 DSC 参数填写到对应 LCM Driver get_lcm_params() 函数中，Kernel 需填写在对应 LCM Driver mtk_panel_params 结构体中。有时 Panel Vendor 提供的可能是原始的 128 bytes pps 数据，这需要进行额外的转换工作，为避免转换过程出错建议请 Vendor 提供转换后的参数直接填入屏驱。开启 DSC 压缩后，需要注意 Driver 中 ext_params 中的 .data_rate 设定需要把 Hactive 乘以压缩比。因为 DSC 只会对 Active 的数据做压缩，Blanking 的部分是保持不变的。Data_rate 的计算公式如下：

DPHY Compression

$$\text{blanking ratio} = \frac{(HFP + HBP + HSA + Hact) \times (VFP + VBP + VSA + Vact)}{Hact \times Vact}$$

$$\text{data rate} = \frac{Hact \times Vact \times fps \times \text{compression ratio} \times bpp + Hact \times Vact \times fps \times (\text{blanking ratio} - 1) \times bpp}{\text{lane_num}}$$

对于 Driver 中 drm_display_mode 中 Clock 的设定则不需要乘以压缩比，设定跟没有开 DSC 的情况是一样的。

```
struct mtk_panel_dsc_params {
    unsigned int enable;
    unsigned int dual_dsc_enable;
    unsigned int bdg_dsc_enable;
    unsigned int ver; /* [7:4] major [3:0] minor */
    unsigned int slice_mode;
    unsigned int rgb_swap;
    unsigned int dsc_cfg;
    unsigned int rct_on;
    unsigned int bit_per_channel;
    unsigned int dsc_line_buf_depth;
    unsigned int bp_enable;
    unsigned int bit_per_pixel;
    unsigned int pic_height; /* need to check */
    unsigned int pic_width; /* need to check */
    unsigned int slice_height;
    unsigned int slice_width;
    unsigned int chunk_size;
    unsigned int xmit_delay;
    unsigned int dec_delay;
    unsigned int scale_value;
    unsigned int increment_interval;
    unsigned int decrement_interval;
    unsigned int line_bpg_offset;
    unsigned int nfl_bpg_offset;
    unsigned int slice_bpg_offset;
    unsigned int initial_offset;
    unsigned int final_offset;
    unsigned int flatness_minqp;
    unsigned int flatness_maxqp;
    unsigned int rc_model_size;
    unsigned int rc_edge_factor;
    unsigned int rc_quant_incr_limit0;
    unsigned int rc_quant_incr_limit1;
    unsigned int rc_tgt_offset_hi;
    unsigned int rc_tgt_offset_lo;
    unsigned int rc_buf_thresh[14];
    struct dsc_rc_range_parameters rc_range_parameters[15];
};
```

DSC 参数	描述
enable	DSC enable flag
dual_dsc_enable	Enable 2 DSC engine flag
bdg_dsc_enable	MT6382 DSC enable flag (无需配置)
ver	DSC version
slice_mode	Select slice division as 1-slice mode or 2-slice mode. 0: 1-slice mode 1: 2-slice mode
rgb_swap	Select 24-bit input pixel format as RGB or BGR type 0: RGB type 1: BGR type
dsc_cfg	Config flatness determination threshold 2: 8bpc 8: 10 bpc Config ICH enable; 0: ICH disable; 1: ICH enable
rct_on	Converts RGB to YcoCg 0: YUV in 1: RGB in
bit_per_channel	Bit per channel, unit is bit
dsc_line_buf_depth	Align setting with DDIC vendor

DSC 参数	描述
bp_enable	Block prediction enable flag
bit_per_pixel	Bit per pixel * 16, unit is bit
pic_height	Equal to frame_height
pic_width	Equal to frame_width
slice_height	Align setting with DDIC vendor; there are usually 8, 16, 32, 40, etc.
slice_width	Frame_width/silce_mode
chunk_size	Equal slice_width × bit_per_pixel/8
rc_buf_thresh	Parameters that must be filled
dsc_rc_range_paramsters	Parameters that must be filled

10.3 DSC SW 介绍

DSC 的 Source Code 仅需要关注 LK ddp_dsc.c 和 Kernel mtk_disp_dsc.c 两个文件，这两个文件也相对简单，仅实现了如下几个函数：

LK ddp_dsc.c

```
struct DDP_MODULE_DRIVER ddp_driver_dsc = {
    .init = dsc_init,
    .deinit = dsc_deinit,
    .config = dsc_config,
    .dump_info = dsc_dump,
};
```

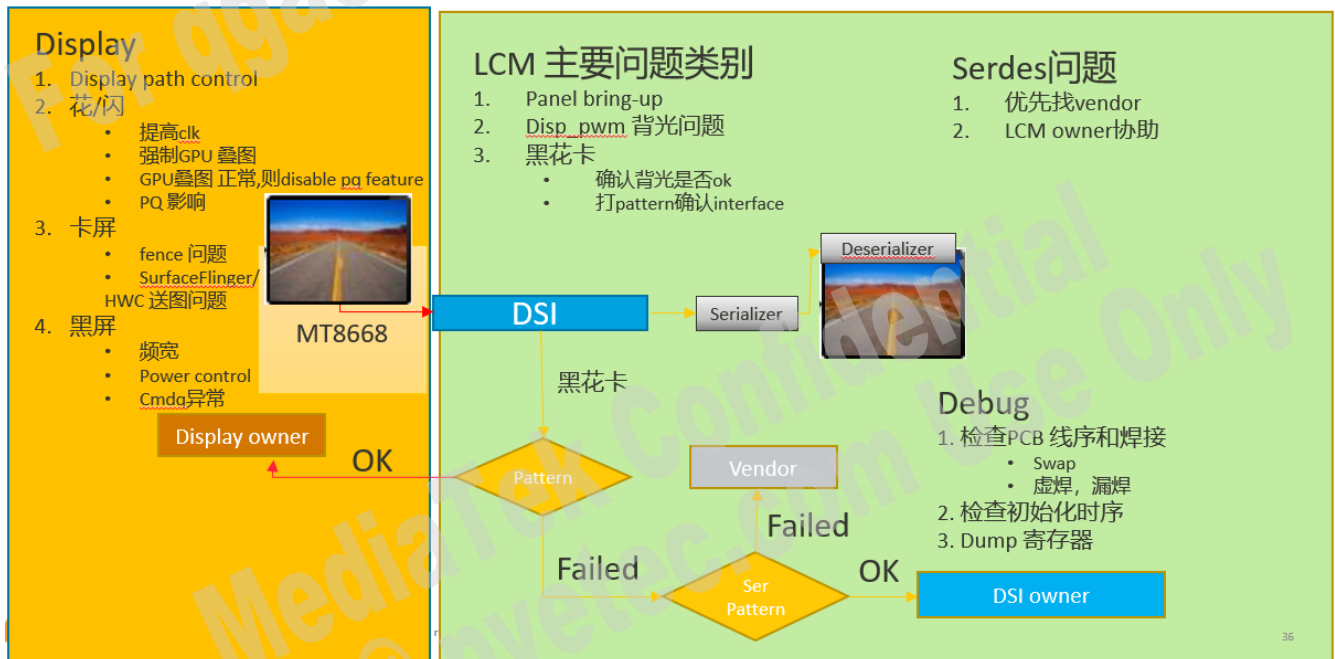
Kernel mtk_disp_dsc.c

```
static const struct mtk_ddp_comp_funcs mtk_disp_dsc_funcs = {
    .config = mtk_dsc_config,
    .start = mtk_dsc_start,
    .stop = mtk_dsc_stop,
    .prepare = mtk_dsc_prepare,
    .unprepare = mtk_dsc_unprepare,
};
```

在 Module init 后就会从 LCM Driver 中把 DSC 参数通过 dsc_cofig 和 mtk_dsc_config 接口写进对应的 DSC 寄存器。如果 LCM Driver 中 DSC 参数错误就会导致 DSC Encoder 压缩与 DDIC DSC Decoder 不一致导致花屏问题，因此 LCM Driver 中 DSC 参数完整性和正确性就显得尤为重要。

11 LCM 问题调试

屏幕在点屏阶段和 DV 测试阶段的问题会有些差异。在点板点屏阶段，针对屏幕的问题，除了分析软件部分的设定还需要考虑 HW 的设计是否正确和焊接是否可靠等问题。DV 测试阶段的黑花卡更侧重分析 SW Flow 的 Bug。下图针对常见的问题做了一个简单的分析步骤，这样在针对复杂的屏幕的问题的时候，通过 DSI 的 Pattern 是否显示正常作为一个切入点，能够快速切割定位问题，而不会觉得无从下手。另外需要提到的是黑屏的问题，首先需要确认的是背光是否正常，再做后面的分析。

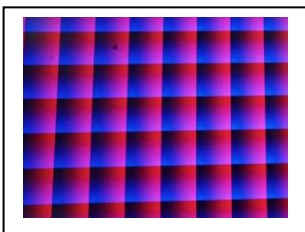


11.1 DSI Pattern

1. Userdebug 版本可以使用 GCE 的 CMD 来打 Pattern:

DSIO

```
adb shell "echo gce_wr:0x1401a178,0x61,0xffffffff > /sys/kernel/debug/mtkfb"
```



DSI1

```
adb shell "echo gce_wr:0x1401b178,0x61,0xffffffff > /sys/kernel/debug/mtkfb"
```

可以通过 gce_rd 确认是否有写成功:

```
adb shell "echo gce_rd: 0x1401a178 > /sys/kernel/debug/mtkfb; cat /sys/kernel/debug/mtkfb | grep gce_rd"
```

2. User 版本 GCE CMD 不能使用，可以使用 Clock Debug 的 CMD，需要在 Kernel config 中先将 CONFIG 打开后才能使用，Userdebug 版本也可以用 Clock Debug 的 CMD，同样需要打开 CONFIG。

```
CONFIG_MTK_CLKMGR_DEBUG=y
```

DSI0

```
adb shell "echo reg_write 0x1401a178 0x61 > /proc/clktb; cat /proc/clktb"
```

DSI1

```
adb shell "echo reg_write 0x1401b178 0x61 > /proc/clktb; cat /proc/clktb"
```

可以通过 reg_read 确认是否有写成功：

```
adb shell "echo reg_read 0x1401a178 > /proc/clktb; cat /proc/clktb"
```

11.2 DSI Register 介绍

DSI0 Base register	0x1401a000	DSI0 mip tx base	0x11e50000
DSI1 Base register	0x1401b000	DSI1 mip tx base	0x11e10000

DSI Register 都是 Base 地址加上一个 Offset，具体的 Offset 以及定义可以参考 Register Map。下表中介绍一下常用的 DSI 的 Register 的含义，方便 Debug 的时候使用。

地址	名称	介绍
000	DSI_START	Bit0 为 1 表示 DSI 信号有输出
004	DSI_INTSTA	中断的状态, 例如 bit12 表示有 buffer underrun, 可以查看 DSI 信号的状态
038	DSI_SIZE_CON	Panel active 的 size, Bit30 ~ Bit16: 高; Bit14 ~ Bit0: 宽
010	DSI_COM_CON	Bit0: DSI_RESET, software reset Bit2: DPHY_RESET, MIPI TX software reset
014	DSI_MODE_CON	Bit1~Bit0 Video mode
018	DSI_TXRX_CON	Bit16 为 0 代表 continue clock Bit5 ~ Bit2: Lane number
01c	DSI_PS_CON	Bit19 ~ Bit16: Data format 3: RGB888 5: DSC 压缩数据

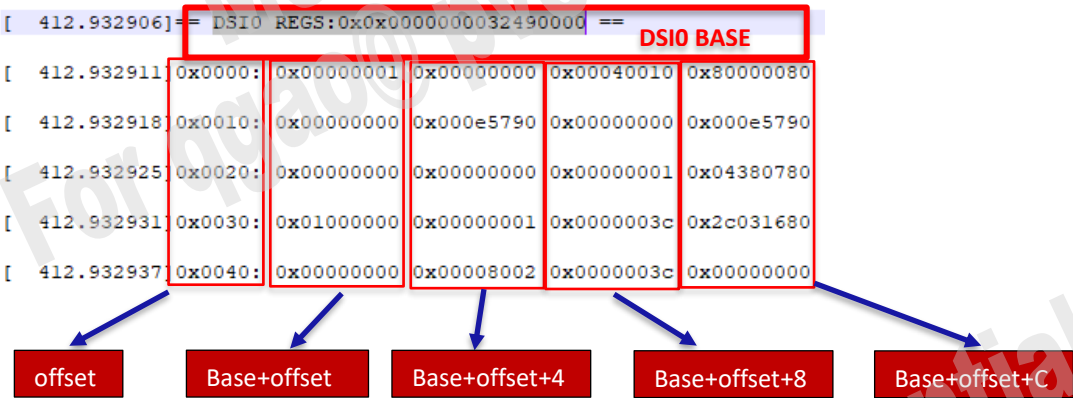
地址	名称	介绍															
020	DSI_VSA_NL	= Panel VPW															
024	DSI_VBP_NL	= Panel VBP															
028	DSI_VFP_NL	= Panel VFP															
02c	DSI_VACT_NL	= Panel height															
050	DSI_HSA_WC	= Panel HSA*3-10															
054	DSI_HBP_WC	= Panel HBP*3-10															
058	DSI_HFP_WC	Bit14 ~ Bit0 = Panel HFP*3-12 Bit31 为 1 表示开启 active 的数据每个 frame 回 LP, HFP 一直 keep HS															
110	DSI_PHY_TIMCON0	MIPI Data Lane Timing HQA Test 可能会调整 <table> <tr> <th>Bit(s)</th><th>Name</th><th>Description</th></tr> <tr> <td>31:24</td><td>DA_HS_TRAIL</td><td>timing parameter: T_HS-Trail</td></tr> <tr> <td>23:16</td><td>DA_HS_ZERO</td><td>timing parameter: T_HS-Zero</td></tr> <tr> <td>15:8</td><td>DA_HS_PREP</td><td>timing parameter: T_HS-Prepare</td></tr> <tr> <td>7:0</td><td>LPX</td><td>timing parameter: T_LPX</td></tr> </table>	Bit(s)	Name	Description	31:24	DA_HS_TRAIL	timing parameter: T_HS-Trail	23:16	DA_HS_ZERO	timing parameter: T_HS-Zero	15:8	DA_HS_PREP	timing parameter: T_HS-Prepare	7:0	LPX	timing parameter: T_LPX
Bit(s)	Name	Description															
31:24	DA_HS_TRAIL	timing parameter: T_HS-Trail															
23:16	DA_HS_ZERO	timing parameter: T_HS-Zero															
15:8	DA_HS_PREP	timing parameter: T_HS-Prepare															
7:0	LPX	timing parameter: T_LPX															
114	DSI_PHY_TIMCON1	MIPI Data Lane Timing HQA Test 可能会调整															
11e50008	MIPITX_CDPHY_VOLTAGE_SEL	MIPI Swing 调整 <table> <tr> <td>9:6</td><td>RG_DSI_HSTX_LDO_REF_SEL</td><td>Selects 0.4V/0.5V ref voltage (20mV/step, covers 0.3 to 0.6V)</td></tr> <tr> <td colspan="3">4'b0000: Min. voltage</td></tr> <tr> <td colspan="3">4'b1000: Typical voltage</td></tr> <tr> <td colspan="3">4'b0111: Max. voltage</td></tr> </table>	9:6	RG_DSI_HSTX_LDO_REF_SEL	Selects 0.4V/0.5V ref voltage (20mV/step, covers 0.3 to 0.6V)	4'b0000: Min. voltage			4'b1000: Typical voltage			4'b0111: Max. voltage					
9:6	RG_DSI_HSTX_LDO_REF_SEL	Selects 0.4V/0.5V ref voltage (20mV/step, covers 0.3 to 0.6V)															
4'b0000: Min. voltage																	
4'b1000: Typical voltage																	
4'b0111: Max. voltage																	
1401a178	DSI_SELF_PAT_CON0	DSI Pattern, 可以出纯色, 灰阶等 Pattern															
11e5002C	MIPITX_PLL_CON0	MIPI Clock PCW, Clock 不同设定会有差异															
11e50030	MIPITX_PLL_CON1	MIPI Clock 开关和 Divide															
11e50034	MIPITX_PLL_CON2	MIPI Clock 展频设定 Bit1 为 1 表示开启展频															
11e50038	MIPITX_PLL_CON3	MIPI Clock 展频设定															
11e50004	MIPITX_CDPHY_LANE_CON	<table> <tr> <th>Bit(s)</th><th>Name</th><th>Description</th></tr> <tr> <td>9</td><td>RG_DSI_DEM_EN</td><td>Enables DSI de-emphasis</td></tr> <tr> <td colspan="3">1'b0: Off</td></tr> <tr> <td colspan="3">1'b1: On</td></tr> </table>	Bit(s)	Name	Description	9	RG_DSI_DEM_EN	Enables DSI de-emphasis	1'b0: Off			1'b1: On					
Bit(s)	Name	Description															
9	RG_DSI_DEM_EN	Enables DSI de-emphasis															
1'b0: Off																	
1'b1: On																	

11.3 DSI Register Dump 介绍

可以通过 CMD Dump 所有的 DSI Register:

```
adb shell "echo diagnose>/proc/mtkfb && cat /proc/mtkfb" > d:\mtkfb.txt
```

Dump 的结果如下图中所示



通过 reg_read_len CMD 读寄存器，使用前需要打开 config:

```
CONFIG_MTK_CLKMGR_DEBUG=y
```

```
adb shell "echo reg_read_len addr len> /proc/clkgdb ; cat /proc/clkgdb"
```

其中 len 指的是偏移地址长度，会依次读出 addr+offset 的寄存器值。

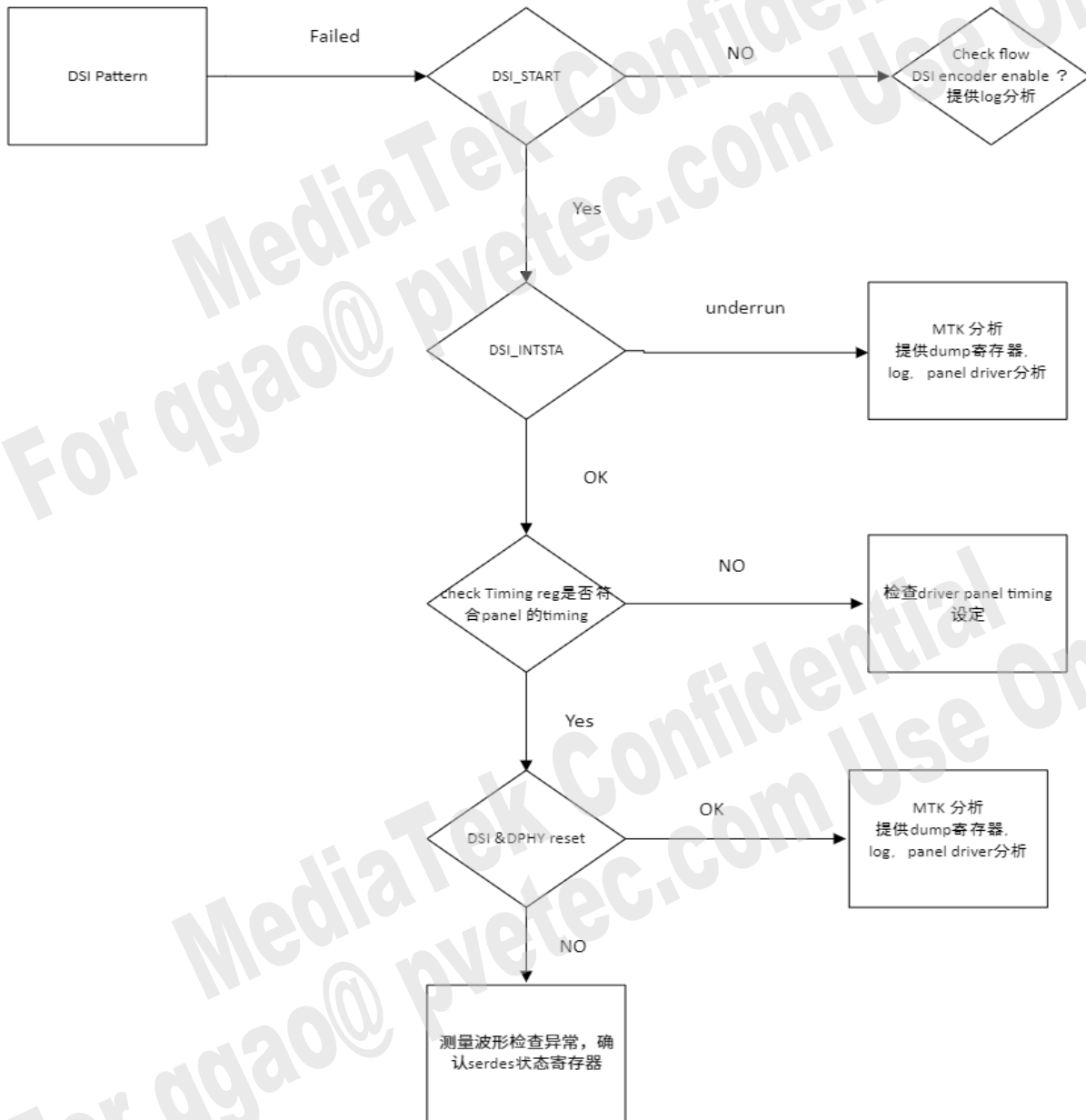
假设想要 Dump DSIO 0x00~0x100 的地址的值命令如下:

```
adb shell "echo reg_read_len 0x1401a000 0x100> /proc/clkgdb ; cat /proc/clkgdb"
```

当 diagnose CMD 使用异常的时候，可以使用这个 CMD Dump 相关的寄存器。

11.4 DSI 调试流程

下图为简单的 DSI Debug 流程图，可以结合 SerDes 的状态一起来看，max96789 结合章节 8.3 同步分析。



11.5 帧率异常

检查 Panel Driver 中 **Pixel Clock** 设定与 **data_rate** 设定是否正常。文档中搜索关键字，可以找到 Driver 中参数的介绍位置确认。

11.6 I2C 通信异常

1. 确认加串器电源晶振是否正常，确认软件时序和 I2C 地址设定，测量 I2C 信号电压是否正常。
2. 请参考章节 3.3.10 确认 DWS 设定是否正常。

3. 确认 I2C 上拉电阻和驱动电流设定是否正常。

1). dts 中找到使用的 I2C 节点:

```
&i2c4 {
    pinctrl-names = "default";
    pinctrl-0 = <&i2c4_pins>;
};
```

2). 在&pio 节点下添加如下内容:

```
&pio {
    i2c4_pins: i2c4-default {
        pins-bus {
            pinmux = <PINMUX_GPIO190__FUNC_SCL4>,
            <PINMUX_GPIO191__FUNC_SDA4>;
        bias-pull-up = <MTK_PULL_SET_RSEL_111>;
        drive-strength = <MTK_DRIVE_2mA>;
    };
};
```

参数	定义
PINMUX_GPIO190__FUNC_SCL4 PINMUX_GPIO191__FUNC_SDA4	include/dt-bting/pinctrl/mt6881-pinfunc 查找对应的 I2C pinmux
bias-pull-up=<MTK_PULL_SET_RSEL_111>	I2C 上拉电阻设定: 000: PU75K, PD75K 001: PU10K, PD5K 010: PU5K, PD75K 011: PU4K, PD5K 100: PU3K, PD75K 101: PU2K, PD5K 110: PU1.5K, PD75K 111: PU1K, PD5K 建议使用 111, 1K 上拉电阻, 不要超过 5K
drive-strength = <MTK_DRIVE_2mA>	include/dt-bting/pinctrl/mt65xx.h 查看驱动电流的范围
关掉上拉	+ //bias-pull-up = <MTK_PULL_SET_RSEL_111>; + bias-disable;

12 DSI DPHY CTS 测试问题

CTS 测试的主要目的是验证设备是否符合 MIPI DSI DPHY 规范。CTS 测试主要包含一下几个方面：

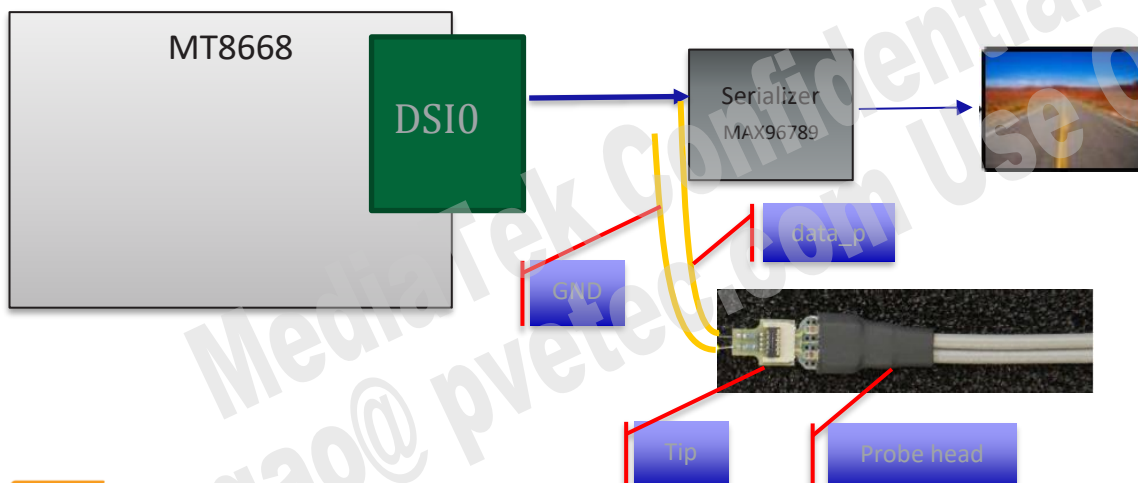
1. 电气特性测试：电压，边沿等是否满足规格
2. 协议一致性测试：MIPI Timing 是否满足规格
3. 信号完整性测试：测试信号质量，眼图，Skew 是否满足规格

12.1 测试仪器

由于是高速信号，对信号的测试环境和仪器要求比较高。需要使用专用的带 DPHY 协议的测试仪器，并且使用专用的探头，探尖，tip 来连接信号到仪器上，同时在使用前需要对仪器做校准。如果没有相关的测试仪器，需要找第三方机构做测试。

12.2 测试平台准备

1. 测试需要焊接差分信号 clock_p, clock_n, data_p, data_n 的测试点。测试点建议选在靠近 Bridge 的输入端。同时焊接的 GND 线尽量不要离的很远，线长尽量控制在 1~2cm。data_p 的接法示意图如下图所示，4 根信号共需要接 4 个 Tip 到示波器上。
2. 测试的平台需要接屏测试，并且屏幕可以显示正常。这样是为了保证 Bridge 或者 RX 的设定都正常，并且负载端是正常的。
3. 测试的平台可以播放视频，视频的内容建议随机一些，越随机越好。或者放一张比较花的图片，避免大面积的黑和白。测试平台接 adb，方便 Debug 使用。
4. 测试前需要检查每个信号都是稳定正常的，确保焊接，Tip 和 Probe Head 都正常。
5. DSI 输出设定要关闭展频，同时要记住 DSI Clock，或者测试前量测一下 Clock。



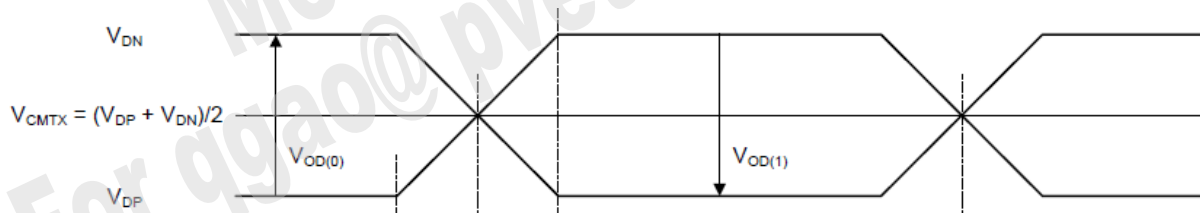
12.3 常见 Failed 项目分析

Continue Clock HS 部分的测试项目如下，红色方框中的项目主要是电气特性相关，绿色方框的项目主要是 MIPI Timing 协议一致性相关，蓝色方框的项目主要是信号质量相关。

Pass#	Failed#	Trials	Test Name	Actual Value	Margin	Pass Limits
✓	0	1	1.3.7 HS Data TX Static Common Mode Voltage(Vcmtx)	187.22 mV	37.2 %	150.00 mV <= VALUE <= 250.00 mV
✓	0	1	1.3.8 HS Data TX Vcmtx Mismatch	160 μV	3.2 %	0.00000 V <= VALUE <= 5.00 mV
✓	0	1	1.3.10 HS Data TX Common-Level Variations Above 450MHz (VCMTX(HF))	5.49 mV	63.4 %	VALUE < 15.00 mV
✓	0	1	1.3.9 HS Data TX Common-Level Variations Between 50-450MHz (VCMTX(LF))	12.75 mV	49.0 %	VALUE < 25.00 mV
✓	0	1	1.3.4 HS Data TX Differential Voltage(VOD0 Pulse)	193.55 mV	41.2 %	-270.00 mV <= VALUE <= -140.00 mV
✓	0	1	1.3.4 HS Data TX Differential Voltage(VOD1 Pulse)	184.94 mV	34.6 %	140.00 mV <= VALUE <= 270.00 mV
✓	0	1	1.3.5 HS Data TX Differential Voltage Mismatch (Pulse)	8.61 mV	38.5 %	0.00000 V <= VALUE <= 14.00 mV
✓	0	1	1.3.6 HS Data TX Single Ended Output High Voltage(VOHHS Pulse)	287.28 mV	20.2 %	VALUE <= 360.00 mV
✓	0	1	1.3.11 HS Data TX 20%-80% Rise Time (tR)	313 ps	10.1 %	100 ps <= VALUE <= ComplianceTranTimeMax s
✓	0	1	1.3.12 HS Data TX 20%-80% Fall Time (tF)	313 ps	10.1 %	100 ps <= VALUE <= ComplianceTranTimeMax s
✓	0	1	1.4.7 HS Clock TX Static Common Mode Voltage(Vcmtx)	180.95 mV	31.0 %	150.00 mV <= VALUE <= 250.00 mV
✓	0	1	1.4.8 HS Clock TX Vcmtx Mismatch	161 mV	27.8 %	0.00000 V <= VALUE <= 5.00 mV
✓	0	1	1.4.10 HS Clock TX Common-Level Variations Above 450MHz (VCMTX(HF))	5.47 mV	56.9 %	VALUE < 15.00 mV
✓	0	1	1.4.9 HS Clock TX Common-Level Variations Between 50-450MHz (VCMTX(LF))	10.54 mV	57.8 %	VALUE < 25.00 mV
✓	0	1	1.4.4 HS Clock TX Differential Voltage(VOD0 Pulse)	149.31 mV	7.2 %	-270.00 mV <= VALUE <= -140.00 mV
✓	0	1	1.4.4 HS Clock TX Differential Voltage(VOD1 Pulse)	151.84 mV	9.1 %	140.00 mV <= VALUE <= 270.00 mV
✓	0	1	1.4.5 HS Clock TX Differential Voltage Mismatch (Pulse)	2.53 mV	18.1 %	0.00000 V <= VALUE <= 14.00 mV
✓	0	1	1.4.6 HS Clock TX Single Ended Output High Voltage(VOHHS Pulse)	263.67 mV	26.8 %	VALUE <= 360.00 mV
✓	0	1	1.4.11 HS Clock TX 20%-80% Rise Time (tR)	241 ps	40.5 %	100 ps <= VALUE <= ComplianceTranTimeMax s
✓	0	1	1.4.12 HS Clock TX 20%-80% Fall Time (tF)	235 ps	43.0 %	100 ps <= VALUE <= ComplianceTranTimeMax s
✓	0	1	1.3.1 LP Exit DATA TLXP	75.46 ns	50.9 %	VALUE >= 50.00 ns
✓	0	1	1.3.2 LP Exit DATA TX THS-PREPARE	30.15 ns	34.7 %	ComplianceTimingMin s <= VALUE <= ComplianceTimingMax s
✓	0	1	1.3.3 LP Exit DATA TX THS-PREPARE+THS-ZERO	202.84 ns	31.2 %	VALUE >= ComplianceTimingMin s
✓	0	1	1.3.13 HS Exit DATA TX THS-TRAIL	32.21 ns	28.8 %	VALUE >= ComplianceTimingMin s
✓	0	1	1.3.14 HS Exit DATA TX TREQT	14.73 ns	57.9 %	VALUE <= 35.00 ns
✓	0	1	1.3.15 HS Exit DATA TX TEOT	36.94 ns	16.8 %	VALUE <= ComplianceTimingMax s
✓	0	1	1.3.16 HS Exit DATA TX THS-EXIT	195.62 ns	95.6 %	VALUE >= 100.00 ns
✓	0	1	1.4.17 HS Clock Instantaneous (Ulinst)	965 ps	92.3 %	VALUE < 12 500 ns
✓	0	1	1.5.3 HS Clock Rising Edge Alignment to First Payload Bit	0.0000000000000000	100.0 %	VALUE <= 500.0000000000 m
✓	0	1	1.5.4 Data-to-Clock Skew (TSKEW(TX))	0 mUlinst	35.0 %	ComplianceTranTimeMin Ulinst <= VALUE <= ComplianceTranTimeMax Ulinst

12.3.1 1.3.4/1.4.4 VOD0/VOD1 Pulse & 1.3.7 VCMTX

Clock 和 Data 默认设定都是 200mV，如果偏小，可以调整 HSTX 输出电压，增大驱动能力。



参考 DSI 寄存器介绍中 MIPITX_CDPHY_VOLTAGE_SEL 寄存器 0x11e50008，20mV 一个 Step，可以通过 adb 更改寄存器逐渐调大满足 Spec。建议调大之后观察这几组值的变化看是否符合预期。

	名称	默认值	建议调整范围
D-PHY	RG_DSIO_HSTX_LDO_REF_SEL	4'b100	0110: 0.36V 0111: 0.38V 1000: 0.4V 1001: 0.42V 1010: 0.44V 1011: 0.46V

以寄存器 DSIO 为例，DSI1 的寄存器请看寄存器介绍的部分。

```
adb shell "echo gce_rd:0x11e50008 > /sys/kernel/debug/mtkfb; cat /sys/kernel/debug/mtkfb |
grep gce_rd"
[ 83.560777][mtkfb_dbg] gce_rd:0x11e50008
[ 83.560784]display_debug cmd gce_rd:0x11e50008
[ 83.561045][reg_dbg] gce_rd: addr(0x11e50008) = 0x44441200
```

更改 Bit6 ~ Bit9 的值:

电压值	Bit6 ~ Bit9	寄存器
0.4V	1000	0x44441200
0.42V	1001	0x44441240
0.44V	1010	0x44441280
0.46V	1011	0x444412C0
0.48V	1100	0x44441300

根据情况调整需要增加的电压值，例如调整到 0.42V。

```
adb shell "echo gce_wr: 0x11e50008,0x44441240,0xffffffff > /sys/kernel/debug/mtkfb"
```

12.3.1.1 SW 如何调整电压

LK:

```
--- a/platform/mediatek/mt8668/disp/ddp_dsi.c
+++ b/platform/mediatek/mt8668/disp/ddp_dsi.c
@@ -2293,6 +2293,8 @@ void DSI_DPHY_clk_setting(enum DISP_MODULE_ENUM module, void *cmdq,
LCM_DSI_PARA
#endif
    if (data_Rate < 2500)
        MIPITX_OUTREGBIT(DSI_PHY_REG[i]+MIPITX_VOLTAGE_SEL, FLD_RG_DSI_PRD_REF_SEL,
0x0);
+
+MIPITX_OUTREGBIT(DSI_PHY_REG[i]+MIPITX_VOLTAGE_SEL,
FLD_RG_DSI_HSTX_LDO_REF_SEL, 0x9); //0.42V
    #if 0
        if (data_Rate > 2000)
            MIPITX_OUTREGBIT(DSI_PHY_REG[i]+MIPITX_VOLTAGE_SEL, FLD_RG_DSI_V2I_REF_SEL,
0x4);

--- a/platform/mediatek/mt8668/disp/ddp_reg_mipi.h
+++ b/platform/mediatek/mt8668/disp/ddp_reg_mipi.h
@@ -13,6 +13,7 @@
#define MIPITX_LANE_CON                (0x0004UL)
#define MIPITX_VOLTAGE_SEL              (0x0008UL)
#define FLD_RG_DSI_PRD_REF_SEL          REG_FLD(6, 0)
+#define FLD_RG_DSI_HSTX_LDO_REF_SEL     REG_FLD(4, 6)
#define FLD_RG_DSI_V2I_REF_SEL          REG_FLD(4, 10)
```

Kernel:

```

--- a/drivers/gpu/drm/mediatek/mediatek_v2/mtk_mipi_tx.c
+++ b/drivers/gpu/drm/mediatek/mediatek_v2/mtk_mipi_tx.c
@@ -2308,6 +2308,9 @@ static int mtk_mipi_tx_pll_dphy_config_mt6991(struct mtk_mipi_tx
 *mipi_tx)

    mtk_mipi_tx_update_bits(mipi_tx, MIPITX_VOLTAGE_SEL_MT6983,
        FLD_RG_DSI_PRD_REF_SEL, 0x4);
+   mtk_mipi_tx_update_bits(mipi_tx, MIPITX_VOLTAGE_SEL_MT6983,
+       FLD_RG_DSI_HSTX_HLO_REF_SEL, 0x9 << 6); //0.42V

#ifdef IF_ZERO
    /* No need keep as default */
    if (rate > 2000)

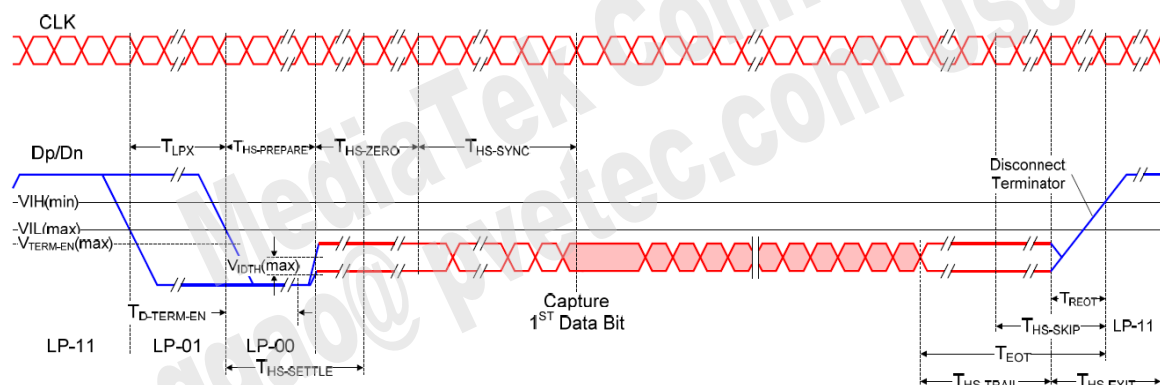
```

12.3.2 1.3.8 Voltage Mismatch & 1.4.8 VCMTX Mismatch

检查测试环境，探头是否有做校准，PCB Layout 是否等长。

12.3.3 1.3.1~3 & 1.3.13~16 MIPI Timing

MIPI Timing 如下图中所示可以通过寄存器调整，寄存器 DSI_PHY_TIMCON0 中有说明。



上图中参数需要满足下图中 Spec 规范的范围，其中 $UI = 1/\text{datarate}$ ，SW 中设定最小单位是 $8UI$ 。

Parameter	Description	Min	Typ	Max	Unit	Notes
T _{HS-PREPARE}	Time to drive LP-00 to prepare for HS transmission	40 ns + 4*UI		85 ns + 6*UI	ns	
T _{HS-PREPARE} + T _{HS-ZERO}	T _{HS-PREPARE} + Time to drive HS-0 before the Sync sequence	145 ns + 10*UI			ns	
T _{HS-SKIP}	Time-out at RX to ignore transition period of EoT	40		55 ns + 4*UI	ns	
T _{HS-TRAIL}	Time to drive flipped differential state after last payload data bit of a HS transmission burst	max(n*8*UI, 60 ns + n*4*UI)			ns	2, 3
T _{INIT}	Initialization period (PHY might calibrate)	100			μs	
T _{LFX}	Length of any Low-Power state period	50			ns	4
Ratio T _{LFX}	Ratio of T _{LFX} (MASTER/ T _{LFX} (SLAVE) between Master and Slave side	2/3		3/2		
T _{TA-GET}	Time to drive LP-00 by new TX	5*T _{LFX}			ns	
T _{TA-GO}	Time to drive LP-00 after Turnaround Request	4*T _{LFX}			ns	
T _{TA-SURE}	Time-out before new TX side starts driving	T _{LFX}		2*T _{LFX}	ns	
T _{WAKEUP}	Recovery time from Ultra-Low Power State	1			ms	

12.3.3.1 SW 如何调整 Timing

以下图中 1.3.15 TEOT Failed 为例，现在测试结果为 299.77ns，此值偏大，Pass Limit Max 值为 133.85ns。

✗	1	1.3.15 HS Exit: DATA TX TEOT	299.77 ns	-124.0 %	VALUE <= ComplianceTimingMax s
✓	0	1.3.16 HS Exit: DATA TX THS-EXIT	681.27 ns	581.3 %	VALUE >= 100.00 ns
✓	0	1.4.17 HS Clock Instantaneous (Ulnst)	2.404 ns	80.8 %	VALUE < 12.500 ns

1.3.15 HS Exit: DATA TX TEOT

Test Summary: **FAIL** Test Description: Time from start of THS-TRAIL period to start of LP-11 state. ComplianceTimingMax is based on 105ns+n*12*UI.

Pass Limits: <= ComplianceTimingMax s TEOT 299.77 ns

Result Details

TEOT (see image) DataLane Lane0 Pass Limit Max (ComplianceTimingMax) 133.85 ns Number of HS burst 1

Trial 1

Trial 1: TEOT

现在需要调整设定，使得 TEOT 小于 133.85ns Pass Spec:

- 从 9.3.3 MIPI Timing 图中，找到 TEOT 信号的位置和 Spec 的范围。
- 1.4.17 确认 UI=2.404ns。
- 根据 UI 计算 TEOT 阈值：TEOT=[~, 105ns+12*UI] = [~, 133.848ns]，也可以直接看提示的 Pass Limit 的值。
- 计算 SW 设定最小单位 = 8 * UI = 8*2.404 = 19.232ns，SW 设定寄存器值 < 133.848ns/19.232ns = 6.96。
- 根据 UI 计算 HS_TRAIL 阈值：THS_TRAIL > 60ns + 4UI=69.6ns，SW 设定寄存器值 > 69.6ns/19.232= 3.6。
- HS_TRAIL 属于 TEOT 一部分，降低 HS_TRAIL，既是减小 TEOT，同时要保证 THS_TRAIL 也要满足 Spec，SW 设定值为整数，则可以取 4~6 中的一个值通过 adb 设定进去，再测试看看。

12.3.3.2 如何将调好的值写进 SW

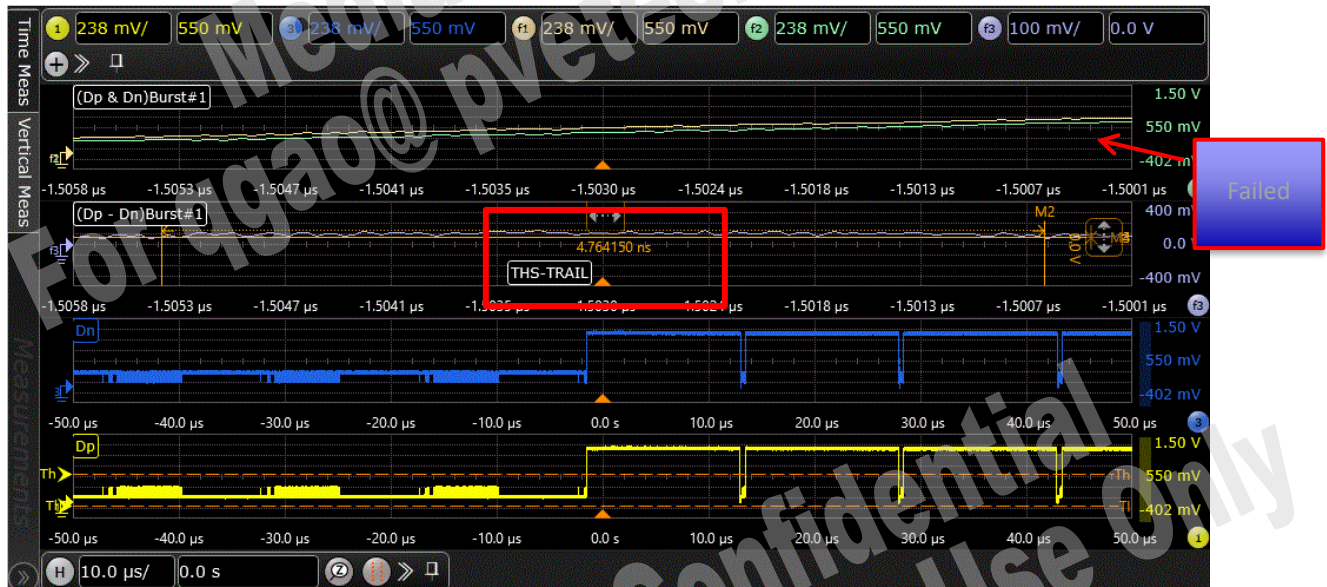
SerDes 的 Driver 可以将参数放到 dts 里面客制化，再给到如下的参数里面。

LK Panel Driver	lcm_get_params()里面添加 <pre>static void lcm_get_params(LCM_PARAMS *params) { memset(params, 0, sizeof(LCM_PARAMS)); params->type = LCM_TYPE_DSI; }</pre>	<pre>params->dsi.PLL_CLOCK = 497; params->dsi.clk_lp_per_line_enable = 0; params->dsi.ssc_disable = 1; params->dsi.conf_clock = 1; /* Keep HS : params->dsi.DA_HS_EXIT = 10; params->dsi.CLK_ZERO = 16; params->dsi.HS_ZERO = 25; params->dsi.HS_TRAIL = 10; params->dsi.CLK_TRAIL = 1; params->dsi.CLK_HS_POST = 8; params->dsi.CLK_HS_EXIT = 6; /* params->dsi.CLK_HS_PRRP = 1; */ params->dsi.TA_GO = 8; params->dsi.TA_GET = 10;</pre>
Kernel Panel Driver	ext_params 里面添加需要调整的参数 <pre>#if defined(CONFIG_MTK_PANEL_EXT) static struct mtk_panel_params ext_params = { .pll_clk = 270,</pre>	<pre>.phy_timcon = { .hs_trail = 10, .hs_zero = 25, .ta_get = 10, .ta_go = 8, .da_hs_exit 10, .clk_trail = 1, .clk_zero = 16, .clk_hs_post = 8, .clk_hs_exit = 6,</pre>

12.3.3.3 THS_TRAIL Failed

THS_TRAIL 比较常见的 Failed 情况是示波器没有抓到正确的 HS_TRAIL。这样调整寄存器也是没有用的，可以看到的现象是测出来的 HS_TRAIL 值非常小，如下图中所示只有 4ns 左右。

Keysight Infiniium : Wednesday, July 10, 2024 9:16:53 PM



可以对比下图中正常的波形，正常的 HS TRAIL 是一段 HS 1 的数据，所以碰到这种情况请确认示波器是否有正确设定，或者重新测试，或者可以手动测量确认。



12.3.4 1.3.11~12 Tr/Tf & 1.5.4 Data to Clock Skew

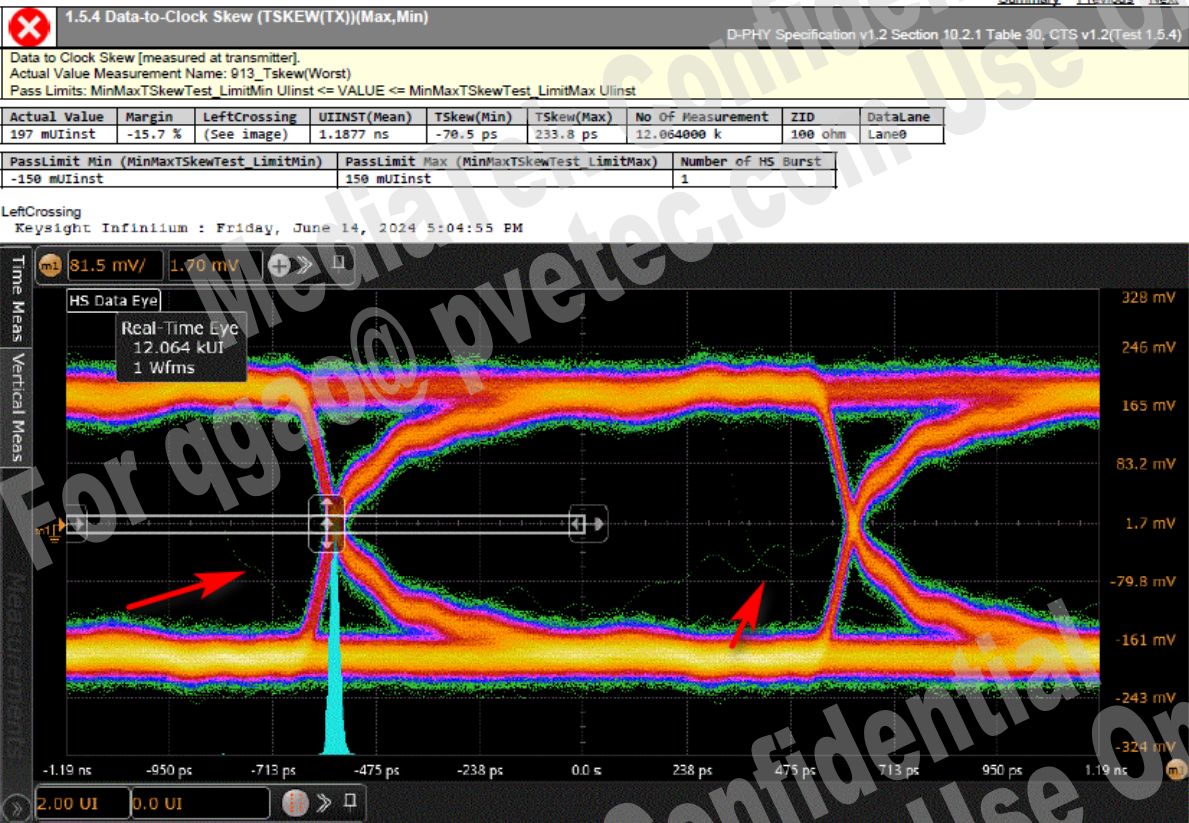
信号的质量对 Data 和 Clock 的 Tr/Tf 以及 Data to Clock Skew 的测试项目影响很大。而影响信号质量的根本原因就是差分信号阻抗的控制，SoC 输出单端 50 欧姆，差分 100 欧姆。要求 PCB Layout 也遵循这个原则，并且要做阻抗控制，这部分可以跟 HW 对齐，查看具体的 Layout 要求和 PCB 厂商要求阻抗控制。同样 RX 的部分也要按照同样的阻抗设定，这样才能保证信号的完整性。

12.3.4.1 SoC 的阻抗调整

SoC 的 MIPI TX 有单端的阻抗调整寄存器，但是不建议调整。目前 SW 设定默认输出有做阻抗的 Calibration 保证 Performance，如果单独调整某根信号的阻抗，很难保证一致性。

12.3.4.2 干扰导致 Skew Fail

除了测试的信号接线稳定，还要确认平台和仪器的接地也要稳定，注意排查杂讯的干扰。下图中红色箭头不预期的杂讯，会影响测试结果，需要确认测试环境。



12.3.5 什么情况开 DEM

当 datarate > 2.5Gbps 时，可以开启 DEM (de-emphasis)。DEM 开启后信号的幅度会降低，同时需要调大信号的幅度，确保可以满足 Spec。

以寄存器 DSI0 为例，DSI1 的寄存器请看寄存器介绍的部分。

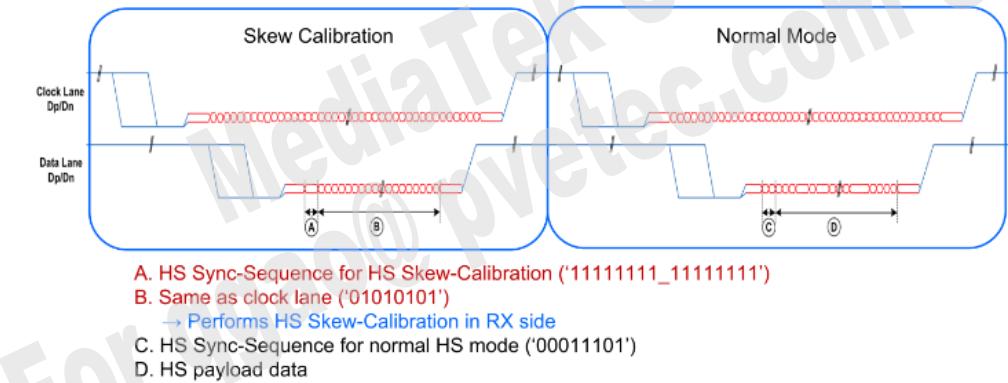
```
adb shell "echo gce_rd:0x11e50004 > /sys/kernel/debug/mtkfb; cat /sys/kernel/debug/mtkfb | grep gce_rd"
[ 417.152740][mtkfb_dbg] gce_rd: 0x11e50004
[ 417.152746]display_debug_cmd gce_rd: 0x11e50004
[ 417.153133][reg_dbg] gce_rd: addr(0x11e50004) = 0x80
```

0x11e50004 bit9 为 DEM 开关的寄存器，默认为 off，设 1 为开启：

```
adb shell "echo gce_wr:0x11e50004,0x280,0xffffffff > /sys/kernel/debug/mtkfb"
```

12.3.6 1.5.5 & 1.5.6 HS Skew Calibration Burst

当 datartate > 1.5Gbps 时，Spec 规定 TX 需要向 RX 发送 Skew Pattern，RX 收到后，可以用来校正 PCB Layout 等导致的 Data 和 Clock 的偏移，为高速信号接收提供保障。Skew Pattern 与 Normal 信号的差异如下图：



Skew Pattern 有两种，Initial 的方式和 Periodic 的方式，Periodic 的方式在 Spec 中是可选的，SoC 不支持这种方式，只支持 1.5.5 Initial 的方式。

Support	1.5.5 Initial HS Skew Calibration Burst(TSKEWCAL-SYNC)	2.93778 UI	-256E+01 %	15.75000 UI <= VALUE <= 16.25000 UI
	1.5.5 Initial HS Skew Calibration Burst(TSKEWCAL)	1 UI	-100.0 %	VALUE >= 32.768 kUI
	1.5.6 Periodic HS Skew Calibration Burst(TSKEWCAL-SYNC)	3.04041 UI	-254E+01 %	15.75000 UI <= VALUE <= 16.25000 UI
Not Support	1.5.6 Periodic HS Skew Calibration Burst(TSKEWCAL)	900 mUI	-100.0 %	VALUE >= 4.096 kUI

12.3.6.1 1.5.5 Initial HS Skew 测试

Initial 的方式是指在 DSI Normal 的信号之前发送 Skew Pattern，只会发送一次，不会周期重复。如果要测试这个信号，需要在亮屏之前正确的抓住 Timing。当屏幕亮的时候信号已经发送完毕，是测不到的。这对信号的测试有难度，不建议测试。

12.3.6.2 SW Deskew Enable

Code Flow 中默认大于 1.5G datarate 会开启，需要加串器也要打开 Deskew 的功能，才能实现此功能，完成信号的校准。加串器设定好后，收到 Pattern，并做完校准，会返回 IRQ，可以通过 IRQ 的状态确认。

```
co_device_module > drivers > gpu > drm > mediatek > mediatek_v2 > C mtk_dsi.c > mtk_output_dsi_enable
5221 static void mtk_output_dsi_enable(struct mtk_dsi *dsi,
5367
5368     if (get_ap_data_rate() > bdg_rx_v12)
5369         DSI_MIPI_deskew(dsi);
5370 }
5371 if (mtk_dsi_default_rate(dsi) > 1500) // data rate > 1.5Gbps, do skew calibration
5372     DSI_MIPI_deskew(dsi); //DSI deskew pattern enable
5373
```

13 LCM Timing 介绍

LCM Timing 包括 Power On/Off Sequence、Interface Timing 和 Display Timing 三部分，这是 LCM 能否正常工作的灵魂三部曲，缺一不可，所以这部分都会明确的写在 LCM 规格中。当我们要 Bringup 的时候首先就是要明确这三部分，碰到问题时也是从这三方面着手分析。另外车机的屏幕显示中还要包含 SerDes 的部分，加串器将 SoC 的 Interface 的显示数据转为串行的数据传输，解串器再将串行数据转为屏幕的 Interface 的数据，再送给屏幕 T-con 显示。车机的屏幕模组都将解串器包含进来，这样屏幕的显示系统相对于手机会更复杂。

13.1 Power On/Off Sequence

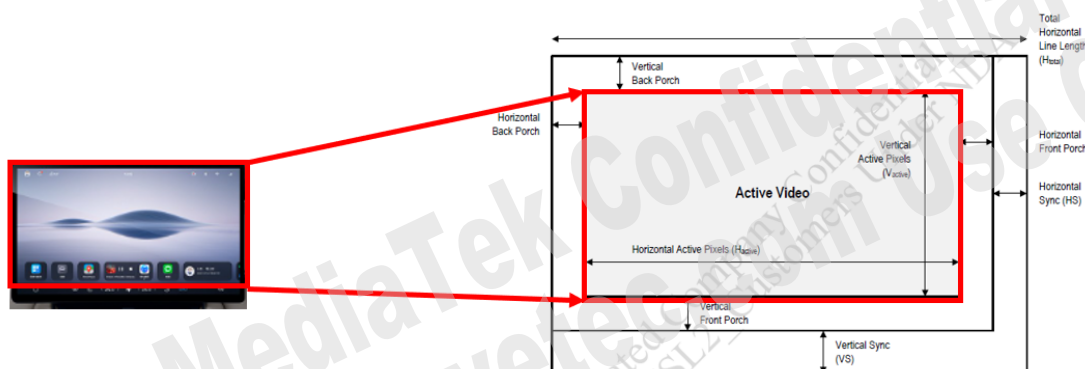
上下电时序->Power 和 Reset 部分的控制，Driver 中 Power On/Off 的控制要遵循 Spec 中的时序来设定。这部分与 HW 的设计强相关，需要结合原理图来实现。同样 SerDes 的 Power 和初始化的顺序，也会影响屏幕的显示，这个是车机也需要考虑的部分。

13.2 Interface Timing

LCM 的 Interface 有很多种 DSI, LVDS, eDP 等，每种 Interface 都有规格。车机中常用的有 LVDS 和 eDP，其中分辨率比较大的，例如 2.5K 屏幕或者以上基本都是采用 eDP 的 Interface。

13.3 Display Timing

虽然 Interface 有很多种，但是传输的 Display 信号都是基本都是包括 Active 和 Blanking 两部分。Active 信号是可视区，包括 Hactive 和 Vactive，Blanking 信号是非可视区，包括 H blanking 和 V blanking，Blanking 部分又分 sync, Front Porch 和 Back Porch。这部分在 LCM Spec 中也有明确的说明，我们需要按照 Spec 的设定给 LCM 的 T-CON 输出正确的 Timing，LCM 才会显示正常。



下图中是 Dual LVDS 1920x1080@60 的屏幕的 Timing Spec，Dual LVDS 的屏幕 Spec 中水平方向要 x2，这部分 Timing 对应的就是 panel-mode-setting 里面的设定。请与屏厂确认保证这部分参数的正确。

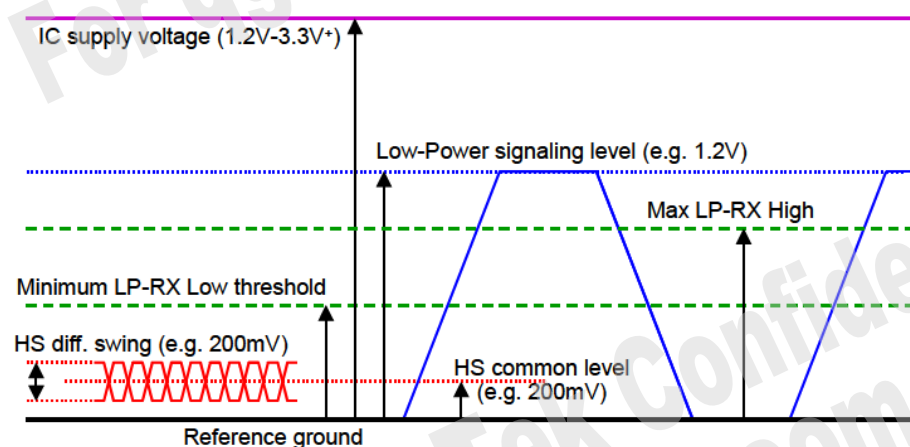
Parameter	Symbol	Min.	Typ.	Max.	Unit	Remark
Clock frequency	RxFCLK	66.05	68.18	70.31	MHz	
Horizontal Display Area	Thd	960			DCLK	
HS Pulse Width	Thpw	10	20	30	DCLK	
HS back porch	Thbp	20	40	60	DCLK	
HS front porch	Thfp	10	20	30	DCLK	
1 horizontal line	Th	990	1020	1050		
Vertical Display Area	Tvd	1080			DCLK	
VS Pulse Width	Tvpw	2	2	2	DCLK	
VS back porch	Tvbp	10	10	10	DCLK	
VS front porch	Tvfp	22	24	26	DCLK	
1 vertical field	Tv	1112	1114	1116		
Frame rate	Fframe	-	60	-	HZ	

14 DSI Interface 介绍

DSI (Display Serial Interface) 是 MIPI 联盟定制的显示接口规范。协议包含应用层，协议层，链路层和物理层。传输模式分为 HS->High Speed 和 LP->Low Power。

14.1 HS

D-PHY HS Swing 200mV, Video 的数据都是通过 HS 传输, 最大传输速度与 D-PHY 版本有关。

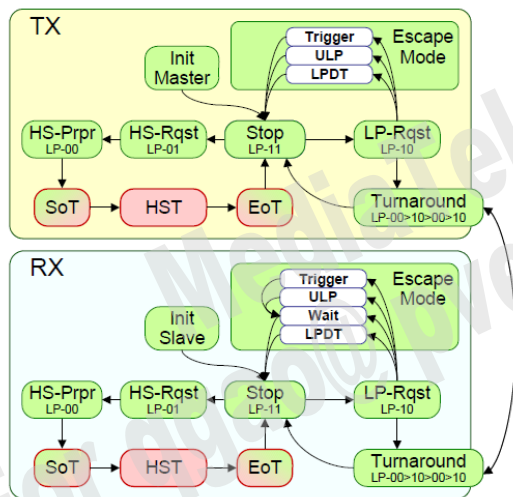


14.2 LP

信号幅度为 1.2V, 最大传输速度 80M, 一般用于发送 Command 和状态切换。

14.3 HS & LP 状态切换图

DSI TX 和 RX 都是从 LP11 开始, 然后通过 LP01, LP00 SOT 进入 HS 传输状态。

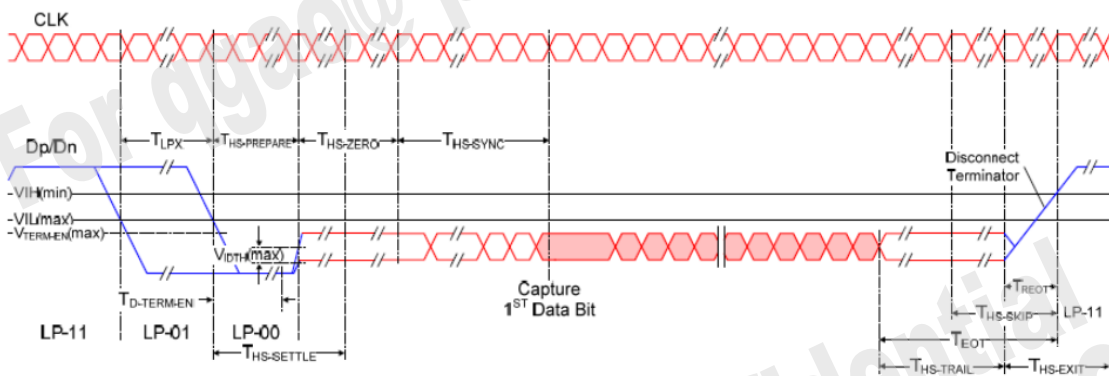


14.4 DSI MIPI D-PHY Timing Spec

MIPI 的 TX 和 RX 是没有握手协议的，而是要求 TX 和 RX 严格按照 Spec 的 Timing 来发送数据和接受数据。Spec 规定了从 LP ->HS 和 HS->LP 具体 Timing，TX 和 RX 的 Timing 满足 Spec 由为重要，两边如果不匹配，就会导致数据异常。

14.4.1 Data Lane Spec

HS-PREPARE 和 HS-TRAIL 等参数是经常需要调整的参数，需要明白下图中各参数的含义。Spec 中有规定每个参数的含义和时间要求。



Parameter	Description	Min	Typ	Max	Unit	Notes
$T_{HS-PREPARE}$	Time to drive LP-00 to prepare for HS transmission	$40\text{ ns} + 4*UI$		$85\text{ ns} + 6*UI$	ns	
$T_{HS-PREPARE} + T_{HS-ZERO}$	$T_{HS-PREPARE}$ + Time to drive HS-0 before the Sync sequence	$145\text{ ns} + 10*UI$			ns	
$T_{HS-SKIP}$	Time-out at RX to ignore transition period of EoT	40		$55\text{ ns} + 4*UI$	ns	
$T_{HS-TRAIL}$	Time to drive flipped differential state after last payload data bit of a HS transmission burst	$\max(n*8*UI, 60\text{ ns} + n*4*UI)$			ns	2, 3
T_{INIT}	Initialization period (PHY might calibrate)	100			μs	
T_{LPX}	Length of any Low-Power state period	50			ns	4
Ratio T_{LPX}	Ratio of $T_{LPX(MASTER)}/T_{LPX(SLAVE)}$ between Master and Slave side	2/3		3/2		
T_{TA-GET}	Time to drive LP-00 by new TX	$5*T_{LPX}$			ns	
T_{TA-GO}	Time to drive LP-00 after Turnaround Request	$4*T_{LPX}$			ns	
$T_{TA-SURE}$	Time-out before new TX side starts driving	T_{LPX}		$2*T_{LPX}$	ns	
T_{WAKEUP}	Recovery time from Ultra-Low Power State	1			ms	

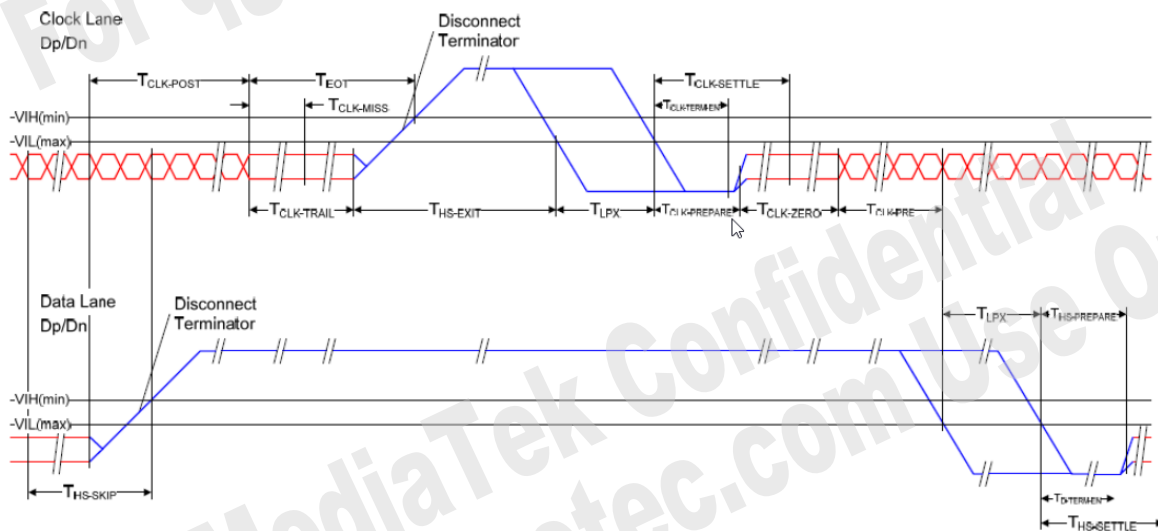
Notes

- The minimum value depends on the bit rate. Implementations should ensure proper operation for all the supported bit rates.
- If $a > b$ then $\max(a, b) = a$ otherwise $\max(a, b) = b$
- Where $n = 1$ for Forward-direction HS mode and $n = 4$ for Reverse-direction HS mode
- T_{LPX} is an internal state machine timing reference. Externally measured values may differ slightly from the specified values due to asymmetric setup and hold times.

UI Unit Interval, equal to the duration of any HS state on the Clock Lane (UI 即 MIPI 的 CLK lane 的高速时钟 High-Speed clock cycles 周期的一半, 这是因为 MIPI 采用 DDR 时钟传输方式)

14.4.2 Clock Lane Spec

MIPI 协议中有规定 Clock Lane 可以不需要回到 LP, TX 可以直接设定为 Continue Mode, 那么下图中 Clock 的 Timing 不是必须的。



Parameter	Description	Min	Typ	Max	Unit	Notes
$T_{CLK-MISS}$	Detection time that the clock has stopped toggling			60	ns	1
$T_{CLK-POST}$	Time that the transmitter shall continue sending HS clock after the last associated Data Lane has transitioned to LP mode	$60\text{ ns} + 52*UI$			ns	
$T_{CLK-PRE}$	Time that the HS clock shall be driven prior to any associated Data Lane beginning the transition from LP to HS mode	8			UI	
$T_{CLK-PREPARE}$	Time to drive LP-00 to prepare for HS clock transmission	38		95	ns	
$T_{CLK-TERM-EN}$	Time to enable Clock Lane receiver line termination measure from when Dn crosses V_{IL_MAX}	Time for Dn to reach $V_{TERM-EN}$		38	ns	
$T_{CLK-TRAIL}$	Time to drive HS differential state after last payload clock bit of a HS transmission burst	60			ns	
$T_{CLK-PREPARE} + T_{CLK-ZERO}$	$T_{CLK-PREPARE}$ + time for lead HS-0 drive period before starting Clock	300			ns	
$T_{D-TERM-EN}$	Time to enable Data Lane receiver line termination measured from when Dn crosses V_{IL_MAX}	Time for Dn to reach $V_{TERM-EN}$		$35\text{ ns} + 4*UI$		
T_{EOT}	Time from start of $T_{HS-TRAIL}$ or $T_{CLK-TRAIL}$ period to start of LP-11 state			$105\text{ ns} + n*12*UI$		3
$T_{HS-EXIT}$	Time to drive LP-11 after HS burst	100			ns	

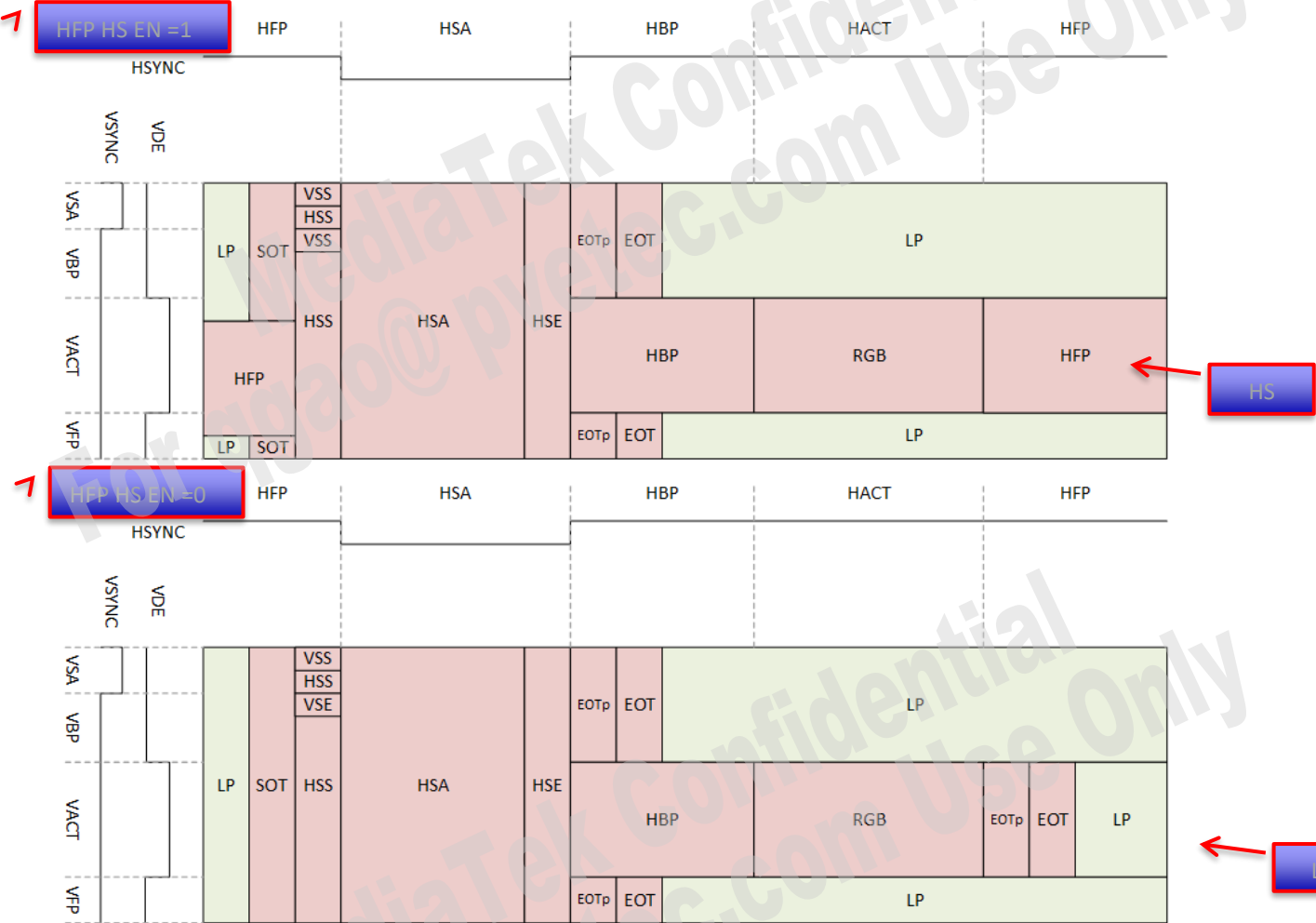
14.5 Video Mode 介绍

Video Mode 分为 Sync Plus, Sync-event, Burst 三种 Mode, 对于 MT8668, 三种 Mode 都可以支持, 对于 RX 端 MIPI 规定支持一种 Mode 就可以。目前最常使用的是 Sync Plus Mode。

14.5.1 HFP Keep HS

Sync Plus Mode 从 DPI 信号而来, 有完整的 H 和 V 同步信号, 数据包摆放如下图所示。绿色为 LP 的部分, 红色为 HS 的部分。MT8668 可以支持 Active 数据每个 Frame 回 LP, HFP 的时候 keep HS; HFP Keep HS mode 可以让输出的 HFP 更准确, 如果 Panel 或者 Bridge 要精确的 HFP, 就建议使用这个 Mode。否则当 HFP HS EN = 0 时, HFP 会多出来回 LP 的 Data Cycle, 实际输出的 HFP 会偏大, 同时 HFP 会有最小值的要求。两个 Mode 对比, 如下图所示。

Driver 中对应的参数为 `vdo_per_frame_lp_enable`, 设为 1 为 HFP_HS_EN=1。



14.5.2 DSI_Frame_Rate_Cal_for_Customer 介绍

这个 excel 表格可以计算出比较精确 FPS 所对应的 DSI datarate，同时也可以确认 Panel Timing 是否可以满足要求。

输入参数	不可动	可自行填入	不需要输入参数								
1 DSI Frame Size			Vertical (unit: line)				Horizontal (unit: pixel)				
H_Total/2*lane_num	H_Total (pixel)	V_Total (line)	VACT	VFP	VPW	VBP	HACT	HFP	HSA	HBP	
0	2520	1138	1080	18	30	10	2400	76	26	18	
Option							Real Horizontal (unit: pixel)				
3*H_Total/2*lane_num	DSI_BLLP_WC	Video Mode Keep HS Mode	Command Mode Keep HS Mode	Null Packet Word Count (Command Mode)	PROGSEQ_SKIP_EN	PHY Timing Post Processing	Display Pipe	HFP	HSA	HBP	
0	10	Y	HS mode w/ Null Packet	13	0	1	1	76.00	26.00	18.00	

0: D-PHY 1: C-PHY	D-PHY: 1~4 C-PHY: 1~3	Compression Ratio	24: RGB888 30: RGB101010	range : 0~3	Bit Rate (D-PHY) Symbol Rate (C-PHY)	D-PHY: 1/8 bit rate C-PHY: 1/7 symbol rate
C/D-PHY	lane_num	DSC	PixelBit (bit/pixel)	mode selector	Ideal Bit_Freq 3	DSI_CLK (MHz,Byte)
0	4	1.0000	24	1	1040.0000	130.000
PHY Timing Spec. (unit: UI)		Unit: Frame per Second(FPS)	Line Time Unit: us	Frame Time Unit: ms	Unit: MHz	
t3-prebegin	t3-post	Real Frame Rate (FPS) 3	Real LineTime	Real FrameTime	Disp Clock	VFP Time
0	0	60.4421	14.538	16.545	181.5873016	261.684

步骤:

1. 填入屏的 H Timing & V Timing, Video Mode, Lane Num 等参数。
2. 根据所需要帧率计算 Ideal Bit_Freq, 这个参数对于 D-PHY 就是 datarate。
3. 微调 datarate 得到想要的帧率, datarate 可以精确到 kHz, Driver 中需要填写 data_rate_khz 参数才会生效。
4. 如果要 timing 比较精准建议打开 Video Mode keep HS MODE, Htotal 要可以被 2 * lane_number 整除。

在做完第 3 步调整 Ideal Bit_Freq 后得到准确的帧率, 需要满足 4 的要求, 如果只有 Htotal 不满足, 建议 HFP 参数设定微调以满足 (屏幕一般都是 DE Mode, 一般可以水平方向微调, 同时新设定的 HFP 参数, 建议知会屏厂 double confirm) 。

另外第 4 步中提到的 Video Mode Keep HS MODE 就是指 vdo_per_frame_lp_enable, 可以参考章节 14.5.1。

Htotal 不满足的时候可以看到实际输出的 HFP 会有小数的情况, 这样可能会导致屏幕闪烁或者锯齿。

15 DSI 波形量测

波形的量测是常用的 Debug 手段。

15.1 如何测量帧率

图 1 中是一个每 Line 都回 LP 的 Data 的波形，需要将时间轴拉大一些找到 V Blanking 区域，按照下图测量 1 Frame 的时间，60fps 1 Frame 的时间大概为 16.6ms。图 2 中是将 V Blanking 区域放大的波形，可以放大确认抓的信号是否正确。

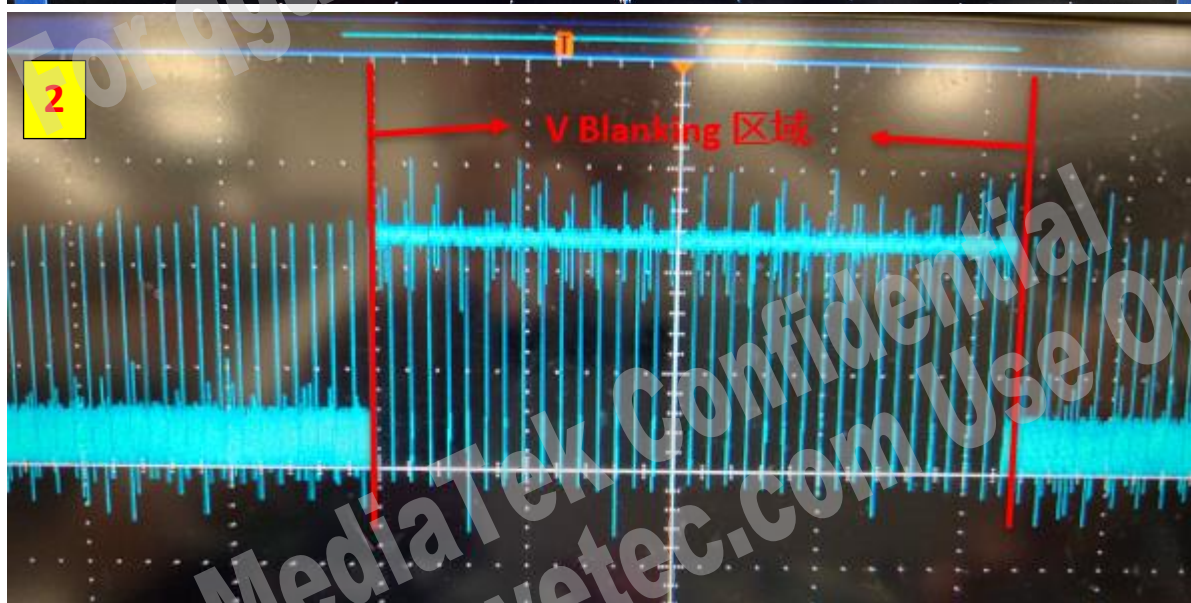
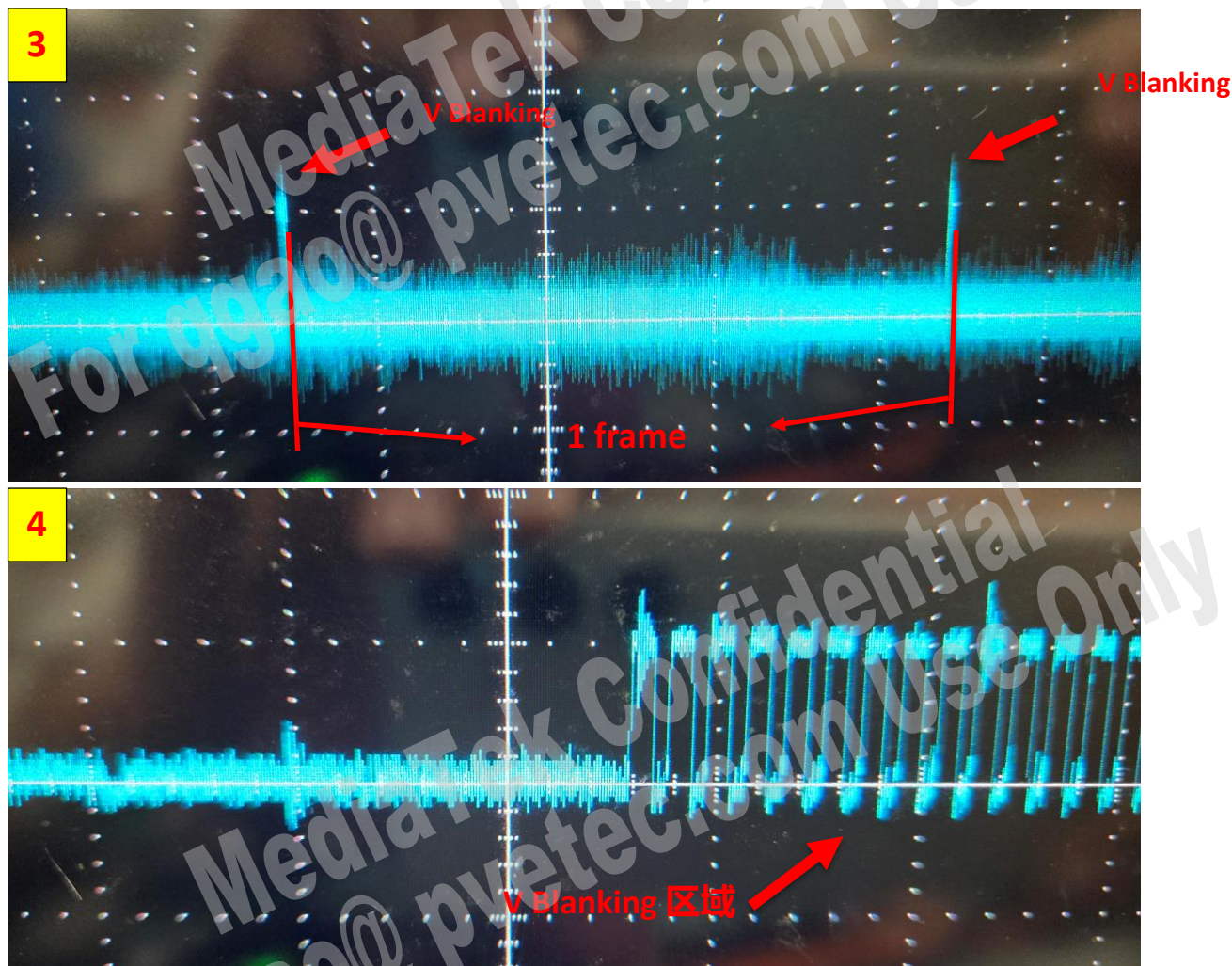
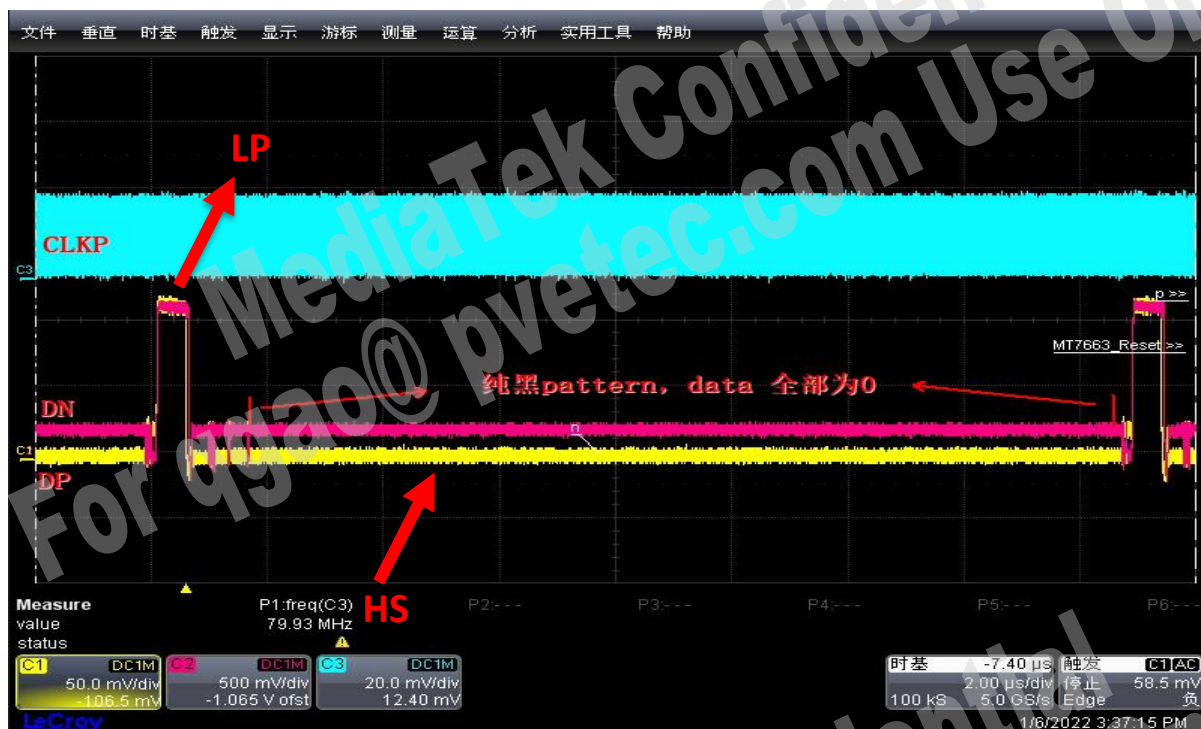


图 3 是开启 Video 每个 frame 回一次 LP，可以看到只有在 Blanking 区域有 LP 波形，其它的时间都是 Keep HS。按照下图测量 1 frame 的时间，60fps 1 frame 的时间大概为 16.6ms。图 4 是 V blanking 展开的图形，供参考。



15.2 如何测量信号

下图是 1 Line 的 Data & clock 的波形，DP & DN 可以看到 LP 和 HS 部分的波形。其中 Data 上 LP 为 1.2V，HS 为 200mV，Clock P 的幅度为 200mV。可以确认信号的电压等是否有异常。



LP 部分放大，可以看到 LP 进入 HS 的时序，再放大可以测量 HS prepare 等参数。



16 DSI 屏幕评估

在未知屏幕的具体 Timing，只知道 resolution 的情况下，如何粗估屏幕带宽？可以按照如下公式计算 datarate 看是否满足 Spec：

1. DPHY Un-compression

$$\text{datarate} = \frac{Hact \times Vact \times fps \times blanking\ ratio \times bpp}{lane_num}$$

2. DPHY Compression

$$\text{datarate} = \frac{Hact \times Vact \times fps \times compression\ ratio \times bpp + Hact \times Vact \times fps \times (blanking\ ratio - 1) \times bpp}{lane_num}$$

3. CPHY Un-compression

$$\text{datarate} = \frac{Hact \times Vact \times fps \times blanking\ ratio \times bpp \times \frac{7}{16}}{trio_num}$$

4. CPHY Compression

$$\text{datarate} = \frac{Hact \times Vact \times fps \times compression\ ratio \times bpp \times 7/16 + Hact \times Vact \times fps \times (blanking\ ratio - 1) \times bpp \times 7/16}{trio_num}$$

$blanking\ ratio = 1.25$,

RGB888:

DSC compression ratio = 1/3;

RGB101010:

DSC compression ratio = 1/3.75;

如果计算出的结果比较接近 Spec，建议拿具体的 Panel Timing 来评估。

粗估带宽满足后，在拿到 Panel 的 Timing 后还需要确认 V Blanking 是否满足，具体参考[章节 14.5.2](#)。

在带宽和 V Blanking 满足后，最后需要给 Display 做 Display Path 等评估，才算完整的评估。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿赔偿责任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。